arXiv:2608.27370v2 [cs.CL] 3 Sep 2026

PURO-2B: *Poor* Lab's Qwen2-1.5B Trained on RTX 5090 within $5090

Kairong Luo[1]*, Jiarui Cui[1]*, Yaorui Yin[1]*, Shengqi Chen[1]*, Yiming Yang[1], Linxiang Gao[1], Yanmohan Wang[1], Chengxia Li[1], Mingzhe Zhang[1], Kaifeng Lyu[1†], Wenguang Chen[1,2†]
[1]Tsinghua University [2]Pengcheng Laboratory
luokr24@mails.tsinghua.edu.cn {klyu,cwg}@tsinghua.edu.cn

Abstract

Language model pretraining has become almost synonymous with prohibitive cost, placing it out of reach for much of the academic and open-source communities. Although strong open-source efforts already exist, including open-weight models and open-source training recipes, a *cost-efficient, hardware-accessible, and open-source* pretraining recipe has long been missing. Even at a small scale, training Llama-3.2-3B costs over $1.5M, and reproducing SmolLM3-3B needs over $700K. In this report, we present an open pretraining recipe designed to lower this barrier. Using this recipe, we train a collection of PURO-2B (普罗-2B) models from scratch on up to 1.4 trillion tokens with FP8 precision on consumer-grade RTX 5090 GPUs[1]. The models in the collection differ in token budgets and selected recipe variants. Our best model is trained at a compute cost of less than $6.9K and approaches Qwen2.5-1.5B performance under our evaluation protocol. This cost efficiency is enabled by a combination of approaches, including hardware selection, low-precision training, hyperball optimization, curriculum model averaging, and the data recipe. Beyond the recipe itself, we provide two additional results. First, across the PURO-2B collection, we derive a *Puro Cost Scaling Law* that relates training cost to average model performance; the fitted law suggests that about $4.4K, *less than $5,090*, is sufficient to reach the performance of Qwen2-1.5B. Second, as an end-to-end case study, we examine how pretraining data curricula shape downstream performance after post-training. Such controlled studies are enabled by having access to the full pretraining pipeline rather than model weights alone. We release the full training recipe for PURO-2B, including data, code, and model weights under Apache 2.0 at `https://huggingface.co/collections/thu-pacman/puro-2b`.

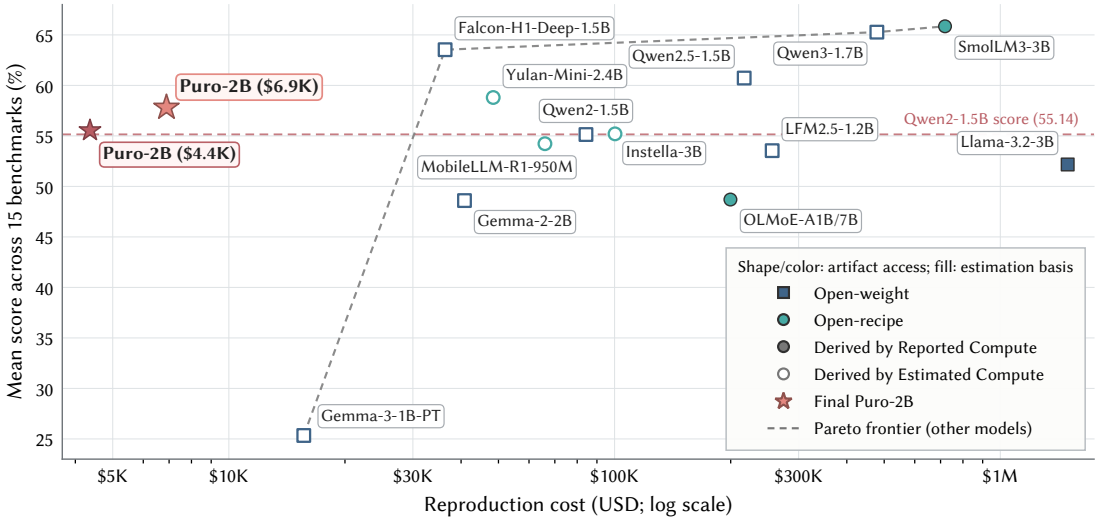


Figure 1: Model performance versus reproduction cost under the accounting protocol in Sections 2.2 and 4.3.2. Performance is measured by the average scores over the 15 mathematics, code, reasoning, and knowledge benchmarks in Tables 4 and 5. The cost summary of two PURO-2B runs can be found in Table 1.

*Equal core contribution.
†Corresponding authors.
[1]We sincerely thank Yanfu Investments for generously providing GPU computing resources to support this research.

Contents

1 Introduction **4**

2 Overview **6**

2.1 Pipeline at a Glance . . . 6

2.2 Reproduction-Cost Boundary . . . 8

3 Training Recipe **8**

3.1 Training Infrastructure . . . 8

3.1.1 RTX 5090 as a Cost-Effective GPU Choice . . . 8

3.1.2 Hardware Setup and Tweaks . . . 9

3.1.3 Efficient Training System . . . 10

3.2 FP8 Mixed-Precision Training . . . 11

3.3 Hyperball Optimization . . . 11

3.3.1 Effective Learning Rate . . . 12

3.3.2 Learning Rate Schedule Design . . . 14

3.4 Curriculum Model Averaging . . . 15

3.5 Post-Training Recipe . . . 16

3.6 Data Recipe . . . 17

3.6.1 How Do We Select a Good Data Recipe? . . . 17

3.6.2 How Do We Preprocess Data and Reproduce the Shards? . . . 19

4 Evaluation **20**

4.1 Evaluation Setup . . . 20

4.1.1 Model Selection . . . 20

4.1.2 Benchmarks . . . 20

4.1.3 Implementation Details . . . 21

4.2 Resulting Performance . . . 21

4.3 Cost Estimation . . . 22

4.3.1 Cost-Saving Factors . . . 22

4.3.2 Cross-Model Reproduction Cost . . . 24

4.4 Post-Training . . . 24

5 Related Works **25**

5.1 Open-Recipe Language Models . . . 25

5.2 Low-Cost Language Model Pretraining . . . 26

6 Conclusion and Future Direction **26**

7 Acknowledgments **27**

A Limitations **38**

B Cost Assumptions **38**

C Training Details **40**

D Scaling Ladder **42**

E Post-Training Details **45**

E.1 Shared Construction and Evaluation Protocol 45

E.2 GSM8K-Based SFT 46

E.3 Math&Code SFT with Replay 47

E.4 Tulu-3 Mixed-Domain SFT 48

E.5 Summary of the Three Comparisons 48

F LR Schedule Analysis **49**

F.1 Multi-Power Law for Effective Learning Rate Schedules 49

F.2 WSD Sweeps and Limited-Compute Schedule Estimation 51

F.3 A Prescribed Effective LR Can Induce a Hill-Like Scalar LR in Ordinary Muon 51

G Scaling Checkpoint Ledger **52**

H Curriculum and CMA **54**

H.1 Scalable Construction of Component-Local Curriculum Buckets 54

H.2 Phase Transition and UD Control 55

H.3 Constant-LR Continuation and Checkpoint Averaging 56

I Data Recipe **57**

1 Introduction

Scaling model parameters and training data has substantially improved the capabilities of large language models (LLMs) [42, 44]. However, the cost of pretraining limits the ability of academic and open-source researchers to study training behavior, reproduce complete pipelines, and test alternatives at meaningful scales. Lowering this barrier requires not only open model weights, but also reproducible data, software, infrastructure, training details, and transparent cost accounting.

Existing model releases span three levels of openness. *Closed models*, including proprietary families such as GPT, Claude, and Gemini [71, 30], expose capabilities through hosted services and technical reports but do not release model weights or the artifacts required to inspect pretraining. *Open-weight models*, including the Qwen, Gemma, and Llama families [103, 104, 102, 81, 31, 60], release checkpoints that enable local evaluation, deployment, and post-training, but generally withhold the exact pretraining data, sample order, and complete training state. *Open-recipe models* go further by releasing reconstructible data mixtures, training code, configurations, and checkpoints. Projects such as OLMo and OLMoE, SmolLM, YuLan, Marin, and Instella therefore make the pretraining process itself available for scientific study [95, 64, 2, 7, 108, 52, 37].

Greater openness, however, does not by itself make pretraining affordable to reproduce. Even at a small scale, the compute cost of pretraining can be prohibitive. Under the rental-equivalent accounting in Figure 1 and Section B, training Llama3.2-3B costs over $1.5M in our estimation. For open-recipe models, reproducing OLMoE-1B-7B would cost $200K, and for SmolLM3-3B, the estimated reproduction cost rises to $719K. These cost budgets are derived from reported or estimated compute of these models. Therefore, even transparent open-recipe releases can remain *open to the world but out of reach for many small and resource-constrained research labs — the poor labs*, leaving a practical gap between reproducibility and accessibility.

To bridge this gap, we present a systematic reproducibility recipe covering system, algorithm, and data design for training a collection of **PURO-2B (普罗-2B)** checkpoints[2]. The recipe targets low-cost, accessible infrastructure for dense billion-parameter/trillion-token pretraining and produces a 2B model with performance matched against Qwen2-1.5B and Qwen2.5-1.5B under our protocol. These checkpoints share the same 2B-parameter architecture but differ in training budget and recipe variants. Training uses predominantly open-source datasets and runs on a cluster of consumer-grade RTX 5090 GPUs. Figure 1 compares counterpart models under a common cost-accounting protocol. PURO-2B attains competitive performance at the lowest reproduction cost among the plotted models under the stated accounting protocol, placing it beyond the baseline cost–performance frontier. Our best checkpoint surpasses Qwen2-1.5B overall and approaches Qwen2.5-1.5B. Detailed accounting is discussed in Section 4.3 and Section B As shown in Table 1, the production schedule uses 438.8B Phase 1 tokens and 960.0B Phase 2 tokens. These stages total 1.4T scheduled tokens and 22,514 active-training GPU-hours, corresponding to a compute cost of about $6.9K and 17.6 elapsed days. The pipeline covers the data recipe, hardware infrastructure, supporting software, and training strategy.

We have released the following artifacts under the Apache 2.0 license unless otherwise noted separately; upstream data terms remain component-specific.

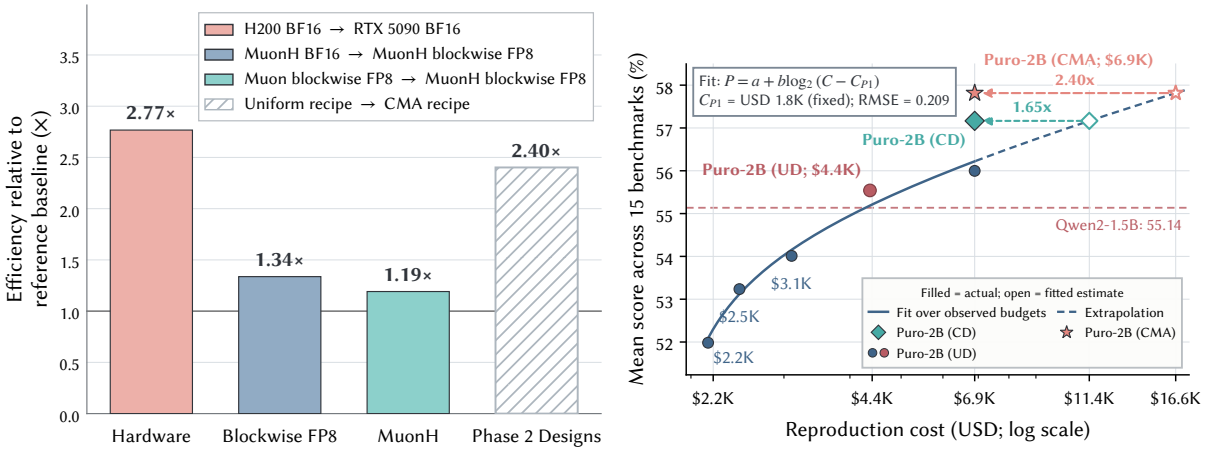
- Model weights: `https://huggingface.co/thu-pacman/Puro-2B-Base` and variants (10 different versions of checkpoints in total for better transparency and reproducibility)
- Data manifests and materialized components[3]: `https://huggingface.co/datasets/thu-pacman/Puro-2B`
- Training code: `https://github.com/thu-pacman/Puro-Megatron`
- Data-processing code (KAIYUAN-SPARK): `https://github.com/thu-pacman/Kaiyuan-Spark`

The recipe is built around five efficiency-oriented components spanning hardware, numerical precision, optimization, data ordering, and data selection.

1. We use **consumer-grade RTX 5090 GPUs** as an accessible training platform that provides a favorable cost-efficiency trade-off in our setting (Section 3.1).
2. Blockwise **FP8 training** reduces per-token execution time while maintaining comparable model quality to bfloat16 (BF16) training (Section 3.2).
3. We apply the **MuonH optimizer**, which extends the Muon optimizer with hyperball constraints on parameter weights and updates, together with a carefully designed learning rate schedule (Section 3.3).

[2]Unless otherwise noted, we will use PURO-2B to denote the best one in this collection.

[3]Different licenses may apply to different dataset components, some of which may not be permissive.



(a) Relative Cost-Efficiency Improvement.

(b) Puro Cost Scaling Law (scale-down)

Figure 2: **(a)** Relative cost-efficiency improvement for hardware, FP8 precision, MuonH, and various Phase 2 designs. The gains come from different sources but are all converted into equivalent savings in USD. Each bar uses its own baseline, so the numbers should not be directly multiplied together. **(b)** Puro Cost Scaling Law is fitted from Phase 2 training results with uniform data at different token budgets. UD denotes it using **u**niform data ordering with LR **d**ecay. The $4.4K UD checkpoint exceeds Qwen2-1.5B in average performance over the 15 benchmarks reported in Tables 4 and 5. At the $6.9K reproduction cost, using a data **c**urriculum with LR **d**ecay (CD) in Phase 2 training achieves 1.65× cost-efficiency gains over the uniform data recipe. Incorporating more joint designs (**c**urriculum **m**odel **a**verage, CMA) in Phase 2 yields our canonical Puro-2B, achieving cost-efficiency gains of 2.40× relative to the uniform scaling curve (Section 3.4). The details are given in Section 3.4 and fig. 13a. Filled markers denote actual reproduction costs, while hollow markers denote fitted uniform-equivalent costs. See Section 3.4 and Tables 18 and 19 for details.

4. **Curriculum Model Averaging (CMA)** organizes training over coarse-grained data chunks according to configured source-local preferences and averages selected checkpoints [56] (Section 3.4).
5. Proxy experiments provide empirical signals for dataset selection and mixture design under a large candidate data pool (Section 3.6).

Furthermore, we conduct targeted ablations over the first four design choices and report cost-efficiency estimates (Figure 2 and Section 4.3). Our systematic recipe covers the full efficiency stack of pretraining: data selection reduces the required token budget, MuonH and the CMA recipe improve token efficiency by extracting more capability from training tokens, FP8 improves hardware utilization by increasing computational throughput, and hardware selection reduces the cost per unit of computation. Together, these choices optimize different bottlenecks across the training pipeline rather than targeting a single part.

Importantly, these components are not independent optimizations combined in isolation; they form a co-designed training system with interactions across different layers of the pipeline. For example, the Blackwell architecture of RTX 5090 provides the hardware foundation for efficient FP8 training. Both MuonH and CMA involve careful learning rate schedule design, and data curriculum relies on preprocessing decisions that determine the ordering of source-local data chunks. Therefore, the effectiveness of our recipe comes from the coordinated design of data, optimization, algorithmic, and system-level choices as an integrated approach to affordable pretraining.

In addition, we formulate the *Puro Cost Scaling Law*, a recipe-specific scale-down relationship between rental-equivalent training budget and model capability. As illustrated in Figure 2b, the law is designed for scale-down scenarios, serving users with limited compute and time budgets. There are three Phase 2 recipes used: **U**niform ordering with LR **D**ecay (UD), data **C**urriculum with LR **D**ecay (CD), and CMA. Their detailed ordering, learning rate schedule, and averaging choices are defined in Section 3.4. The scale-down recipe uses the UD recipe. We resume training from the same Phase 1 checkpoints and adopt different Phase 2 token budgets. We then evaluate the resulting checkpoints and fit the trend of model performance as a function of total training cost. This scaling curve provides a reference for estimating the model capability achievable under a limited budget with our recipe. Interestingly, the Puro Cost Scaling Law suggests that our recipe can cross the Qwen2-1.5B performance aggregate at a cost of USD 4.4K. Its accounting assumptions and recipe-level interpretation are detailed in Section 4.3. Moreover, the CD and CMA adoption can provide a further speedup compared to the UD scaling trend.

Beyond pretraining, the following post-training reveals that the difference between the CMA and UD Phase 2 recipes persists after matched supervised adaptation. In GSM8K-based SFT and Math&Code SFT with replay, CMA-based SFT yields higher GSM8K accuracy in repeated runs. In the Tulu-3 mixed-domain SFT setting, CMA-based SFT also improves the comprehensive capabilities and instruction following. Our Puro-2B recipes make these kinds of end-to-end comparisons possible at a meaningful scale and within an affordable budget.

In summary, our contributions are:

1. We provide a cost-effective, from-scratch pretraining recipe and train a Puro-2B model collection. The model in this collection exceeds Qwen2-1.5B at about $4.4K, and its best model checkpoint approaches Qwen2.5-1.5B with $6.9K. More broadly, *our main contribution is to show that affordable, open pretraining is practical today*; we expect future efforts to push this frontier toward even lower cost and higher efficiency.
2. We evaluate the main recipe choices: RTX 5090 infrastructure, blockwise FP8, MuonH, and curriculum model averaging with supporting ablation experiments. Moreover, we formulate the Puro Cost Scaling Law for the recipe's scale-down budget capability trend.
3. We release the datasets, model weights, intermediate checkpoints, training configurations, and implementation provided to reproduce and inspect this pretraining recipe, along with a post-training case study to show how this recipe helps end-to-end comparison for one pretraining design.

2 Overview

Before discussing individual design choices in detail, we first provide an overview of the complete training recipe. The pipeline starts with constructing the training corpus from publicly accessible sources and selecting cost-efficient, widely accessible hardware. It then proceeds through two phases of pretraining, including our optimization and data-curriculum designs, followed by model averaging, post-training, and evaluation. We also define the cost-accounting protocol and its boundary to illustrate the reported training costs. The following sections then discuss the motivation, implementation, and evidence for each major design choice. The overall pipeline is illustrated in Figure 3.

2.1 Pipeline at a Glance

Table 1: We summarize two Puro-2B run cost accounts in Figure 1. Both runs share Phase 1 and use different Phase 2 recipe variants. The $4.4K run uses uniform ordering with LR decay (UD), while the canonical $6.9K run uses CMA (Section 3.4). We use 24 GPUs for Phase 1 and extend our compute resources to 96 GPUs in Phase 2. We compute the cost from GPU-hours and the unit compute cost specified in Section 2.2.

Checkpoint	Phase 1 (shared)				Phase 2				Total	
	GPUs	Tokens	Wall time	Cost	GPUs	Tokens	Wall time	Cost	Active GPU-h	Cost
Puro-2B (UD; $4.4K)	24	438.84B	10.43d	$1.84K	96	480.00B	3.58d	$2.53K	14,262	$4.37K
Puro-2B (CMA; $6.9K canonical)	24	438.84B	10.43d	$1.84K	96	959.99B	7.16d	$5.05K	22,514	$6.89K

Model architecture and pretraining system (Sections 3.1 and 3.2). We train a dense decoder-only Transformer from scratch through two consecutive pretraining phases. The model adopts the Qwen3-1.7B architectural configuration, but unties the input embedding matrix from the output language-model head. This yields approximately 2B parameters in total. The Qwen3 architecture is familiar to the community and broadly supported. The model scale is also compatible with our replication budget. Our training is implemented in Megatron Core and adapted to an RTX 5090 cluster. To exploit the GPUs' Blackwell FP8 support, the main Transformer linear-layer GEMMs use blockwise FP8, while numerically sensitive operations and persistent training states, including master weights and optimizer states, remain in BF16 or FP32 (Section 3.2). Phase 1 runs on 24 GPUs, and Phase 2 continues from the Phase 1 endpoint on 96 GPUs [4]. The distributed parallel configuration is adjusted for the larger GPU count. The detailed low-precision compute flow, RTX 5090 system adaptations, and phase-specific parallel configurations are provided in Sections 3.1.1 and 3.1.3.

Two-phase pretraining, curriculum, and optimization (Sections 3.3 and 3.4). Using the architecture and training system above, we pretrain PURO-2B in two phases. Phase 1 processes 438.8B tokens using the Phase 1

[4] Yanfu investments help extend our compute resources in Phase 2.

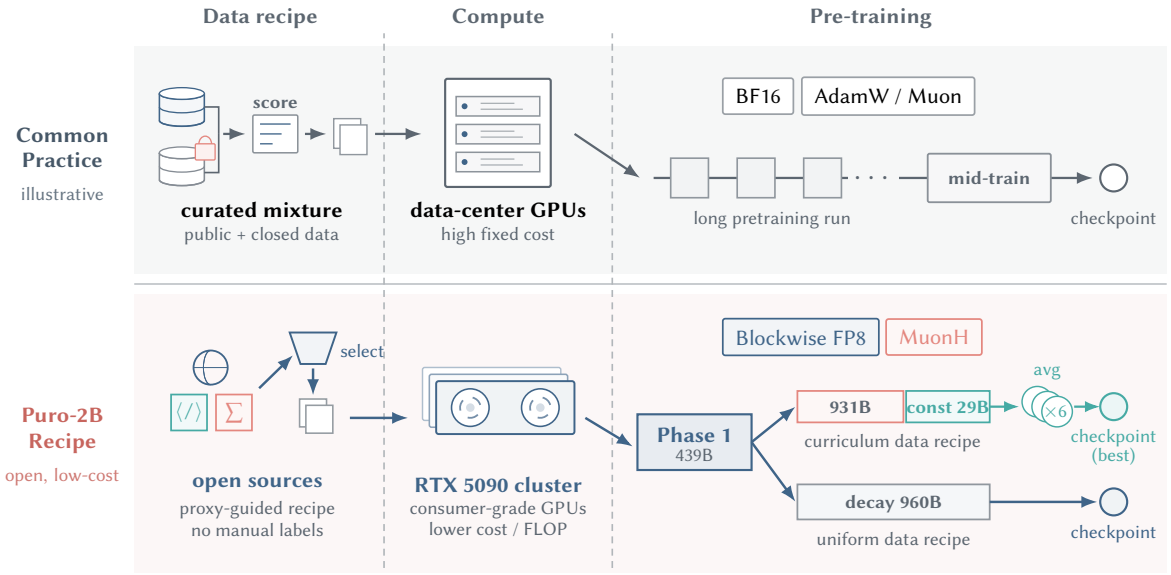


Figure 3: **Puro-2B pipeline for *poor*-lab design.** The upper row presents the work flow in common practice, and the lower row shows *our open, low-cost Puro-2B recipe.* We collect and select publicly accessible datasets to build a training dataset (Section 3.6), without elaborative data scoring and curation. We use an RTX 5090 cluster, which consists of consumer-grade GPUs and features higher cost-efficiency (Section 3.1), in contrast to costly data-center GPUs. Our pretraining process consists of two phases, and Phase 2 has two variants: uniform data recipe with LR decay (UD) and curriculum model averaging (CMA) with late constant-LR continuation and six-checkpoint averaging. Both variants use 960B tokens in Phase 2 (Section 3.4). As a comparison, common practice goes through a long pretraining phase and a shorter mid-training phase with LR annealing to get the final checkpoint. To further improve cost efficiency, we adopt blockwise FP8 training (Section 3.2) and Muon with Hyperball optimizer (Section 3.3) throughout Phase 1 and Phase 2.

mixture, while Phase 2 consumes 960.0B tokens in total. The canonical run applies a data curriculum over the Phase 2 data pool (Section 3.4), to present the more preferred portion of each scored source later in training. Rather than defining a global quality score across datasets, we primarily use the quality labels provided by source datasets. Within each dataset with usable score labels, examples are ordered from lower to higher score and partitioned into coarse chunks. Datasets without usable scores are instead randomly ordered and partitioned in the same way. Chunks from different sources are then aligned by their normalized within-source ranks, so that Phase 2 progresses from lower to higher configured ranks while approximately preserving the cross-component mixture. An illustration of this curriculum design is provided in Figure 7.

To preserve the influence of data presented late in the curriculum, we jointly design the late-stage learning rate schedule and checkpoint averaging, following the curriculum-aware averaging principle of Luo et al. [56], and use the averaged weights as the final model. Phase 2 therefore has recipe variants: *UD* uses a uniform data ordering with LR decay, *CD* uses the component-local data curriculum with LR decay, and *CMA* adds the late constant-LR continuation and six-checkpoint average. For comparison, the UD variant uses the same Phase 1 checkpoint and Phase 2 pool but globally reshuffles the Phase 2 data instead of following the curriculum ordering.

Across both phases, we use Hyperball optimization for selected approximately scale-invariant matrices. MuonH updates these matrices and projects each one back to its initial Frobenius radius after every step, while AdamW updates the remaining parameters (Section 3.3). Both parameter groups share a base LR schedule. MuonH group uses a 10 times base LR schedule as Hyperball weight LR. The base schedule follows a power schedule [86] in Phase 1 and continues from its terminal Phase 1 value with linear decay along the main Phase 2 trajectory. The late CMA continuation instead holds the base LR fixed at its resume-point value before checkpoint averaging (Section 3.4). Both phases use a sequence length of 4,096 tokens and a global batch size of 1,536 sequences. The optimizer configuration, learning rate multipliers, curriculum construction, transition schedule, checkpoint-averaging rule, and exact token accounting are reported in Sections 3.3 and 3.4, Section C, and Table 8.

Data acquisition, filtering, and mixture (Section 3.6). We construct the training corpus from publicly accessible datasets with licenses or terms that permit their use, enabling the corpus to be reproducibly reconstructed from the original sources. The corpus spans web text, code, and mathematics, and also includes synthetic ex-

amples. We first use our KAIYUAN-SPARK framework (Section 3.6.2) to deduplicate the large-scale web portion. Because the available data far exceeds our training budget, we must select source datasets, filter samples within each source, and combine the retained portions to improve overall data utility and match the desired capability profile. However, it is difficult to design a single sample-level quality score that is comparable across domains, apply it to every sample, and use it to determine which samples to retain. Instead, we run proxy experiments on candidate sources and representative data slices (Section 3.6.1). Specifically, we train controlled small-scale models on these sources or slices and evaluate the resulting models on downstream tasks. The resulting vector of benchmark scores defines the proxy profile of each source or slice, which guides source selection and mixture design (Section 3.6.1). For example, when stronger mathematical capability is desired, we can assign higher weights to datasets whose proxy profiles show stronger performance on mathematics benchmarks. Some open-source datasets also provide sample-level quality scores. Although these scores are not directly comparable across sources, they can support within-source filtering. For each such dataset, we sample proxy slices from different score ranges, compare their proxy profiles, and use the results to select a dataset-specific threshold for retaining high-scoring samples. The final domain-level recipe for the two pretraining phases is summarized in Table 3 and Figure 8, while Table 22 reports family-level retained token counts, selection modes, assigned phases, source identifiers, and license terms rather than a complete component-level manifest.

Evaluation and post-training (Sections 3.5 and 4). Our evaluation combines a broad pretrained-model comparison with controlled tests of downstream adaptation. We first evaluate PURO-2B against open-weight and reproducible open-recipe models of comparable scale on general reasoning, mathematics, and coding. Under the report's common budget accounting, PURO-2B surpasses Qwen2-1.5B and Gemma-2-2B while using less than one sixth of the stated training cost of comparable open recipes. We then use SFT as a probe of how the Phase 2 recipes transfer through post-training. Within each paired experiment, the CMA and UD initializations receive identical supervised data and optimization; mathematics checkpoints use one frozen generation and extraction protocol, while broad tasks use benchmark-appropriate prompts and scorers. The three settings are GSM8K-based SFT, Math&Code SFT with replay, and Tulu-3 mixed-domain SFT, with complete definitions and results in Section 3.5 and Section E.

2.2 Reproduction-Cost Boundary

Throughout this report, reproduction cost refers specifically to the compute cost of rerunning the finalized two-phase pretraining recipe once. The accounting boundary begins after the phase-specific training shards have been materialized and includes only the compute time of the Phase 1 and Phase 2 production runs. We report measured GPU-hours separately and convert them using the normalized RTX 5090 rental-equivalent rate specified in Section 4.3 and Section B. The ledger retains CNY(RMB) for traceability, while the main cost figures use the corresponding USD values. Both are reproducible cost proxies under the stated rate, not the authors' total cash expenditure.

We do not estimate costs outside these production pretraining runs. The headline therefore excludes data acquisition and preprocessing, proxy-model experiments, scaling and ablation studies, failed or exploratory runs, post-training, evaluation, checkpoint averaging, and research labor. It also excludes non-accelerator resources such as CPU processing, storage, and networking, as well as taxes, depreciation, and other ownership costs not represented by the rental-equivalent rate. These exclusions do not imply that the activities are free; they mean that the reported number should be interpreted as a narrowly defined marginal accelerator cost for reproducing the final pretraining run, rather than as the total cost of developing the model or reconstructing every artifact from scratch.

3 Training Recipe

We organize the training recipe as follows. We first describe the infrastructure design that supports our training setup. We then present the pretraining recipe, focusing on optimization and data curriculum. Next, we explain how the training dataset is constructed. Finally, we present a post-training case study to demonstrate how our recipe enables end-to-end approach comparison.

3.1 Training Infrastructure

3.1.1 RTX 5090 as a Cost-Effective GPU Choice

To make our recipe cost-efficient and the hardware accessible to the broader community, we made a non-standard infrastructure choice: we run all pretraining and post-training on a cluster equipped with consumer-grade GPUs

(RTX 5090), instead of renting conventional data-center-oriented accelerators. Although data-center GPUs are generally expected to be more stable and compute-efficient, our cost analysis showed that RTX 5090 is a better fit for this workload.

Table 2: Peak hardware capability and cost efficiency of candidate NVIDIA GPUs.

GPU Model	Peak tensor performance (TFLOPS)[a]			Memory capacity	Memory bandwidth	TDP	Intra-node P2P BW (Bi-Di)	Single-GPU price[b]	Peak compute/$ (EFLOP/USD)		
	BF16	FP8	NVFP4						BF16	FP8	FP4
A100 SXM4	312	N/A	N/A	80 GB	2 TB/s	400 W	600 GB/s	$1.79/h	0.63	N/A	N/A
H200 SXM5	989.5	1979 [c]	N/A	141 GB	4.8 TB/s	700 W	900 GB/s	$4.00/h	0.89	1.78	N/A
RTX PRO 6000	503.8	1007.6	2015.2	96 GB	1.8 TB/s	600 W	128 GB/s [d]	$1.89/h	0.96	1.92	3.85
RTX 5090	209.5	419	1676	32 GB	1.8 TB/s	575 W	128 GB/s [d]	**$0.31/h**	**2.43**	**4.87**	**19.46**

[a] All performance numbers are from official NVIDIA documentation / datasheets / blog posts [4, 22, 23, 24] and refer to dense matrix operations with FP32 accumulation.

[b] The prices do not include CPU / storage / network, as they are marginal and often free on GPU rental platforms. A100, H200, and RTX PRO 6000 prices were collected from a public GPU rental platform on 16 Aug 2026. RTX 5090 has no widely available public rental channel due to NVIDIA EULA restrictions; we estimate its effective price by amortizing our hardware and electricity costs over five years. An 8-GPU node costs approximately 12,000 CNY ($1763) per month for our setup. Some private compute providers confirmed similar rental pricing for RTX 5090.

[c] Hopper FP8 Tensor Cores use a reduced-precision path for nominal FP32 accumulation, with 22 effective precision bits [112], which may affect numerical accuracy.

[d] PCIe Gen5 × 16 provides approximately 64 GB/s of theoretical bandwidth per direction, or approximately 128 GB/s of aggregate bidirectional bandwidth. Driver tweaks needed to achieve this bandwidth are discussed in Section 3.1.2.

Table 2 shows the features and prices of different generations of NVIDIA GPUs. RTX 5090 has a clear gap in absolute peak throughput compared with data-center GPUs, especially because its Tensor Cores are artificially limited to half of actual peak performance when using FP32 accumulation. However, its much lower effective price gives substantially higher compute per dollar: its BF16 and FP8 cost efficiency is about 2.7× that of H200. Moreover, because policy and cost constraints made Blackwell data-center accelerators such as GB200 unavailable to us, RTX 5090 is also the most practical option with FP4 support.

RTX 5090 still has significant hardware limitations, including smaller memory capacity and the lack of NVLink for high-bandwidth GPU-to-GPU scaling. These drawbacks cannot be ignored. Nevertheless, our target model is small enough that, with the hardware configuration and parallel training setup described below, the RTX 5090 cluster reaches a mixed-precision effective MFU of approximately 73% under the stated precision-weighted peak convention. Section 3.1.3 defines the convention and measurement boundary; this is not a BF16-only full-scale result. Its cost advantage therefore remains valid for our setting.

3.1.2 Hardware Setup and Tweaks

Our training cluster consists of multiple x86 GPU servers. Each server is equipped with dual CPUs, eight RTX 5090 GPUs (four in each CPU socket), and sufficient host memory. The performance numbers in Section 3.1.1 are single-GPU metrics; for multi-GPU training, both intra-node P2P bandwidth and inter-node network bandwidth must be considered to sustain training throughput.

Intra-node bandwidth. Since RTX 5090 does not provide NVLink, the only intra-node interconnect is PCIe. NVIDIA disables PCIe P2P on non-data-center GPUs, which forces GPU communication traffic to be staged through host memory, also known as "ping-pong" transfer, which can substantially reduce communication efficiency. However, this restriction appears to be enforced by the NVIDIA GPU driver rather than the limitation of GPU hardware. We used a modified version of the open-source NVIDIA driver [1], along with the necessary platform configuration changes [14], including disabling IOMMU and PCIe ACS as well as adjusting NPS (NUMA Per Socket, also known as Sub-NUMA Clustering on some platforms) to enable GPU P2P on our servers.

On our hardware, enabling P2P improved one-way bandwidth from 31.5 GB/s to 56 GB/s. The former is bounded by half of the PCIe 5.0 ×16 bandwidth, i.e., 32 GB/s, because ping-pong transfer requires two host-memory copies; the latter approaches the 64 GB/s PCIe link limit. Bidirectional bandwidth increased from 32 GB/s to 111 GB/s, compared with theoretical limits of 128 GB/s. Communication latency decreased drastically from 14.3 μs to 0.4 μs. The eight-GPU `AllReduce` bandwidth over PCIe, measured as `busbw` reported by `nccl-tests`, increased from 14.75 GB/s to 27.34 GB/s. When the test was restricted to the four GPUs attached to a single CPU socket, the gain was larger, from 16.33 GB/s to 46.31 GB/s, about 2.8×. Adding a PLX PCIe switch between the GPUs and the root complex might further improve P2P bandwidth toward the link limit; this is a prospective topology change, not a configuration used in the reported run.

Inter-node bandwidth. To scale training beyond a single server, the inter-node network must provide bandwidth comparable to the intra-node communication path. We set up a 400 Gbps InfiniBand network among our servers, i.e., 100 GB/s bidirectional bandwidth: one dual-port Mellanox ConnectX-7 host card adapter (HCA) is installed in each server, with both ports connected to a 200 Gbps InfiniBand HDR switch. Half of the GPUs in the server could communicate with the HCA directly, while the other half must route through the inter-socket link to reach the HCA.

Even after PCIe P2P is enabled, non-data-center NVIDIA GPUs still do not support GPUDirect RDMA (shortened as GDR), another performance-critical feature for inter-node communication. GDR builds on P2P and allows a GPU to access remote GPU memory over RDMA NICs without staging through host memory. Through testing, we found that this restriction is also software-enforced: a minimal binary modification to the CUDA user-space driver can enable GDR on RTX 5090 [13]. Due to EULA constraints, we are not able to disclose the details in this report. When this configuration is unavailable, the same training recipe can be run with the stock NVIDIA driver, falling back to the standard communication path at lower inter-node bandwidth. In end-to-end tests, enabling GDR improved the 24-GPU (3 servers) `AllReduce` bandwidth, measured as `busbw`, from approximately 8.87 GB/s to 19.93 GB/s. We believe that if a more balanced network topology is used (e.g., one InfiniBand HCA on each socket), the performance gain could be even higher.

Caveats. These modifications should be used with caution. First, modifying drivers is unsupported by the vendor and must be done at the user's own risk. Second, these changes cannot exceed the physical PCIe bandwidth limit. Third, P2P and GDR should not be enabled blindly; the optimal setting depends on the actual hardware topology and platform capability. For example, if the PCIe root complex of a CPU has limited peer-switching capability (which is often the case on low-end models), heavy P2P traffic among many devices may become congested, and host-memory ping-pong transfer can be faster.

3.1.3 Efficient Training System

We build our training system on Megatron Core [88], specifically release v0.16.0 [68], together with its Transformer Engine dependency. This software stack provides native support for blockwise FP8 training and the Muon optimizer, making it a suitable basis for a model at our scale. Nevertheless, the combination of consumer GPUs, a mixed-precision training recipe, and our model architecture still requires careful system-level tuning.

Communication-aware parallelism. The limited communication bandwidth of RTX 5090 motivates us to use only parallel dimensions with modest communication demands. We combine data parallelism (DP), whose dominant collectives are gradient `ReduceScatter` and parameter `AllGather` around optimization, with pipeline parallelism (PP), which exchanges hidden-state tensors only at stage boundaries [66]. We do not use tensor parallelism (TP), which introduces frequent collectives within every Transformer layer. Compared with data-center-class systems, our communication disadvantage is substantially more pronounced within a node. We therefore depart from Megatron Core's default rank ordering and use a `pp-dp` ordering that places each PP group on topologically closer GPUs within a node, thereby reducing sensitivity to communication latency.

Appropriate micro-batch size. FP8 kernels complete their arithmetic more quickly while introducing additional quantization operations, so they become memory-bound more readily than BF16 kernels. This effect is amplified by the relatively small hidden dimension of our model. Increasing the micro-batch size (MBS) improves the arithmetic intensity and Tensor Core utilization, but an excessive MBS exhausts device memory: DP does not partition activations; and 1F1B pipeline schedule [66] helps control the number of simultaneously resident microbatches, but it does not eliminate the per-microbatch activation-memory growth caused by increasing MBS. We therefore extend Megatron-LM's analytical FLOP estimator to report the invocation shapes of GEMM and FlashAttention kernels. For each shape induced by our model configuration, we benchmark the corresponding Transformer Engine kernel and select the smallest MBS beyond which achieved FLOP/s no longer increases materially, analogous to locating the knee of a roofline curve.

Overcoming load imbalance. Although our model is relatively small, its vocabulary is large; consequently, the embedding and LM head account for a non-negligible fraction of both computation and memory. FP8 GEMMs accelerate the internal Transformer layers, further increasing their performance gap with the LM head. As a result, the LM head alone entails computation comparable to several Transformer layers, creating substantial pipeline imbalance. Guided by our extended FLOP estimator, which attributes theoretical work by layer, submodule, operator, and precision, we assign fewer Transformer layers to the stage containing the LM head. This balances pipeline computation at the cost of greater memory pressure on the first stage. Tensors assigned to Muon are preferably kept intact rather than sharded. We replace the zigzag placement policy of the layer-wise

distributed optimizer in this version of Megatron Core, which over-concentrates the embedding and LM-head tensors, with a memory-aware allocation [27]: Muon tensors are greedily assigned by memory footprint like bin packing, after which flattened Adam tensors fill the remaining per-device capacity as evenly as possible. This hybrid placement balances optimizer memory across GPUs and is important for fitting the training state into the 32 GB memory available on each RTX 5090.

These principles yield the same fastest configuration as an exhaustive enumeration of the candidate parallel strategies. Together with the hardware optimizations described above and standard Megatron Core options such as the `[T,H,D]` layout for packed-sequence attention, the Phase 1 production run sustains a median 238 TFLOP/s per GPU with global batch size 1536 on 24 GPUs across three nodes. The best configuration is MBS $= 2$, PP $= 2$ with layout `(18|10)`[5], and DP $= 12$. The measured throughput corresponds to an equivalent MFU[6] of approximately 73%. For larger-scale Phase 2 training, strong scaling is limited by Amdahl's law, and gradient synchronization accounts for a larger fraction of the step time. We consequently use PP $= 4$ with layout `(9|9|9|1)` and DP $= 24$ on 96 GPUs, while still achieving median throughput 192 TFLOP/s per GPU. The run-level setup and final losses are reported in Table 8.

3.2 FP8 Mixed-Precision Training

We use FP8 mixed precision from random initialization onward, without a BF16 warm-up stage or a later precision switch, using our training stack [68] and its blockwise FP8 support [69]. The training state and numerically sensitive operations retain the standard BF16/FP32 mixed-precision path, while Transformer linear layers use E4M3 operands. FP8 is therefore an online compute and activation-storage format rather than the persistent dtype of the whole model.

E4M3 [63] provides more mantissa bits than E5M2 but has a narrower dynamic range. With one scale for an entire tensor, a few outliers can enlarge the quantization interval and reduce the effective precision of ordinary values [26]. We instead compute scales online for local groups: activations and activation gradients use one-dimensional groups of 128 consecutive values along the GEMM reduction dimension, while weights use two-dimensional 128×128 blocks [101]. This fine-grained numerical design follows the recipe introduced and validated at scale by DeepSeek-V3 [26].

On our RTX 5090 (SM 120) [23], each logical block scale is constrained to a power of two and therefore has the exponent-only expressiveness of E8M0. Transformer Engine [69] implements these blocks through Blackwell's native MXFP8 path. Thus, our logical block sizes follow DeepSeek-V3 [26], while the scaling-factor representation and execution path follow MXFP8; the fact that DeepSeek-V3 was trained on Hopper GPUs may account for this implementation difference.

The complete precision flow is shown in Figure 14. Here, Transformer linear layers include the QKV projections, attention output projection, and MLP projections; their Fprop, Dgrad, and Wgrad GEMMs all use the FP8 path [26]. Core attention refers only to the FlashAttention/SDPA computation between these projections and remains in BF16 [101]. A linear layer quantizes its BF16 inputs and weights immediately before GEMM and materializes a BF16 output for the surrounding model. Its quantized input is retained for Wgrad, reducing the dominant saved-activation footprint [101]. FP8-accelerated operations account for 72% of the overall computation; the cost impact and net benefit of FP8 are analyzed in Figure 2.

The model-weight portion of each training checkpoint consequently remains BF16; the desired release or deployment representation is produced by a separate post-training conversion. In the matched 20-token-per-parameter scaling ladder, blockwise FP8 increases validation loss by 0.0031–0.0039 relative to BF16 across the five tested model sizes. The shared-shape fit maps this difference to 98.0% BF16-equivalent compute retention. At the 1.7B scale, however, blockwise FP8 improves median training throughput by $1.36\times$, yielding a quality-adjusted net gain of $1.34\times$. The complete setup, fitting procedure, and size-wise results are reported in Section 4.3.1.

3.3 Hyperball Optimization

In our recipe, we adopt Muon with the Hyperball wrapper (MuonH), introduced by Wen et al. [99], as the training optimizer for selected weight matrices. We apply MuonH to approximately scale-invariant weights, including

[5]The PP layout lists the numbers of Transformer layers assigned to successive stages. In `(18|10)`, the first stage contains the embedding and the first 18 Transformer layers, while the second contains the remaining 10 layers and the LM head.

[6]Model FLOPs utilization (MFU) is conventionally defined as achieved model FLOP/s divided by the accelerator's peak FLOP/s [45]. For mixed-precision training, we weight the peak Tensor Core performance by the fraction of theoretical work at each precision. In our recipe, FP8 accounts for 72% of the theoretical Tensor Core operations and BF16 for the remaining 28%, giving an effective peak of $P_{\text{eff}} = (0.72/419 + 0.28/209.5)^{-1} \approx 327$ TFLOP/s per GPU.

attention and MLP matrices, while embeddings, normalization layers, the language-model head, and the remaining parameters use AdamW. Following Wen et al. [99], MuonH normalizes the Muon update and constrains each wrapped matrix to a fixed-radius sphere. More specifically, for each wrapped matrix W_t, MuonH fixes the radius at $R = \|W_0\|_F$ and normalizes the Muon update u_t as $\widehat{u}_t = u_t / \|u_t\|_F$. It then applies

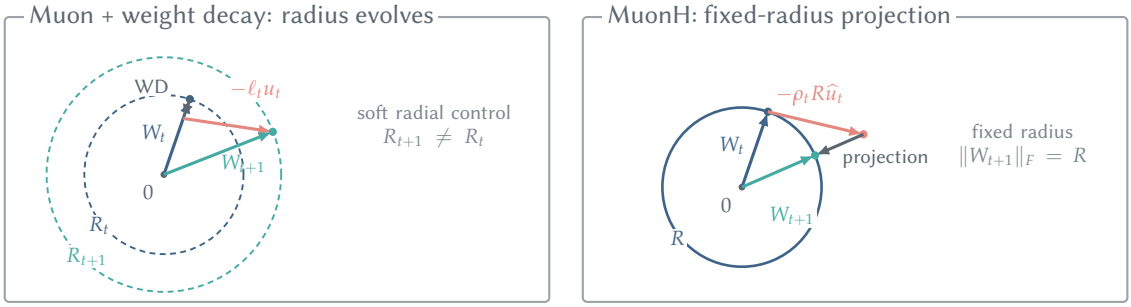
$$\widetilde{W}_{t+1} = W_t - \eta_t R \widehat{u}_t, \qquad W_{t+1} = R \ \mathrm{Normalize}(\widetilde{W}_{t+1}), \tag{1}$$

where $\mathrm{Normalize}(A) = A / \|A\|_F$. The first operation normalizes the update scale, while the second projects the updated weight matrix back to its initial Frobenius radius. The illustration of MuonH optimizer is shown in Figure 4.

We vary the learning rate η_t according to a two-phase learning rate schedule. The design of this schedule is motivated by an analysis of how the loss gap between MuonH and Muon arises from the perspective of effective learning rate, as we elaborate below.

3.3.1 *Effective Learning Rate*

First, we briefly revisit the rationale behind Hyperball optimization and explain how it makes the effective LR schedule explicit [99]. We then examine the loss gap between ordinary Muon and MuonH runs and find that the effective LR schedule strongly affects the loss convergence rate. This observation leads us to focus on learning rate schedule design for the MuonH optimizer.



	Muon + weight decay	**MuonH**
Weight radius	Evolves under weight decay and optimizer updates	Fixed at $R = \|W_0\|_F$ after each step
Displacement norm	$\eta_t \|u_t\|_F$ (state dependent)	$\rho_t R$ (scheduled)
Effective LR	$\eta_t \|u_t\|_F / \|W_t\|_F$ (indirect)	ρ_t (explicit)

Figure 4: Effective-LR control for one matrix. Ordinary Muon (left) applies a scalar learning rate to a state-dependent update norm, so both its effective LR and matrix radius emerge indirectly. MuonH (right) normalizes the update, prescribes a displacement of length $\rho_t R$, and projects the result back to the fixed-radius sphere.

Scale invariance motivates the effective learning rate. Hyperball is motivated by approximately scale-invariant parameter matrices. For a selected weight matrix W, we call the parameter approximately *scale invariant* when positively rescaling it has little effect on the model function or loss,

$$\mathcal{L}(cW) \approx \mathcal{L}(W), \qquad c > 0, \tag{2}$$

with the remaining parameters held fixed [99]. Normalization in modern Transformers makes this approximation relevant for several attention and MLP weight matrices. In this regime, the update magnitude is naturally interpreted relative to the current weight scale. We therefore consider the matrix-wise effective LR, also called the *effective learning rate* (ELR). For ordinary Muon, with Muon update u_t, scalar learning rate η_t, and decoupled weight-decay coefficient λ,

$$W_{t+1} = (1 - \lambda \eta_t) W_t - \eta_t u_t, \tag{3}$$

and, ignoring the radial shrinkage from weight decay, the effective LR is

$$\rho_t(W) = \frac{\eta_t \|u_t\|_F}{\|W_t\|_F}. \tag{4}$$

Thus, ordinary Muon's effective LR is not determined by the scalar learning rate alone, because both the weight norm and the Muon-update norm evolve during training.

MuonH makes the effective LR schedule explicit. Under the Hyperball update in Equation (1), the pre-projection displacement $\eta_t R\widehat{u}_t$ has norm $\eta_t R$, while the wrapped matrix W_t has norm R. Its effective LR is therefore $\rho_t(W) = \eta_t \|u_t\|_F / \|W_t\|_F = \eta_t R/R = \eta_t$, numerically equal to the Hyperball weight learning rate, and the subsequent projection restores the fixed radius. Thus, ordinary Muon induces its effective LR indirectly through the scalar learning rate and evolving norms, whereas *MuonH directly controls the effective LR schedule through its learning rate schedule.* This explicit control is the property of Hyperball that is most relevant to our subsequent schedule design.

In our experiments, we use different learning rates for the MuonH-wrapped and AdamW-optimized parameter groups. We define a base learning rate schedule η_t^{base} for the AdamW parameter groups and set the Hyperball weight learning rate to a fixed multiple, $\eta_t^H = m\,\eta_t^{\text{base}}$. For MuonH-wrapped matrices, η_t^H is numerically equal to the prescribed effective LR. In the production run, we use $m = 10$, while the remaining parameter groups follow η_t^{base} directly with AdamW.

Matching the effective LR schedule also aligns validation loss. The MuonH optimizer can make training more efficient than the ordinary Muon optimizer, and we find that the efficiency gap can be attributed to the difference in their effective LR schedule. We compare three 170M-parameter BF16 runs with the same architecture, batch size, sequence length, seed, and evaluation protocol. The MuonH run uses a predefined effective LR schedule ρ_t^H. The ordinary-LR run uses the base LR schedule of the MuonH run as the LR schedule and does not compensate for the evolving weight and update norms. Then, as shown in Figure 5, the ordinary-LR Muon run produces an effective LR trace that decays rapidly early in training and finishes close to zero. The MuonH run instead follows the predefined linear-decay effective-LR schedule. Although the ordinary-LR Muon run achieves lower validation loss early in training, MuonH overtakes it near the end. The MuonH run finishes at a validation loss of 3.029, compared with 3.073 for ordinary-LR Muon.

For comparison, we conduct an effective-LR aligned run, which uses the ordinary Muon optimizer but adjusts its scalar learning rate online to follow the same effective LR curve,

$$\eta_t = \rho_t^H \frac{\|W_t\|_F}{\|u_t\|_F}, \tag{5}$$

so that the effective LR $\rho_t(W) = \rho_t^H$ without projecting W_{t+1} back to a fixed-radius sphere. As shown in Figure 5, effective-LR aligned Muon follows the prescribed linear post-warmup LR decay and reaches a final validation loss of 3.030, close to MuonH at 3.029. Thus, in this selected diagnostic, *ordinary Muon largely recovers the MuonH loss curve when its effective LR schedule is aligned.* This comparison supports the effective-LR schedule as an important factor in MuonH's training behavior. It also suggests that better control of the effective-LR schedule can improve loss convergence. Recent work also discusses related behavior from the perspective of angular update size [120, 100]. We provide an additional ordinary Muon control in Section F.3 that can even use a counterintuitive hill-like scalar learning rate schedule to achieve loss convergence while maintaining the prescribed effective LR trajectory.

Effective LR provides a more informative descriptor of the loss trend. The experiments above show a close relationship between effective LR and loss dynamics. We further test whether effective LR provides a more informative descriptor than the raw optimizer learning rate. We apply the Multi-Power Law (MPL), a LR-schedule scaling law that predicts loss-curve shape from the LR schedule, to both the effective-LR schedule and the raw learning rate schedule [57]. For the two ordinary-Muon runs above, we fit the post-warmup validation curves using either the scalar learning rate or the induced effective LR, while holding out the final 20% of validation targets. As shown in Figure 16, using the effective LR reduces the mean held-out RMSE across the two runs from 0.0265 to 0.0210. The improvement is driven mainly by the base-Muon run; for effective-LR aligned Muon, the two LR signals are comparably predictive. This small diagnostic therefore supports effective LR as a more informative descriptor for schedule-dependent training dynamics. The MPL formulation and fitting results are provided in Section F.1.

Effective LR MPL helps interpret the late-stage crossover. In the selected 170M diagnostic, MuonH and effective-LR aligned Muon converge more slowly at the beginning but outperform ordinary Muon near the end of training. This pattern is consistent with the learning rate analysis in Section F. In particular, the MPL fit [57] provides a simple diagnostic relationship: *a decrease in learning rate can induce an approximately proportional reduction in loss, with the loss decrease taking effect over subsequent training steps.* Applying this view to the effective LR, ordinary Muon decays its effective LR more aggressively early in training, which is consistent with its faster early loss reduction but leaves less effective LR decay for the later stage. MuonH and aligned Muon instead follow a more gradual, approximately linear effective LR decay, giving up some early convergence speed

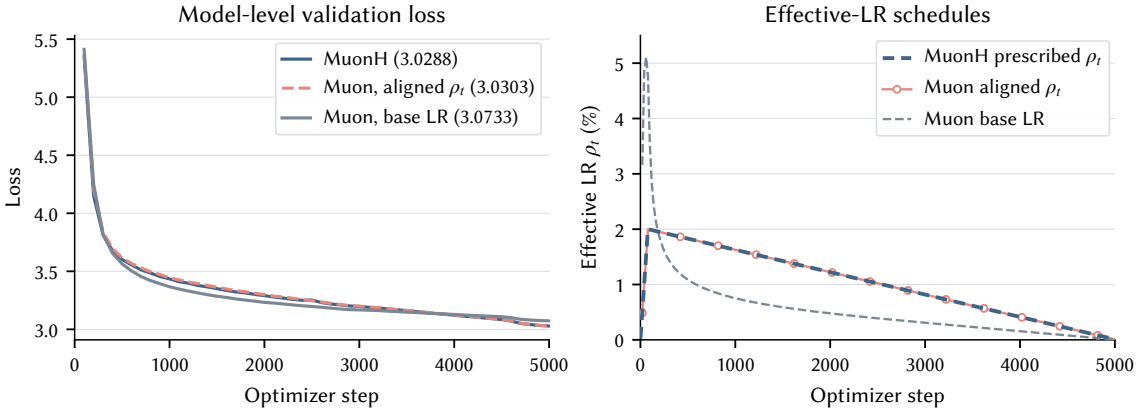


Figure 5: 170M BF16 comparison of MuonH, effective-LR aligned Muon, and ordinary-LR Muon. **Left:** validation loss. **Right:** effective LR traces for the selected MLP down-projection matrices. The training run with aligned-effective-LR and the normal Muon optimizer follows the predefined MuonH effective LR schedule. This run reaches a similar final loss, whereas ordinary-LR Muon produces a faster early decay of its induced effective LR and finishes with higher loss.

while retaining more decay toward the end of training. This view provides additional motivation for using a linear-decay schedule in our production design.

Together, these diagnostics motivate treating effective LR as the primary hyperparameter for MuonH-wrapped weights. They also suggest that the placement of its decay can affect where loss reduction occurs during training. We therefore next study the practical design of the LR schedule for production run.

3.3.2 Learning Rate Schedule Design

Continual training motivates an open-ended schedule followed by a controlled terminal decay. In practice, the total token budget is commonly undetermined in the beginning [43, 86, 2]. Hence, continual pre-training is a practical setting and may change the data mixture after the initial horizon. We therefore want to retain an open-ended schedule before applying a controlled terminal decay. Warmup-Stable-Decay (WSD) schedule provides a testbed for choosing the decay duration [43]. Motivated by the previous section's experiments, we use a linear decay function in WSD. All sweeps use a 0.6B model with the Qwen3-0.6B architecture and global batch size 512. At 20 tokens per parameter (TPP), or 11.3B tokens, we vary the effective peak from 0.008 to 0.024 in increments of 0.004 and the WSD decay ratio from 0.2 to 1.0 in increments of 0.2. At peaks 0.008 and 0.012, we additionally test TPP 50 and 100.

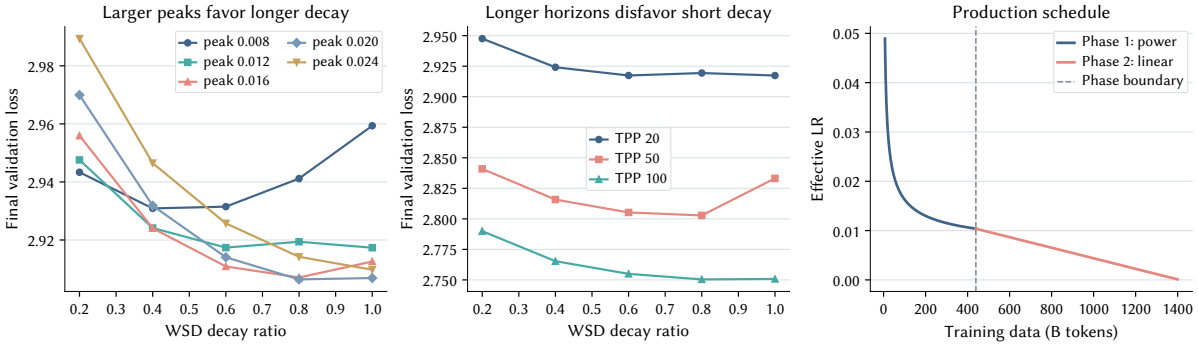


Figure 6: Experiments that support our observations. **Left:** At TPP 20, larger effective peaks increasingly penalize short WSD decay. **Center:** At fixed effective peak 0.012, longer horizons make short decay less competitive. Each sweep curve reports final validation loss from one run per configuration. **Right:** The production effective LR schedule uses an open-ended power-decay phase followed by a long linear-decay phase.

Observation 1: a larger peak LR calls for a longer decay. In the left panel of Figure 6, the best grid points at effective peaks 0.008, 0.012, 0.016, 0.020, and 0.024 are decay ratios 0.4, 1.0, 0.8, 0.8, and 1.0. These point

optima fluctuate because the long-decay region is broad. A more stable summary is the lower edge of the region within 0.01 validation loss of the minimum: it moves nondecreasingly as 0.4, 0.4, 0.6, 0.6, and 0.8, as shown in Figure 18. At peaks 0.020 and 0.024, ratios 0.8 and 1.0 differ by only 0.0005 and 0.0044 loss. Thus an almost fully linear decay is already competitive near the high end of the tested peak range. The two-anchor interpolation in Figure 17 provides a complementary check: using only ratios 0.2 and 1.0 at peak 0.024, it places the estimated minimum at ratio 0.85 within the same long-decay region.

Observation 2: a longer training horizon also calls for a longer decay. At effective peak 0.012, the center panel of Figure 6 shows the competitive region shifting toward longer decay. Its lower edge moves from 0.4 at TPP 20 to 0.6 at TPP 50 and 100, while the point optima are 1.0, 0.8, and 0.8. At peak 0.008, the lower edge remains 0.4, although the point optimum moves from 0.4 to 0.6. The latter TPP100 grid lacks ratio 0.2, so we treat it only as supporting evidence. The complete peak-0.012 lower-edge trace appears in Figure 18; the conclusion is a broad shift toward longer decay, not a precise monotone point optimum.

The production schedule combines these three practical features. The right panel of Figure 6 shows the complete production effective-LR schedule. Here, we present the Hyperball weight LR, which is numerically the effective LR for wrapped matrices, instead of the base LR. Phase 1 uses the open-ended power schedule in Equation (10), decreasing from approximately 5×10^{-2} to 1.04×10^{-2} over 438.8B. Its tail preserves a continuation path as the data distribution evolves [86]. Phase 2 joins near 10^{-2} and uses a long linear decay over 960.0B rather than spending most of training in a stable segment followed by a short terminal drop. The design therefore combines continual trainability, a high initial effective LR, and a long final decay.

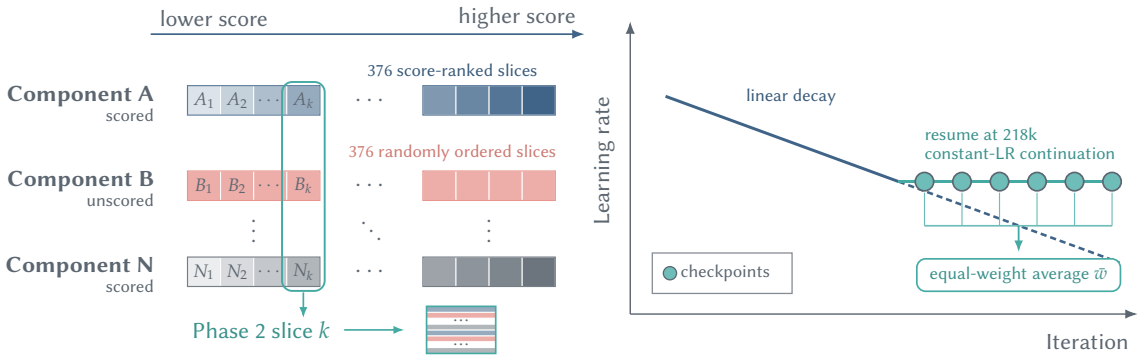
Multi-power law provides a back-of-the-envelope, cross-schedule estimator under limited compute. Before a long run, exhaustively sweeping every decay ratio is impractical. Therefore, we fit a multi-power law (MPL) [57] as a diagnostic, using only the two endpoint schedules. The runs are at TPP 20 and with decay ratios 0.2 and 1.0. Two curves per effective peak recover the increasing peak-to-decay trend: the estimated ratio rises from 0.33 at peak 0.008 to 0.85 at peak 0.024. Although it is not a certificate of optimality, we use this result as a back-of-the-envelope trend diagnostic. The fitting protocol, unseen schedules, and errors are reported in Section F.

3.4 Curriculum Model Averaging

When open-source datasets provide usable sample-level scores, we rank samples within each source and construct a mixture-preserving curriculum that places more preferred samples later in training. This ordering is designed to improve the data efficiency of higher-quality samples. However, a conventional terminal learning-rate decay can work against this objective: *examples presented near the end of the curriculum are also processed when parameter updates have become small* [56]. Curriculum Model Averaging (CMA) [56] addresses this conflict by combining an ascending data curriculum with constant-LR training and averaging late checkpoints. Our production recipe adopts this principle in Phase 2. We first follow the scheduled learning-rate trajectory, then resume from a selected late checkpoint, hold the base learning rate fixed, and average six checkpoints from the continuation. The complete data and optimization flow is summarized in Figure 7, with implementation details provided in Sections H.1 and H.3. We next describe the curriculum construction, the selected constant-LR continuation with checkpoint averaging, and the corresponding ablation results.

For convenience, we label the Phase 2 variants: **UD** (**U**niform data with LR **D**ecay) globally reshuffles the Phase 2 pool and follows the scheduled learning-rate decay. **CD** (**C**urriculum data with LR **D**ecay) keeps that decay but presents the data pool in the component-local curriculum order. **CDC** (**C**urriculum data, with LR **D**ecay then **C**onstant LR) uses the curriculum order, follows the decay, then continues at a constant learning rate, and reports the final checkpoint. **CMA** (**C**urriculum **M**odel **A**veraging) is the production variant: it averages the last several checkpoints from the CDC recipe.

The data curriculum arranges the order within each component while keeping the target mixture weight. We first establish a reproducible order separately for every retained component. If a component provides a usable sample-level quality signal, we sort its examples according to the source-specific interpretation of its score, placing less preferred score ranges earlier and more preferred ranges later. Components without a usable score instead follow a fixed random order. We then assign each example a normalized within-component rank according to its position in this ordered sequence, measured by cumulative token mass. Intuitively, this rank indicates which source-local score range an example falls into. For example, a rank of 0.25 means that approximately one quarter of the component's tokens appear earlier in its ordered sequence.



(a) Rank within each component and merged by progress (b) Constant-LR continuation & Model Average

Figure 7: **CMA-inspired data and optimization flow in the production Phase 2 recipe.** Component-local ordering, transition, late constant-LR continuation, and six-checkpoint averaging are shown together. Exact bucket construction, transition, and averaging details are given in Sections H.1 to H.3.

We divide the normalized rank range into 376 intervals, with each curriculum bucket formed by taking the corresponding interval from every component. Each bucket contains approximately 2.5B tokens, drawing roughly $1/376$ of tokens from every component and preserving the selected cross-component mixture. As training advances through the buckets, scored components move toward their more preferred score ranges, while unscored components advance through their fixed random orders. Since scores are interpreted only within their own sources and are not compared across components, this construction is not a global quality ranking. The scalable approximation used to construct the 376 buckets and the exact transition schedule are given in Sections H.1 and H.2 and Table 20.

The canonical CMA run uses the best-performing late-stage continuation with model averaging. We compare three late-stage training strategies: averaging checkpoints directly from the terminal-decay trajectory, and two constant-LR continuations resumed from steps 215,000 and 218,000. Among these configurations, the continuation from step 218,000 gives the strongest observed performance and is therefore selected for the production model. For this continuation, training resumes from step 218,000 with the base learning rate fixed at 4.08×10^{-5}. With the production Hyperball multiplier $m = 10$, the Hyperball weight LR is therefore 4.08×10^{-4}, which is also the effective LR for the MuonH-wrapped matrices. During the late stage of this continuation, we save checkpoints at approximately regular intervals and form the released model by equally averaging the parameters of six such checkpoints. Neighboring checkpoints are typically separated by 100 optimizer steps, or approximately 0.63B tokens under the production batch configuration. Full resume, checkpoint-spacing, and averaging details are provided in Section H.3.

Ablation study on curriculum, model average, and const-LR continuation. The endpoint comparisons in Figure 13b support a structured comparison of the jointly selected recipe ingredients, as illustrated in Section G and fig. 13b. First, *curriculum ordering improves over uniform ordering* by 1.18 points without model averaging (CD: 57.17 versus UD: 55.99) and by 1.61 points when comparing the model averages of the corresponding decay trajectories (CD model average: 57.18 versus UD model average: 55.57). Second, *model averaging is not independently beneficial in both ordering conditions*: it changes the UD endpoint by -0.42 points and the CD endpoint by $+0.01$ points without constant-LR continuation. Finally, the two CDC controls score 55.64 and 57.12 for the 215k and 218k continuations, respectively, when evaluated at their unaveraged final checkpoints. With six-checkpoint averaging, the corresponding branches reach 56.80 and 57.81. We therefore select the 218k averaged branch as the production CMA endpoint. It combines curriculum ordering, late constant-LR continuation and six-checkpoint averaging. Part of the observed advantage of CD over UD ordering could be attributed to benchmark contamination in the later-stage data, as discussed in Section A. But the remaining advantage is observed after post-training in Section 4.4, and the contamination alone is unlikely to explain the persistent improvement. We report the exact base LR and Hyperball effective LR, checkpoint IDs, and the model-export procedure in Section H.3.

3.5 Post-Training Recipe

Motivation and question. In the above, we present a complete, low-cost pretraining recipe. The resulting pretrained checkpoints are later used for post-training. Here we focus on supervised fine-tuning (SFT). This

raises a natural but underexplored question: *does the choice of pretraining recipe affect model performance after SFT?* A released checkpoint alone is usually not enough to answer this question. Even when a checkpoint is publicly available, its full training recipe and data may be unavailable. This makes it difficult and expensive to rerun the training and ablate specific pretraining choices. Small-scale ablations are easier, but the undertrained base models may be too weak to show a stable post-training performance [78]. Therefore, our low-cost, fully open recipe allows us to study this question at the scale of the final model.

In particular, we compare the SFT results of checkpoints from two Phase 2 recipes: uniform data ordering with learning rate decay (UD) and Curriculum Model Averaging (CMA). The UD checkpoint performs worse on the base-model benchmark suite than the CMA checkpoint. Although both recipes use the same model architecture and Phase 2 training corpus, they differ in data ordering and checkpoint construction. This comparison therefore requires two separate Phase 2 training runs. In the CMA recipe, the curriculum places more high-quality data near the end of training. This changes the data distribution over the course of Phase 2. The base-model improvement may therefore be temporary and tied mainly to the final stage of pretraining. We test whether this improvement remains after both checkpoints undergo the same SFT procedure.

Data construction and experimental setup. To study this question, we apply the same SFT procedure to the UD and CMA checkpoints for each data setup. All other training conditions are matched within the same setup. We continue to use the MuonH optimizer, as in pretraining [99]. The base learning rate follows a cosine schedule that decays from 1×10^{-5} to 1×10^{-7}. The MuonH-managed matrix parameters use a $10\times$ multiplier to obtain their Hyperball weight learning rate. We set the global batch size to 160.

To examine the results under different SFT data distributions, we construct three data setups. (1) *GSM8K-based SFT*. It combines the original GSM8K training split with GSM8K-related synthetic data from MetaMathQA and OpenMathInstruct-2. It also includes selected Tulu-3 components, which cover mathematics and instruction following. (2) *Math&Code SFT with replay*. It contains a larger collection of mathematics data from GSM8K, MetaMathQA, OpenMathInstruct-1, and OpenMathInstruct-2. It also includes code data from OpenCodeInstruct and Magicoder. We add a small, shuffled replay set from the final part of the Phase 2 curriculum, which contains the highest-quality Phase 2 data. (3) *Tulu-3 mixed-domain SFT*. It uses the English portion of `Tulu-3-sft-mixture-0225` [46]. Because this mixture covers several domains, it provides a broader test of general capability and instruction following.

For all three setups, we deduplicate the training examples and discard empty sequences. For decontamination, we remove any training example that shares a 13-gram with an evaluation test example. Detailed data composition and training settings are provided in Section E.

Evaluation. We use the same evaluation protocol for the UD and CMA models within each setup. For GSM8K-based SFT and Math&Code SFT with replay, we evaluate the resulting models on GSM8K with the `lm-evaluation-harness` framework. Because generated answers can differ in format, we use flexible extraction to identify the final answer. We report the average result over three repeated seeds. For Tulu-3 mixed-domain SFT, we use OpenCompass to evaluate the resulting models on 15 tasks for base models and IFEval. The evaluation protocol of the 15 tasks is adapted to the fine-tuned models. This suite covers knowledge, reasoning, code, and instruction following. We report both the overall average and the individual task results, since the effect may differ across tasks. Further evaluation details are provided in Section E.

3.6 Data Recipe

Our goal is to select a compact set of high-quality and diverse data sources while preserving broad domain coverage. Candidate datasets differ in both quality and capability profile, so source-level metadata alone is not sufficient for data selection. We therefore use *proxy benchmarking* to compare candidate data slices under a shared protocol. As illustrated in Figure 9, each run starts from the same Qwen3-0.6B checkpoint and follows the same continuation-training schedule. After continuation training, we evaluate the resulting checkpoint on a fixed 15-benchmark suite. The score vector from this evaluation serves as the capability profile of the candidate slice. These score vectors can guide our data recipe in heuristic. The following three paragraphs describe how we construct fine-grained slices, select and allocate data sources, and balance different capabilities in the final mixture. Detailed settings are provided in Section I.0.

3.6.1 How Do We Select a Good Data Recipe?

Fine-grained data slice selection. We construct candidate data slices according to source size and the availability of sample-level quality scores. We apply three sampling rules. First, if a dataset contains more than 50B

Domain	Phase 1		Phase 2	
	Tokens	Share	Tokens	Share
English	321.31B	73.2%	557.52B	59.4%
Math	31.56B	7.2%	171.29B	18.3%
Chinese	51.34B	11.7%	88.48B	9.4%
Code	34.62B	7.9%	108.11B	11.5%
SFT / instruction	–	0.0%	12.66B	1.3%
Total	438.84B	100.0%	938.06B	100.0%

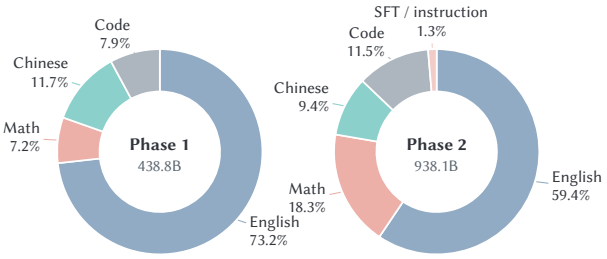


Figure 8: Domain composition of the two pretraining phases. The left table reports materialized token counts and shares for Phase 1 and Phase 2, and the right panel visualizes the same composition. Phase 1 emphasizes broad coverage, while Phase 2 allocates a larger share to mathematics and introduces instruction-like data. The Phase 2 token budget excludes Phase 1 replay. See Section 3.6.2 for details.

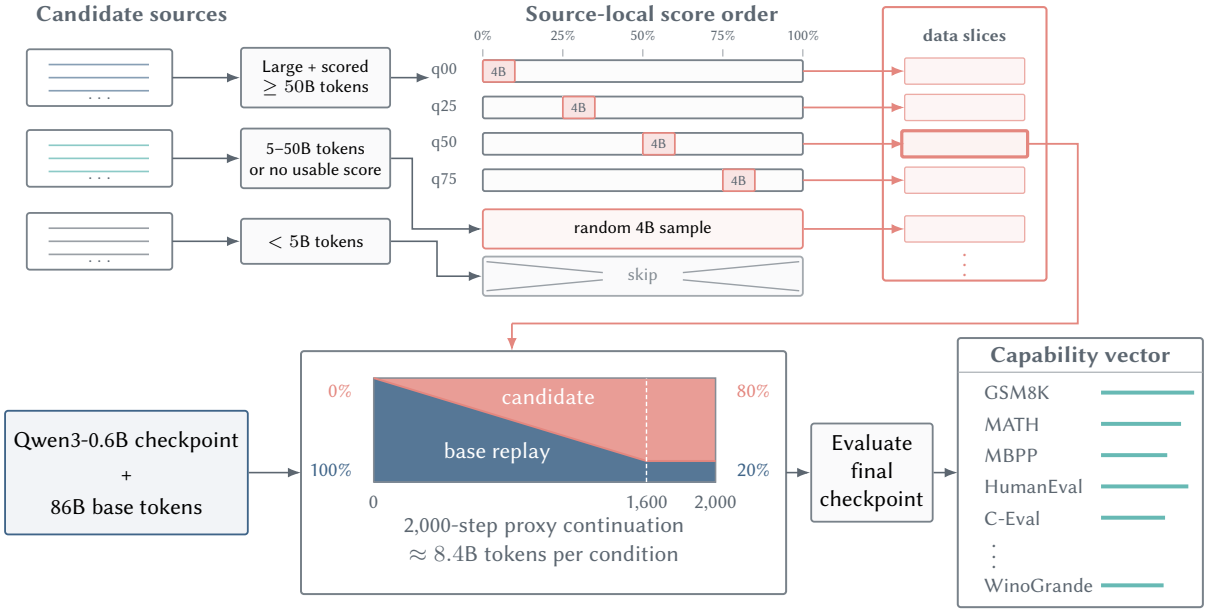


Figure 9: Proxy benchmarking protocol. Candidate slices are first constructed according to corpus size and the availability of sample-level quality scores. Large scored sources contribute four within-source quantile slices, while other eligible sources contribute one random 4B-token slice. Each retained slice is used for a 2,000-step continuation run from the same Qwen3-0.6B checkpoint. During continuation training, the candidate-data ratio gradually rises to 80%, while replay from the base mixture decreases accordingly. The final checkpoint is evaluated on the same 15-benchmark suite to obtain a capability vector. These vectors guide the fine-grained slice selection, source allocation, and capability balancing described below.

tokens and provides usable score labels, we sort its examples by score and sample four local slices around the 0th, 25th, 50th, and 75th percentile positions. Second, for datasets between 5B and 50B tokens, as well as larger datasets without usable score labels, we randomly sample one 4B-token slice. Third, we do not evaluate the remaining smaller datasets. For each large scored source, the four slices form a within-source quantile benchmark. Because these slices come from different positions in the same score ordering, they reveal quality differences that a single source-level result would hide. For example, the q00 slice of DCLM-Dedup ranks third in average proxy performance, while its q25 slice ranks eleventh. This result shows that different regions of the same source can provide substantially different training value.

Feature-guided data selection. We use the measured capability profiles to guide both source selection and data allocation. MegaMath-Web-Pro performs strongly on Math and General, while MegaMath-Code provides the strongest Code signal among the measured candidates. We therefore assign both sources larger shares in Phase 2 to strengthen general and target capabilities. The quantile results also guide within-source filtering.

Because the top-scoring region of DCLM-Dedup performs much better than its lower-ranked regions, we retain 34B tokens from this region in Phase 2. This selection corresponds to approximately the top 5% of DCLM-Dedup. FineWeb-Edu-CN ranks highest on Chinese capability, so we also increase its share to preserve Chinese performance. The proxy results provide useful evidence for increasing, reducing, or filtering different data sources. Here, we take heuristics from these capability profiles and still choose the concrete mixing ratios manually.

Capability balance across domains. The proxy results show that different datasets have substantially different capability profiles. A source that performs well on one group of benchmarks may perform less well on others. For example, FineWeb-Edu-CN ranks first on Chinese capability but is weaker on Math, Code, and General (Section I). We therefore need to balance broad general knowledge with more specialized reasoning capabilities rather than maximize a single benchmark dimension. To make these trade-offs more explicit, we apply PCA to the capability vectors. As shown in Figure 20, General aligns mainly with PC1, while Code points in the opposite direction. Math aligns mainly with PC2 and remains slightly positive on PC1. In this projection, Code shows a stronger trade-off with General than Math does (Section I.0). This observation informs the roles of the two pretraining phases. Phase 1 preserves broad coverage, with English accounting for 73.2% of 438.8B (Table 3 and fig. 8). Phase 2 then adds more specialist data, but allocates a slightly larger share to mathematics than to code. In this way, the two-phase recipe strengthens the target reasoning capabilities while retaining broad general coverage.

3.6.2 How Do We Preprocess Data and Reproduce the Shards?

Data preprocessing pipeline. The pipeline turns the candidate collection into fixed training shards in four stages. *Within-source deduplication* removes repeated documents from web components such as DCLM-Dedup and FineWeb-Edu-EN. We do not deduplicate across sources because our analysis found little additional reduction in data volume. *Proxy benchmarking* evaluates each eligible candidate source or within-source slice under the shared protocol described above and produces a multidimensional capability profile. *Heuristic filtering* combines these profiles with domain coverage, language balance, and token-budget constraints to determine the retained portion and mixture weight of each source. *Final materialization* freezes the source revisions, retention rules, mixture weights, and random seeds. It then merges the retained components into fixed shards that are read sequentially during training, without online mixing or sampling. In addition, we also use PreSelect fastText [89] to evaluate the Nemotron-HQ [58] and Nemotron-HQ-Synthetic datasets. Apart from this step, we do not perform additional compute-intensive data scoring and primarily rely on existing quality scores.

KAIYUAN-SPARK preprocessing framework. We implement the recipe with KAIYUAN-SPARK, a Spark-based data preprocessing framework [55]. The framework is designed for efficient large-scale data processing. For compute-intensive operations such as MinHash deduplication, it uses native C++ kernels through the Chukonu integration. This implementation is inherited from Chukonu, whose original Spark benchmark reports an approximately 2.5× speedup over the corresponding JVM implementation [109]. The framework also supports the curriculum bucket materialization required by the Phase 2 design. Given a fixed component pool and mixture, it assigns normalized progress within each component, aligns the resulting slices into token-sized buckets, and materializes their order using recorded random seeds. The same component pool can therefore produce either an ordered curriculum stream or a uniformly ordered stream by changing only the materialization order. These two data streams support the UD, CD, and CMA recipes in Phase 2 (Section 3.4).

Two-phase data and transition schedule. All training data are materialized as static shards and manifests before training. Production runs therefore do not rely on online scoring or dynamic mixture sampling. The Phase 1-to-Phase 2 transition spans approximately 43.9B training tokens. During this transition, the sampling weight of the Phase 1 replay mixture decreases *linearly*, while that of the Phase 2 mixture increases *linearly*. The transition therefore consumes approximately 21.9B tokens from the Phase 1 replay data and 21.9B tokens from the beginning of the Phase 2 data pool. The full Phase 2 component pool contains approximately 938.1B tokens, including the 21.9B tokens consumed during the transition. After the transition, the remaining 916.1B Phase 2 tokens are consumed without Phase 1 replay. Counting the transition, the Phase 2 training stage consumes approximately 960.0B tokens in total: 21.9B tokens of Phase 1 replay and 938.1B tokens from the Phase 2 pool. Together with Phase 1 training, the complete pretraining run consumes approximately 1.4T tokens.

4 Evaluation

4.1 Evaluation Setup

4.1.1 Model Selection

We compare PURO-2B with recent language models of comparable size, primarily in the 1B–3B parameter range. We group these baselines according to the scope of their public releases. **Open-weight models** make their weights publicly available but do not release the complete training data or recipe, whereas **open-recipe models** additionally release the data and training artifacts needed to study or reproduce training.

Open-weight models.

- **Qwen series** [103, 104, 102, 80]: We include Qwen2-1.5B, Qwen2.5-1.5B, Qwen3-1.7B-Base, and Qwen3.5-2B-Base, which provide successive generations of strong compact foundation models in the 1.5B–2B range.
- **Gemma series** [81, 31, 32]: We select Gemma-2-2B, Gemma-3-1B-PT, and Gemma-4-E2B-Base to cover three generations of Google's lightweight open models.
- **Llama 3.2** [60]: We include Llama-3.2-3B, the latest pretrained Llama checkpoint available in the compact 1B–3B range.
- **LFM2.5** [50]: We include LFM2.5-1.2B-Base as a recent compact base model designed for efficient on-device deployment.
- **Falcon-H1** [121]: We evaluate the standard and deeper 1.5B base variants as efficiency-oriented hybrid-model baselines.

Open-recipe models.

- **Instella** [52]: Instella-3B was trained entirely on openly available data, with model weights, training data, configurations, and code released to support reproducibility. Despite using fewer pretraining tokens than many contemporary models, it is reported to remain competitive with leading open-weight models at a similar scale.
- **OLMoE** [64]: OLMoE-1B-7B-0125 is a sparse model with 1B active and 7B total parameters. Its release includes intermediate checkpoints, training data, code, logs, and evaluation resources. The model is positioned as a strong performance–efficiency baseline among fully-open mixture-of-experts models.
- **Yulan-Mini-2.4B** [108]: Yulan-Mini-2.4B is a 2.4B model trained on 1.1T tokens, with its training recipe and phase-wise data composition publicly documented. It emphasizes data-efficient pretraining and reports competitive performance against similarly sized models trained with substantially more data.
- **SmolLM3** [7]: SmolLM3-3B-Base was pretrained on approximately 11T tokens, with the public data mixture and training configurations released alongside the model. It is presented as a strong fully-open model at the 3B scale, with performance competitive with several larger compact models.
- **MiniCPM5** [72]: MiniCPM5-1B-Base is a compact dense model developed for local and resource-constrained deployment. Its associated web and mathematical training corpora are publicly released, while the MiniCPM5 family targets strong reasoning, coding, and mathematical capabilities within a small deployment footprint.

For a fair comparison, we consistently evaluate the pretrained/base versions of all models.

4.1.2 Benchmarks

Our evaluation covers 15 benchmarks, organized into two groups: mathematics and code, and reasoning and knowledge. For mathematical reasoning, we use GSM8K [18] and the original MATH benchmark [41], which measure grade-school arithmetic and competition-level mathematics, respectively. For code generation, we use sanitized-MBPP [6] and HumanEval [12], where generated Python programs are verified by unit tests. For reasoning and knowledge, we use MMLU [40], MMLU-Pro [97], ARC-Challenge and ARC-Easy [17], BoolQ [16], CommonsenseQA (CSQA) [93], HellaSwag [111], PIQA [11], SocialIQA (SIQA) [83], WinoGrande [82], and BigBench Hard (BBH) [92]. Together, these benchmarks assess mathematical reasoning, code generation, academic knowledge, reading comprehension, commonsense understanding, and general multi-step reasoning.

4.1.3 Implementation Details

We evaluate every model with OpenCompass [21] using a fixed checkpoint and tokenizer revision, dataset snapshot, benchmark-specific prompt, shot examples, generation configuration, and answer postprocessor. GSM8K, MATH, sanitized-MBPP, HumanEval, MMLU-Pro, and BBH use generation-based (GEN) evaluation: the model produces a free-form response from which the final answer or executable code is parsed. GEN decoding is greedy with fixed maximum output and stopping rules. The remaining nine benchmarks use perplexity-based (PPL) evaluation, which ranks the provided candidate answers by their fixed token log-likelihoods and therefore introduces no generation randomness. For GSM8K, a shared deterministic postprocessor prioritizes explicit final-answer markers and boxed answers, preserves signs and fractions, and uses the final numeric expression only when no explicit answer is present.

Following the OLMES [36] convention, for tasks supporting both cloze formulation (CF) and multiple-choice formulation (MCF), we evaluate both formulations for each model and use the better-performing formulation when computing the reported aggregate. This selection is performed independently for each model. All scores are percentages. In each table, Avg is the unweighted arithmetic mean over the benchmarks displayed in that table. Throughout the cost–performance and scaling analyses, overall performance P denotes the unweighted arithmetic mean over all 15 benchmark scores reported in Tables 4 and 5. Every score in our tables is produced by this common, deterministic pipeline.

4.2 Resulting Performance

Table 4: Core capabilities in mathematics and code generation. All scores are produced by the same evaluation pipeline and reported as percentages. Avg is the unweighted arithmetic mean over the four benchmarks.

Model Name	Params	Math		Code		Avg
		GSM8K 4-shot	MATH 4-shot	sanitized-MBPP 3-shot	HumanEval 3-shot	
Open-Weight Models						
Qwen2-1.5B	1.5B	59.82	23.70	50.19	27.44	40.29
Qwen2.5-1.5B	1.5B	67.70	32.28	58.37	31.71	47.52
Qwen3-1.7B-Base	1.7B	76.04	42.44	64.20	51.22	58.48
Qwen3.5-2B-Base	2B	69.83	35.82	43.58	32.32	45.39
Gemma-2-2B	2B	33.81	14.84	38.52	20.12	26.82
Gemma-3-1B-PT	1B	2.58	0.94	9.34	8.54	5.35
Gemma-4-E2B-Base	E2B	30.71	9.68	47.86	24.39	28.16
Llama-3.2-3B	3B	28.20	8.14	50.19	31.10	29.41
LFM2.5-1.2B-Base	1.2B	53.22	28.00	38.13	25.61	36.24
Falcon-H1-1.5B-Base	1.5B	74.68	42.60	61.09	28.05	51.61
Falcon-H1-Deep-1.5B-Base	1.5B	53.53	41.28	63.04	32.93	47.70
Open-Recipe Models						
Instella-3B	3B	64.44	13.30	41.25	14.63	33.41
OLMoE-1B-7B-0125	A1B/7B	54.59	15.48	33.46	12.80	29.08
Yulan-Mini-2.4B	2.4B	68.99	27.18	62.26	50.00	52.11
SmolLM3-3B-Base	3B	81.73	56.12	60.31	37.80	58.99
MiniCPM5-1B-Base	1B	39.88	23.34	46.30	29.27	34.70
MobileLLM-R1-950M-base	950M	68.31	27.24	50.58	43.29	47.36
Ours						
Puro-2B ($4.4K)	2B	56.03	27.86	49.81	20.73	38.61
Puro-2B	2B	59.67	30.30	52.92	31.10	43.50

Mathematics and code. As shown in Table 4, PURO-2B obtains an average score of 43.50 across the four generative benchmarks. It exceeds Qwen2-1.5B by 3.21 points and is within 4.02 points of Qwen2.5-1.5B. Among the open-recipe baselines, PURO-2B outperforms Instella-3B, OLMoE-A1B/7B, and MiniCPM5-1B-Base, while Yulan-Mini-2.4B, SmolLM3-3B-Base, and the newly included MobileLLM-R1-950M-base achieve higher averages. MobileLLM-R1 is a strong math/code-oriented compact baseline in this panel, exceeding PURO-2B by 3.86 points

on the four-task average. Given these differences in model size and capability emphasis, PURO-2B remains a competitive open-recipe model for mathematics and code generation at a comparatively small scale.

Table 5: Reasoning and knowledge capabilities. All scores are produced by the same evaluation pipeline and reported as percentages. Avg is the unweighted arithmetic mean over all eleven benchmarks.

Model Name	Params	Reasoning & Knowledge											Avg
		MMLU 5-shot	MMLU-Pro 5-shot CoT	ARC-C 5-shot	ARC-E 5-shot	BoolQ 5-shot	CSQA 5-shot	HSwag 5-shot	PIQA 5-shot	SIQA 5-shot	Wino 5-shot	BBH 4-shot	
Open-Weight Models													
Qwen2-1.5B	1.5B	56.39	23.62	69.83	83.77	71.90	70.52	59.91	75.19	63.36	59.67	31.73	60.54
Qwen2.5-1.5B	1.5B	61.56	30.43	79.32	90.48	76.39	75.10	64.18	76.17	64.94	59.67	42.64	65.53
Qwen3-1.7B-Base	1.7B	65.47	37.60	80.34	91.89	79.63	74.69	60.47	76.06	68.68	59.19	51.18	67.75
Qwen3.5-2B-Base	2B	63.95	37.34	83.73	93.12	85.63	73.05	63.52	76.50	69.14	58.80	60.37	69.56
Gemma-2-2B	2B	55.22	24.40	66.44	82.54	71.83	66.18	35.22	66.10	65.86	52.01	35.88	56.52
Gemma-3-1B-PT	1B	26.17	11.77	26.78	26.46	62.91	20.56	25.65	49.40	32.55	49.57	26.89	32.61
Gemma-4-E2B-Base	E2B	59.72	25.55	73.56	86.24	77.28	65.77	44.38	67.79	65.66	52.64	39.69	59.84
Llama-3.2-3B	3B	57.55	27.71	72.88	84.83	76.02	66.91	48.12	76.06	64.28	52.57	38.01	60.45
LFM2.5-1.2B-Base	1.2B	57.50	17.74	82.03	87.83	74.40	59.13	54.20	74.97	59.26	57.54	33.54	59.83
Falcon-H1-1.5B-Base	1.5B	65.32	36.44	81.02	90.30	78.84	71.25	62.24	75.73	67.30	57.77	45.35	66.51
Falcon-H1-Deep-1.5B-Base	1.5B	68.74	42.99	85.08	93.12	85.23	72.40	65.24	76.61	70.83	60.14	41.86	69.29
Open-Recipe Models													
Instella-3B	3B	58.55	25.32	74.92	87.30	76.76	75.43	59.92	73.23	69.91	53.99	39.11	63.13
OLMoE-1B-7B-0125	A1B/7B	54.11	19.67	66.44	84.30	72.97	62.82	36.54	71.16	65.15	50.28	30.74	55.83
Yulan-Mini-2.4B	2.4B	51.74	23.68	65.08	82.54	78.47	66.18	59.92	76.22	63.31	62.35	44.12	61.24
SmolLM3-3B-Base	3B	62.24	39.02	79.32	89.95	84.80	72.73	70.23	81.12	70.62	64.25	37.52	68.35
MiniCPM5-1B-Base	1B	45.01	21.98	46.44	70.90	73.12	56.92	51.51	71.87	50.15	54.06	36.51	52.59
MobileLLM-R1-950M-base	950M	48.24	21.95	58.31	78.13	78.87	61.59	54.16	73.50	59.77	55.33	34.15	56.73
Ours													
Puro-2B ($4.4K)	2B	54.82	24.02	66.78	84.66	77.34	67.65	63.62	75.84	62.90	59.91	41.14	61.70
Puro-2B	2B	57.44	25.81	74.24	86.07	76.85	67.81	63.21	75.52	63.61	60.62	42.05	63.02

Reasoning and knowledge. Table 5 shows that PURO-2B achieves a mean score of 63.02 over the eleven reasoning and knowledge benchmarks. It outperforms Qwen2-1.5B by 2.48 points and is within 2.51 points of Qwen2.5-1.5B. Within the open-recipe group, PURO-2B surpasses OLMoE-A1B/7B, Yulan-Mini-2.4B, MiniCPM5-1B-Base, and MobileLLM-R1-950M-base, while trailing the larger Instella-3B by only 0.11 points. SmolLM3-3B-Base achieves a higher average, but it is also a larger model. These aggregate results show that PURO-2B remains competitive with larger open-recipe models and retains a 6.29-point reasoning/knowledge advantage over MobileLLM-R1. Together with the reduced training cost enabled by low-precision training, this performance indicates a favorable balance between broad capability and replication cost.

4.3 Cost Estimation

4.3.1 Cost-Saving Factors

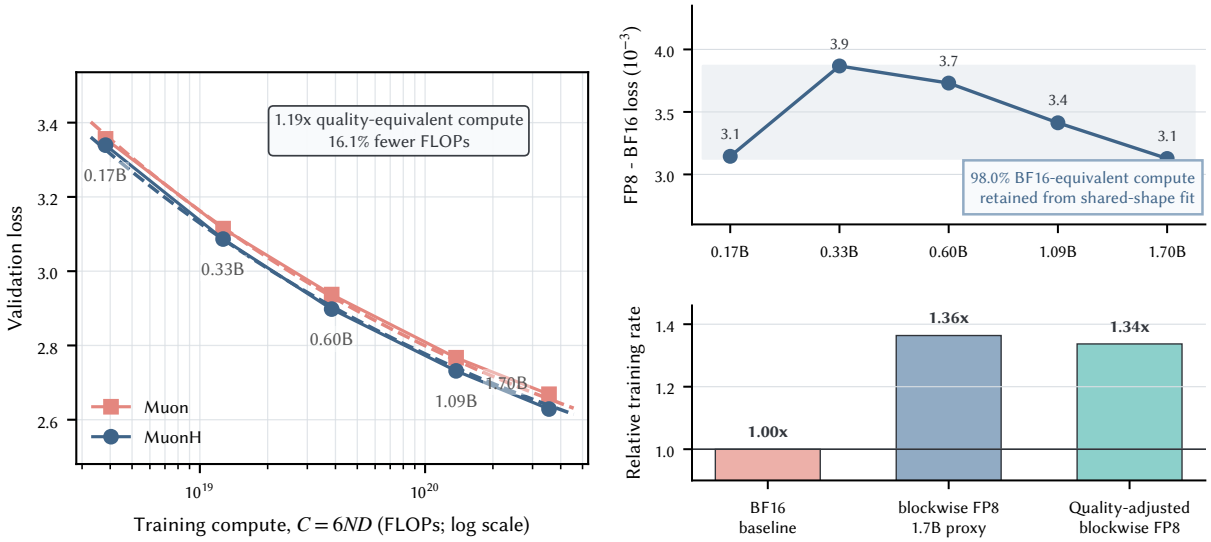
We estimate four cost factors separately: hardware price efficiency, FP8 throughput, MuonH quality-equivalent compute, and the joint Phase 2 recipe. Each factor uses its own reference comparison, so the reported values are not measurements of one sequential end-to-end speedup. The product of these components can only be treated as illustrative transfers. Hardware uses a specification-and-price accounting proxy, blockwise FP8 uses quality-adjusted throughput with MuonH fixed, and the complete MuonH recipe uses quality-equivalent compute with blockwise FP8 fixed. Phase 2 first fits a scaling law of compute cost using the UD recipe (Figure 2b), then measures the equivalent costs of the UD recipe for the CD and canonical CMA endpoints to quantify the corresponding gains. These comparisons do not form a joint factorial ablation due to different references, so their products are accounting estimates rather than directly measured end-to-end speedups[7].

For the ladder evidence below, $C = 6ND$ denotes theoretical training compute, where N is the scaling parameter count and D is the number of training tokens. A compute-equivalent multiplier κ means that a variant trained with C/κ reaches the fitted loss of its baseline at C. For precision, we define BF16-equivalent compute retention

[7]For example, MuonH ablation and FP8 ablation use token-per-parameter of 20 (TPP=20) while the production training is an overtrained setup (TPP=700). We run the alternative compute-optimal setup due to limited compute.

as $\rho_C = C_{\text{BF16}}/C_{\text{FP8}}$ at matched fitted loss; this converts the vertical precision gap into a horizontal compute adjustment.

RTX 5090 exhibits a high cost-performance ratio and utilization. Under the specifications and prices in Table 2, the RTX 5090 proxy provides 2.77× the peak BF16 compute per unit price of H200 and 2.74× the peak FP8 compute per unit price of H200. Moreover, with training configuration tuning, as shown in Section 3.1.3, our recipe can achieve 73% MFU in mixed-precision training, making good use of the RTX 5090's capability.



(a) Complete MuonH versus the tuned Muon baseline.

(b) Blockwise FP8 quality and throughput. The upper figure shows the FP8 validation loss penalty across 5 scales; the lower figure shows wall-time speedup and net benefit of FP8 after considering the loss penalty.

Figure 10: Matched scaling-ladder evidence behind two cost factors. **Left:** Validation loss versus theoretical compute for tuned Muon and MuonH recipes with blockwise FP8 fixed; dashed curves are the shared-floor, shared-exponent fit. The boxed note reports a fitted quality-equivalent compute multiplier. **Right:** With MuonH fixed, the FP8-minus-BF16 validation-loss gap remains within 0.0031–0.0039 across five scales. Only the 1.7B throughput ratio is used as the 2B proxy; its 1.36× measured gain is multiplied by 98.0% fitted BF16-equivalent compute retention to obtain a 1.34× matched-quality speedup, where $\rho_C = C_{\text{BF16}}/C_{\text{FP8}}$ is derived from the shared-shape loss fit. Complete setups and fit diagnostics remain in Section D.

MuonH provides a quality-equivalent compute shift in the matched ladder. Under the definition above, the quality-equivalent compute reduction is $1 - 1/\kappa$. The matched scaling ladder in Figure 10a estimates a 1.19× compute-equivalent multiplier for the complete MuonH recipe relative to Muon, corresponding to 16.1% less theoretical compute at matched validation loss. The scaling estimation is performed over TPP=20 ladder with a shared-shape fit to transfer to production run. At the production run of 2B parameters and 1.4T tokens, using this efficiency estimation, the MuonH optimizer can use 1.41×10^{22} FLOPs to match 1.68×10^{22} FLOPs with the Muon baseline.

Blockwise FP8 retains a stable quality fraction while accelerating the 2B proxy. A precision format may change both execution rate and model quality, so its net efficiency multiplier is the product of throughput speedup and effective-compute retention. As summarized in Figure 10b, blockwise FP8 has a small validation-loss gap at all five ladder scales. The shared-shape fit gives $\rho_C = 98.0\%$. Matching BF16 quality therefore requires approximately 2.0% additional nominal compute. At 1.7B, the closest throughput proxy to the 2B production model, median throughput improves by 1.36×; charging the quality penalty yields a 1.34× net speedup, or 25.2% fewer GPU-hours at matched quality. Then, by this estimation, the 2B-parameter, 1.4T-token canonical run would consequently use 22,654 FP8 GPU-hours to achieve an equivalent performance using approximately 30,286 BF16 GPU-hours.

The Puro Cost Scaling Law summarizes recipe-specific scale-down behavior. Starting from the same Phase 1 checkpoint, we continue Phase 2 training with UD at several budgets and use these runs to characterize

the empirical cost–performance relationship, which we refer to as the Puro Cost Scaling Law. It is a scale-down relationship for this recipe, not a universal law across model families. Here, we focus on the scale-down trend to help the community with limited compute and time budgets decide a feasible performance target. We fit $P = a + b\log_2(C - C_{P1})$, where P is the unweighted arithmetic mean over the 15 benchmarks reported in Tables 4 and 5, C is total reproduction cost, and a and b are the fitted intercept and slope. The checkpoint-level values of P used in the fit are reported in Table 19. Costs shown in the figure use the USD conversion in Table 6. We set C_{P1} = USD 1.84K from the measured Phase 1 active-training GPU-hours and hold it fixed while fitting performance against the incremental Phase 2 cost. The observed UD checkpoint at approximately $4.4K already exceeds Qwen2-1.5B without CMA. This shifted fit reduces in-sample RMSE from 0.452 for the unshifted log-cost fit to 0.209. Inverting it at the CD endpoint score places its UD cost equivalent at approximately $11.36K, or 1.65× the production recipe's $6.89K rental-equivalent estimate from measured active-training GPU-hours. Inverting the same fit at the reported Puro-2B aggregate, which additionally includes a constant-LR continuation and averages the last six checkpoints, gives approximately $16.55K, or 2.40× the production cost. The canonical CMA endpoint optimizes ordering, continuation, and checkpoint averaging together. We therefore report its fitted cost equivalent as a recipe-level point estimate rather than as a curriculum-only gain or a universal cost law.

4.3.2 Cross-Model Reproduction Cost

Estimation protocol. For Figure 1, we prioritize reported monetary cost or accelerator usage. We use reported GPU-hours, or GPU-hours precisely derived from other reported statistics, and convert them with the reference rental rates listed in Table 6 in the appendix. When no such GPU-hour basis is available, we use the token-based route and estimate compute as $C = 6ND$, using the reported accelerator and MFU when available and H100-equivalent GPU-hours at 70% MFU otherwise. Models that disclose neither source are omitted. For PURO-2B, we use measured active-training GPU-hours: 6,009.46 GPU-hours for Phase 1 and 16,504.95 GPU-hours for Phase 2. We convert their sum using the RTX 5090 rate in Table 6. Across all models, this hierarchy distinguishes reproduction-cost estimates based on reported monetary costs or accelerator usage (*reported compute*) from token-based estimates (*estimated compute*). The complete coordinate construction, including the treatment of special cases, and the Pareto-frontier procedure are given in Section B.

Pareto-frontier result. Under the stated accelerator-cost assumptions, PURO-2B appears above and to the left of the comparison-model frontier in Figure 1, combining higher average performance with a lower estimated replication cost. This advantage is clearest relative to the open-recipe models.

4.4 Post-Training Results and Analysis

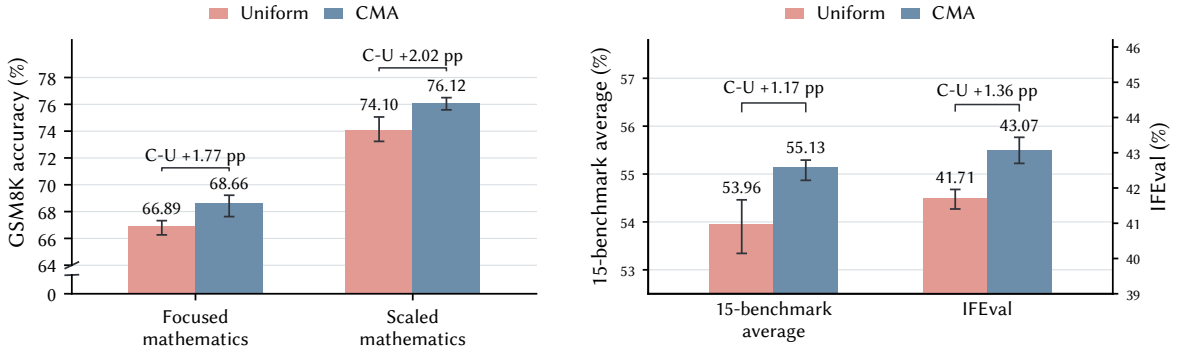
We conduct an ablation study of the two Phase 2 designs to examine how the pretraining recipe affects the results after SFT. The uniform data ordering with learning-rate decay (UD) recipe trains the model on globally reshuffled Phase 2 tokens, while the curriculum model averaging (CMA) recipe follows a data curriculum, keeps the learning rate constant near the end of training, and averages the final checkpoints (Section 3.4). We apply the same SFT procedure to the two pretraining checkpoints and compare the resulting UD-based and CMA-based models. The experimental setup is detailed in Section 3.5 and we analyze the results in this section.

GSM8K-based SFT. The GSM8K-based SFT experiment compares the UD and CMA checkpoints without pretraining replay. After SFT, the UD-based model reaches 66.89% mean GSM8K accuracy, while the CMA-based model reaches 68.66%. On average over three random seeds, the CMA-based model is ahead by 1.77 percentage points (Figure 11a and Table 13). We use flexible answer extraction to limit score differences caused by output formatting. We then compare the sets of questions answered correctly by the two models. Under this protocol, the CMA-based model still solves more questions that the UD-based model misses. The remaining advantage is therefore better explained by stronger mathematical capability than by answer formatting.

Math&Code SFT with replay. The Math&Code SFT with replay experiment extends the comparison to a larger mathematics mixture, together with code data and a small replay component. It also uses a longer SFT process. One possible explanation for the base-model difference is a temporary advantage from placing higher-quality data near the end of pretraining. A longer SFT process on a different data distribution tests whether this advantage persists. After this longer SFT process, the UD-based model reaches 74.10% mean GSM8K accuracy, while the CMA-based model reaches 76.12%. On average over three random seeds, the CMA-based model is ahead by 2.02 percentage points (Tables 4 and 14). The CMA advantage therefore remains after more SFT updates on a broader data distribution. The gap is also slightly larger than in the GSM8K-based SFT setting.

Tulu-3 mixed-domain SFT. The first two setups focus primarily on mathematics. To examine whether the difference extends beyond this domain, we use Tulu-3 mixed-domain SFT and evaluate the resulting models on 15 benchmarks (Tables 4 and 5). As shown in Figure 11b, the CMA-based model scores 1.17 points higher on average and performs better on 10 of the 15 benchmarks. The aggregate advantage therefore extends beyond mathematics, although the gains are not uniform across tasks. The task-level results show a different balance across capabilities rather than a uniform improvement. We separately evaluate instruction following on IFEval, where the CMA-based model scores 1.36 points higher than the UD-based model. The results are averaged over three repeated runs, which show the same overall trend. Further details are provided in Section E.

Taken together, the three experiments show that the difference between the two pretraining recipes remains after increasingly broad SFT. The CMA-based models retain stronger mathematics performance in both the GSM8K-based SFT and Math&Code SFT with replay settings. They also maintain a higher aggregate score after Tulu-3 mixed-domain SFT and perform better on instruction following. Because the task-level gains are uneven, these results do not imply that CMA improves every capability uniformly. This variation makes more controlled curriculum design an important direction for future work.



(a) GSM8K-based SFT and Math&Code SFT with replay.

(b) Tulu-3 mixed-domain SFT.

Figure 11: Post-training results. **(a)** Final GSM8K results under GSM8K-based SFT and Math&Code SFT with replay. Bars report the average GSM8K accuracy over three repeated runs, with error bars spanning the minimum and maximum. UD (uniform data with LR decay) and CMA denote SFT initialized from the corresponding pretraining checkpoints. **(b)** Tulu-3 mixed-domain SFT summary. The two groups show the average over 15 benchmarks in Tables 4 and 5 and the additional IFEval score. Each group contains paired results. Detailed task-level results in Table 17.

5 Related Works

5.1 Open-Recipe Language Models

The core science and engineering of large-scale pretraining remain difficult to study when leading systems are either closed or release weights without the data and training recipe needed to reconstruct the run [55]. The term "open model" therefore covers substantially different release practices. Widely used model families such as Qwen [103, 104, 102, 80], Gemma [81, 31, 32], Llama [60], LFM [50], and Falcon [121] publish model weights and enough implementation information for evaluation and deployment. However, their exact pretraining corpora, sample order, and complete training state are not released, so an independent group cannot reconstruct the original training run. We refer to these releases as *open-weight*. In contrast, a *open-recipe* release aims to expose the model weights, training data or a reconstructible data recipe, training and evaluation code, and sufficiently detailed configurations and intermediate artifacts to support scientific inspection and reproduction [35, 95]. Such releases turn a final checkpoint into a reproducible account of how the model was built, making controlled study possible for resource-limited research communities.

Open-recipe projects provide the broader experimental stack needed to study pretraining itself. Pythia releases ordered data and dense intermediate checkpoints; OLMo and OLMoE release data, training and evaluation code, checkpoints, and logs; and projects such as SmolLM3, Yulan-Mini-2.4B, and Instella provide public data mixtures and reproducible training configurations [10, 35, 64, 7, 108, 52]. These resources allow other researchers to inves-

tigate learning dynamics, data selection, scaling, and optimization without relying on undocumented industrial pipelines. Open-recipe models can also be competitive: as shown in Tables 4 and 5, models such as SmolLM3-3B, Yulan-Mini-2.4B, and Instella-3B match or surpass several open-weight models of comparable scale on parts of our evaluation, making this direction important beyond reproducibility alone.

This openness and capability have nevertheless often required substantial resources. Yulan-Mini-2.4B was trained on 48 A800 GPUs, SmolLM3 on 384 H100 GPUs, and the first stage of Instella-3B on 128 MI300X GPUs [108, 7, 52]. Kaiyuan-2B, our previous work, provides a more resource-conscious example by improving data and training efficiency on Ascend 910A hardware [55]. These examples expose a remaining gap between openness and practical accessibility: a training pipeline may be fully reproducible yet remain too costly for small research groups to rerun. Addressing this gap requires considering training workload, numerical precision, hardware accessibility, and explicit cost accounting together.

5.2 Low-Cost Language Model Pretraining

Training cost is meaningful only under an explicit accounting boundary. Token count and FLOPs alone capture only part of pretraining cost. End-to-end replication cost also depends on the workload required to reach a target quality, realized hardware throughput, and hardware price. We therefore consider cost jointly with model quality rather than using any single measure of compute or hardware efficiency as a proxy for low-cost training.

Low-precision training primarily reduces the cost of executing a fixed workload. FP8 mixed precision accelerates the matrix multiplications that dominate Transformer training while retaining higher precision where needed. DeepSeek-V3 shows that this design can remain stable at large scale [63, 26]. Quartet II extends this direction to NVFP4 and improves its numerical behavior [73]. These methods improve execution efficiency for a given training workload, but do not by themselves reduce the token or nominal FLOP budget required to reach a target quality.

A complementary line of work reduces the training workload through model architecture, data selection, staged training, and systems optimization. TinyLlama, JetMoE, Yulan-Mini-2.4B, and Kaiyuan-2B explore different combinations of these techniques [113, 85, 108, 55]. Collectively, these projects show that useful small-model capability can be obtained under substantially smaller compute budgets than those of large-scale foundation models. The reported runs, however, generally rely on specialized data-center accelerators, so the hardware barrier for a small research group remains.

Hardware accessibility introduces a separate constraint. QLoRA reduces the memory required to fine-tune a pretrained model, leaving the original pretraining cost unchanged [28]. LLMQ studies full-model pretraining on RTX 4090 GPUs using FP8, recomputation, sharding, and CPU offloading [84], while Quartet II measures the throughput benefit of NVFP4 on a single RTX 5090 [73]. The studies considered here establish the feasibility of relevant training operations on consumer GPUs, but focus primarily on systems feasibility or throughput rather than on releasing a trillion-token-scale base model with an explicit replication-cost boundary.

PURO-2B lies at the intersection of these directions. It is a from-scratch 2B base model trained on more than 1.4T tokens using FP8 on a multi-node RTX 5090 cluster, with measured accelerator usage and an explicit replication-cost boundary. We therefore evaluate the training recipe and hardware platform end to end, measuring both the resulting model quality and replication cost. Together, these components provide researchers with a reproducible and cost-accessible end-to-end pipeline for studying pretraining behavior and testing alternative methods at meaningful scales.

6 Conclusion and Future Direction

In this report, we present a cost-efficient, hardware-accessible reproducibility recipe for language model pretraining, and use it to train the PURO-2B model collection from scratch on consumer-grade RTX 5090 GPUs. Across different training budgets and recipe variants, the collection provides a direct view of the cost–performance tradeoff of our pipeline. A model from this collection reaches the performance of Qwen2-1.5B at about $4.4K, while our best checkpoint approaches Qwen2.5-1.5B at the canonical accelerator-only cost of about $6.9K. Based on these runs, we further formulate the *Puro Cost Scaling Law* to characterize how model capability scales down with training cost under our recipe. More broadly, these results demonstrate that useful from-scratch pretraining at the billion-parameter scale is feasible with modest budgets and accessible hardware.

There remain several directions for extending this foundation. First, we plan to extend the reproducibility recipe beyond pretraining to post-training, with particular emphasis on developing and studying agentic capabilities. Second, we plan to broaden the architectural space beyond standard dense Transformers, including looped Trans-

formers, linear-attention models, mixture-of-experts architectures, and other emerging designs. Third, we aim to expand the hardware recipes beyond RTX 5090 GPUs and cover a wider range of training budgets, making it easier to study how the optimal system and training choices change across hardware and scale. We view PURO-2B not as an endpoint, but as evidence that affordable and inspectable model training is practical today, and as a baseline for future efforts to push the frontier toward lower cost, broader accessibility, and higher efficiency.

7 Acknowledgments

The authors would like to thank Yanfu Investments for providing computational resources. Zhenbo Sun and Chenyi Dang contributed during the early stages of this work. The authors also thank Kaiyue Wen, Kexian Tang, Minxing Yang, Zhan Ling, Haodong Wen, Shaowen Wang, Huaqing Zhang and Shuo Huang for their valuable feedback and insightful discussions, and Yanzheng Cai, Yuanwei Wang, Zhixuan Pan, Rui Chen, and Nuo Chen for their assistance with proofreading and manuscript revision. Multiple LLM-based tools were used to assist with drafting, proofreading, and language refinement.

References

[1] aikitoria. 2026. NVIDIA Linux open GPU with P2P support. `https://github.com/aikitoria/open-gpu-kernel-modules`.

[2] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlícek, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM2: When Smol Goes Big - Data-Centric Training of a Small Language Model. *CoRR* abs/2502.02737 (2025). arXiv:2502.02737 doi:`10.48550/ARXIV.2502.02737`

[3] AMD. 2025. Instella-3B Model Card. `https://huggingface.co/amd/Instella-3B` Accessed: 2026-08-20.

[4] Michael Andersch, Greg Palmer, Ronny Krashinsky, Nick Stam, Vishal Mehta, Gonzalo Brito, and Sridhar Ramaswamy. 2022. NVIDIA Hopper Architecture In-Depth. `https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/`.

[5] Alpay Ariyak, Junda Zhang, Junxiong Wang, Shang Zhu, Federico Bianchi, Sanjana Srivastava, Ashwinee Panda, Siddhant Bharti, Chenfeng Xu, John Heo, Xiaoxia Shirley Wu, James Zhou, Percy Liang, Leon Song, Ce Zhang, Ben Athiwaratkun, Zhongzhu Zhou, and Qingyang Wu. 2026. CoderForge-Preview: SOTA Open Dataset for Training Efficient Agents. `https://www.together.ai/blog/coderforge-preview` Project core leads: Alpay Ariyak; Zhongzhu Zhou; Qingyang Wu.

[6] Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *CoRR* abs/2108.07732 (2021). arXiv:2108.07732 `https://arxiv.org/abs/2108.07732`

[7] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlicek, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. 2025. SmolLM3: smol, multilingual, long-context reasoner. `https://huggingface.co/blog/smollm3`.

[8] Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, Alex Kondratenko, Alex Shaposhnikov, Alexander Bukharin, Ali Taghibakhshi, Amelia Barton, Ameya Sunil Mahabaleshwarkar, Amy Shen, Andrew Tao, Ann Guan, Anna Shors, Anubhav Mandarwal, Arham Mehta, Arun Venkatesan, Ashton Sharabiani, Ashwath Aithal, Ashwin Poojary, Ayush Dattagupta, Balaram Buddharaju, Banghua Zhu, Barnaby Simkin, Bilal Kartal, Bita Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Brian Yu, Bryan Catanzaro, Charles Wang, Charlie Truong, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Christian Munley, Christopher Parisien, Dan Su, Daniel Afrimi, Daniel Korzekwa, Daniel Rohrer, Daria Gitman, David Mosallanezhad, Deepak Narayanan, Dima Rekesh, Dina Yared, Dmytro Pykhtar, Dong Ahn, Duncan Riach, Eileen Long, Elliott Ning, Eric Chung, Erick Galinkin, Evelina Bakhturina, Gargi

Prasad, Gerald Shen, Haifeng Qian, Haim Elisha, Harsh Sharma, Hayley Ross, Helen Ngo, Herman Sahota, Hexin Wang, Hoo Chang Shin, Hua Huang, Iain Cunningham, Igor Gitman, Ivan Moshkov, Jaehun Jung, Jan Kautz, Jane Polak Scowcroft, Jared Casper, Jian Zhang, Jiaqi Zeng, Jimmy Zhang, Jinze Xue, Jocelyn Huang, Joey Conway, John Kamalu, Jonathan M. Cohen, Joseph Jennings, Julien Veron Vialard, Junkeun Yi, Jupinder Parmar, Kari Briski, Katherine Cheung, Katherine Luna, Keith W. Ross, Keshav Santhanam, Kezhi Kong, Krzysztof Pawelec, and Kumar Anik. 2025. NVIDIA Nemotron Nano 2: An Accurate and Efficient Hybrid Mamba-Transformer Reasoning Model. *CoRR* abs/2508.14444 (2025). arXiv:2508.14444 doi:10.48550/ARXIV.2508.14444

[9] Loubna Ben Allal, Anton Lozhkov, Guilherme Penedo, Thomas Wolf, and Leandro von Werra. 2024. *SmolLM-Corpus*. https://huggingface.co/datasets/HuggingFaceTB/smollm-corpus

[10] Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. 2023. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 2397–2430. https://proceedings.mlr.press/v202/biderman23a.html

[11] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. 2020. PIQA: Reasoning about Physical Commonsense in Natural Language. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 7432–7439. doi:10.1609/AAAI.V34I05.6239

[12] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374 https://arxiv.org/abs/2107.03374

[13] Shengqi Chen. 2026. 在 RTX 5090 上启用 GPUDirect RDMA 通信支持 - Harry Chen's Blog. https://harrychen.xyz/2026/05/20/enable-gpudirect-rdma-on-rtx-5090/. Accessed: 2026-08-27.

[14] Shengqi Chen. 2026. 在 RTX 5090 上启用 PCIe P2P 通信支持 - Harry Chen's Blog. https://harrychen.xyz/2026/03/22/enable-pcie-p2p-on-rtx-5090/. Accessed: 2026-08-27.

[15] Yukang Chen, Shaozuo Yu, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2023. Long Alpaca: Long-context Instruction-following models. https://github.com/dvlab-research/LongLoRA.

[16] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 2924–2936. doi:10.18653/v1/N19-1300

[17] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *CoRR* abs/1803.05457 (2018). arXiv:1803.05457 http://arxiv.org/abs/1803.05457

[18] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *CoRR* abs/2110.14168 (2021). arXiv:2110.14168 https://arxiv.org/abs/2110.14168

[19] Baptiste Colle, Hanna Yukhymenko, and Leandro von Werra. 2025. Jupyter Agent Dataset.

[20] Together Computer. 2023. *RedPajama: An Open Source Recipe to Reproduce LLaMA training dataset.* https://github.com/togethercomputer/RedPajama-Data

[21] OpenCompass Contributors. 2023. OpenCompass: A Universal Evaluation Platform for Foundation Models. https://github.com/open-compass/opencompass.

[22] NVIDIA Corporation. 2024. NVIDIA H200 Tensor Core GPU. https://resources.nvidia.com/en-us-hopper-architecture/hpc-datasheet-sc23.

[23] NVIDIA Corporation. 2025. NVIDIA RTX Blackwell GPU Architecture. https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf.

[24] NVIDIA Corporation. 2025. NVIDIA RTX Pro Blackwell GPU Architecture. https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/quadro-product-literature/NVIDIA-RTX-Blackwell-PRO-GPU-Architecture-v1.0.pdf.

[25] Yiming Cui, Ziqing Yang, and Xin Yao. 2023. Efficient and Effective Text Encoding for Chinese LLaMA and Alpaca. *arXiv preprint arXiv:2304.08177* (2023). https://arxiv.org/abs/2304.08177

[26] DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, and Wangding Zeng. 2024. DeepSeek-V3 Technical Report. *CoRR* abs/2412.19437 (2024). arXiv:2412.19437 doi:10.48550/ARXIV.2412.19437

[27] DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiaxin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao

Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yuduan Wang, Yuehan Yang, Yuer Xu, Yuhan Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 [cs.CL] https://arxiv.org/abs/2606.19348

[28] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. QLoRA: Efficient Finetuning of Quantized LLMs. In *Advances in Neural Information Processing Systems*, Vol. 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html

[29] Kazuki Fujii, Yukito Tajima, Sakae Mizuki, Hinari Shimada, Taihei Shiotani, Koshiro Saito, Masanari Ohi, Masaki Kawamura, Taishi Nakamura, Takumi Okamoto, Shigeki Ishida, Kakeru Hattori, Youmi Ma, Hiroya Takamura, Rio Yokota, and Naoaki Okazaki. 2025. Rewriting Pre-Training Data Boosts LLM Performance in Math and Code. arXiv:2505.02881 [cs.LG] https://arxiv.org/abs/2505.02881

[30] Gemini Team. 2025. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities. *CoRR* abs/2507.06261 (2025). arXiv:2507.06261 doi:10.48550/ARXIV.2507.06261

[31] Gemma Team. 2025. Gemma 3 Technical Report. *CoRR* abs/2503.19786 (2025). arXiv:2503.19786 doi:10.48550/ARXIV.2503.19786

[32] Gemma Team. 2026. Gemma 4 Technical Report. *CoRR* abs/2607.02770 (2026). arXiv:2607.02770 doi:10.48550/ARXIV.2607.02770

[33] Google. 2024. Gemma 2 Model Card. https://ai.google.dev/gemma/docs/core/model_card_2 Accessed: 2026-08-20.

[34] Google. 2025. Gemma 3 Model Card. https://ai.google.dev/gemma/docs/core/model_card_3 Accessed: 2026-08-20.

[35] Dirk Groeneveld, Iz Beltagy, Evan Walsh, Akshita Bhagia, Rodney Kinney, Oyvind Tafjord, Ananya Jha, Hamish Ivison, Ian Magnusson, Yizhong Wang, Shane Arora, David Atkinson, Russell Authur, Khyathi Chandu, Arman Cohan, Jennifer Dumas, Yanai Elazar, Yuling Gu, Jack Hessel, Tushar Khot, William Merrill, Jacob Morrison, Niklas Muennighoff, Aakanksha Naik, Crystal Nam, Matthew Peters, Valentina Pyatkin, Abhilasha Ravichander, Dustin Schwenk, Saurabh Shah, William Smith, Emma Strubell, Nishant Subramani, Mitchell Wortsman, Pradeep Dasigi, Nathan Lambert, Kyle Richardson, Luke Zettlemoyer, Jesse Dodge, Kyle Lo, Luca Soldaini, Noah Smith, and Hannaneh Hajishirzi. 2024. OLMo: Accelerating the Science of Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 15789–15809. doi:10.18653/v1/2024.acl-long.841

[36] Yuling Gu, Oyvind Tafjord, Bailey Kuehl, Dany Haddad, Jesse Dodge, and Hannaneh Hajishirzi. 2025. OLMES: A Standard for Language Model Evaluations. In *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, 5020–5048. doi:10.18653/V1/2025.FINDINGS-NAACL.282

[37] David Hall, Ahmed Ahmed, Christopher Chou, Abhinav Garg, Rohith Kuditipudi, Will Held, Nikil Ravi, Herumb Shandilya, Jason Wang, Jason Bolton, Siddharth Karamcheti, Suhas Kotha, Tony Lee, Nelson Liu, Joel Niklaus, Ashwin Ramaswami, Kamyar Salahi, Kaiyue Wen, Chi Heem Wong, Sherry Yang, Ivan Zhou, and Percy Liang. 2025. Introducing Marin: An Open Lab for Building Foundation Models. Marin Community Blog. https://marin.community/blog/2025/05/19/announcement/ Blog post.

[38] Conghui He, Zhenjiang Jin, Chao Xu, Jiantao Qiu, Bin Wang, Wei Li, Hang Yan, Jiaqi Wang, and Dahua Lin. 2023. WanJuan: A Comprehensive Multimodal Dataset for Advancing English and Chinese Large Models. arXiv:2308.10755 [cs.CL] https://arxiv.org/abs/2308.10755

[39] Conghui He, Wei Li, Zhenjiang Jin, Chao Xu, Bin Wang, and Dahua Lin. 2024. OpenDataLab: Empowering General Artificial Intelligence with Open Datasets. arXiv:2407.13773 [cs.DL] https://arxiv.org/abs/2407.13773

[40] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring Massive Multitask Language Understanding. In *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net. https://openreview.net/forum?id=d7KBjmI3GmQ

[41] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, Joaquin Vanschoren and Sai-Kit Yeung (Eds.). https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/be83ab3ecd0db773eb2dc1b0a17836a1-Abstract-round2.html

[42] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Thomas Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karén Simonyan, Erich Elsen, Oriol Vinyals, Jack Rae, and Laurent Sifre. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 30016–30030. https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf

[43] Shengding Hu, Yuge Tu, Xu Han, Ganqu Cui, Chaoqun He, Weilin Zhao, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Xinrong Zhang, Zhen Leng Thai, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, dahai li, Zhiyuan Liu, and Maosong Sun. 2024. MiniCPM: Unveiling the Potential of Small Language Models with Scalable Training Strategies. In *First Conference on Language Modeling*. https://openreview.net/forum?id=3X2L2TFr0f

[44] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *CoRR* abs/2001.08361 (2020). arXiv:2001.08361 https://arxiv.org/abs/2001.08361

[45] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing Activation Recomputation in Large Transformer Models. In *Proceedings of the Sixth Conference on Machine Learning and Systems, MLSys 2023, Miami, FL, USA, June 4-8, 2023*, Dawn Song, Michael Carbin, and Tianqi Chen (Eds.). mlsys.org. https://proceedings.mlsys.org/paper_files/paper/2023/hash/80083951326cf5b35e5100260d64ed81-Abstract-mlsys2023.html

[46] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Oyvind Tafjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. 2024. Tülu 3: Pushing Frontiers in Open Language Model Post-Training. *CoRR* abs/2411.15124 (2024). arXiv:2411.15124 doi:10.48550/arXiv.2411.15124

[47] Haoyang Li, Zhanchao Xu, Yiming Li, Xuejia Chen, Darian Li, Anxin Tian, Qingfa Xiao, Cheng Deng, Jun Wang, Qing Li, Lei Chen, and Mingxuan Yuan. 2025. LoopServe: An Adaptive Dual-phase LLM Inference Acceleration System for Multi-Turn Dialogues. arXiv:2507.13681 [cs.CL] https://arxiv.org/abs/2507.13681

[48] Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. NuminaMath. [https://huggingface.co/AI-MO/NuminaMath-CoT](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf).

[49] Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Yitzhak Gadre, Hritik Bansal, Etash Kumar Guha, Sedrick Scott Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee F. Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruba Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah M. Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Raghavi Chandu, Thao Nguyen, Igor Vasiljevic, Sham M. Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alex Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. 2024. DataComp-LM: In search of the next generation of training sets for language models. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (Eds.). http://papers.nips.cc/paper%5Ffiles/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets%5Fand%5FBenchmarks%5FTrack.html

[50] Liquid AI. 2025. LFM2 Technical Report. *CoRR* abs/2511.23404 (2025). arXiv:2511.23404 doi:10.48550/ARXIV.2511.23404

[51] Liquid AI. 2026. Introducing LFM2.5: The Next Generation of On-device AI. https://www.liquid.ai/blog/introducing-lfm2-5-the-next-generation-of-on-device-ai Accessed: 2026-08-20.

[52] Jiang Liu, Jialian Wu, Xiaodong Yu, Yusheng Su, Prakamya Mishra, Gowtham Ramesh, Sudhanshu Ranjan, Chaitanya Manem, Ximeng Sun, Ze Wang, Pratik Prabhanjan Brahma, Zicheng Liu, and Emad Barsoum. 2025. Instella: Fully Open Language Models with Stellar Performance. *CoRR* abs/2511.10628 (2025). arXiv:2511.10628 doi:10.48550/ARXIV.2511.10628

[53] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. 2023. The Flan Collection: Designing Data and Methods for Effective Instruction Tuning. arXiv:2301.13688 [cs.AI] https://arxiv.org/abs/2301.13688

[54] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, Tianyang Liu, Max Tian, Denis Kocetkov, Arthur Zucker, Younes Belkada, Zijian Wang, Qian Liu, Dmitry Abulkhanov, Indraneil Paul, Zhuang Li, Wen-Ding Li, Megan Risdal, Jia Li, Jian Zhu, Terry Yue Zhuo, Evgenii Zheltonozhskii, Nii Osae Osae Dade, Wenhao Yu, Lucas Krauß, Naman Jain, Yixuan Su, Xuanli He, Manan Dey, Edoardo Abati, Yekun Chai, Niklas Muennighoff, Xiangru Tang, Muhtasham Oblokulov, Christopher Akiki, Marc Marone, Chenghao Mou, Mayank Mishra, Alex Gu, Binyuan Hui, Tri Dao, Armel Zebaze, Olivier Dehaene, Nicolas Patry, Canwen Xu, Julian McAuley, Han Hu, Torsten Scholak, Sebastien Paquet, Jennifer Robinson, Carolyn Jane Anderson, Nicolas Chapados, Mostofa Patwary, Nima Tajbakhsh, Yacine Jernite, Carlos Mu noz Ferrandis, Lingming Zhang, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2024. StarCoder 2 and The Stack v2: The Next Generation. arXiv:2402.19173 [cs.SE] https://arxiv.org/abs/2402.19173

[55] Kairong Luo, Zhenbo Sun, Xinyu Shi, Shengqi Chen, Bowen Yu, Yunyi Chen, Chenyi Dang, Hengtao Tao, Hui Wang, Fangming Liu, Kaifeng Lyu, and Wenguang Chen. 2025. PCMind-2.1-Kaiyuan-2B Technical Report. arXiv:2512.07612 [cs.CL] https://arxiv.org/abs/2512.07612

[56] Kairong Luo, Zhenbo Sun, Haodong Wen, Xinyu Shi, Jiarui Cui, Chenyi Dang, Kaifeng Lyu, and Wenguang Chen. 2026. How Learning Rate Decay Wastes Your Best Data in Curriculum-Based LLM Pretraining. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=T5wkZJqzkz

[57] Kairong Luo, Haodong Wen, Shengding Hu, Zhenbo Sun, Zhiyuan Liu, Maosong Sun, Kaifeng Lyu, and Wenguang Chen. 2025. A Multi-Power Law for Loss Curve Prediction Across Learning Rate Schedules. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=KnoS9XxIlK

[58] Rabeeh Karimi Mahabadi, Sanjeev Satheesh, Shrimai Prabhumoye, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. 2025. Nemotron-cc-math: A 133 billion-token-scale high quality math pretraining dataset. *arXiv preprint arXiv:2508.15096* (2025).

[59] Meta AI. 2024. Llama 3.2 Model Card. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md Accessed: 2026-08-20.

[60] Meta AI. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[61] Meta AI. 2024. MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases. https://github.com/facebookresearch/MobileLLM Accessed: 2026-08-20.

[62] Meta AI. 2025. MobileLLM-R1 Official Recipe. https://github.com/facebookresearch/MobileLLM-R1 Accessed: 2026-08-20.

[63] Paulius Micikevicius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, Naveen Mellempudi, Stuart Oberman, Mohammad Shoeybi, Michael Siu, and Hao Wu. 2022. FP8 Formats for Deep Learning. *CoRR* abs/2209.05433 (2022). arXiv:2209.05433 doi:10.48550/ARXIV.2209.05433

[64] Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Evan Pete Walsh, Oyvind Tafjord, Nathan Lambert, Yuling Gu, Shane Arora, Akshita Bhagia, Dustin Schwenk, David Wadden, Alexander Wettig, Binyuan Hui, Tim Dettmers, Douwe Kiela, Ali Farhadi, and et al. 2025. OLMoE: Open Mixture-of-Experts Language Models. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net. https://openreview.net/forum?id=xXTkbTBmqq

[65] Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive Learning from Complex Explanation Traces of GPT-4. arXiv:2306.02707 [cs.CL] https://arxiv.org/abs/2306.02707

[66] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, 1–15. doi:10.1145/3458817.3476209

[67] NVIDIA. 2025. NVIDIA Data Agreement for Model Training. https://huggingface.co/datasets/nvidia/Nemotron-Pretraining-Dataset-sample/blob/main/LICENSE.md. [Accessed 03-12-2025].

[68] NVIDIA Corporation. 2026. *Megatron Core*. https://github.com/NVIDIA/Megatron-LM/releases/tag/core_v0.16.0 Official release published 2026-02-26; Git tag core_v0.16.0, commit 3bec9aa97dda898d16ff5a89bac0ed2b6682b172.

[69] NVIDIA Corporation. 2026. Using FP8 and FP4 with Transformer Engine. https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html Transformer Engine 2.17.0 documentation, accessed 2026-08-09.

[70] Beijing Academy of Artificial Intelligence. 2023. Chinese Corpus Internet Usage Agreement. https://data.baai.ac.cn/resources/agreement/cci_usage_aggrement.pdf. [Accessed 03-12-2025].

[71] OpenAI. 2023. GPT-4 Technical Report. *CoRR* abs/2303.08774 (2023). arXiv:2303.08774 doi:10.48550/ARXIV.2303.08774

[72] OpenBMB Team. 2026. MiniCPM5-1B-Base Model Card. https://huggingface.co/openbmb/MiniCPM5-1B-Base

[73] Andrei Panferov, Erik Schultheis, Soroush Tabesh, and Dan Alistarh. 2026. Quartet II: Accurate LLM Pre-Training in NVFP4 by Improved Unbiased Gradient Estimation. *CoRR* abs/2601.22813 (2026). arXiv:2601.22813 doi:10.48550/ARXIV.2601.22813

[74] Keiran Paster, Marco Dos Santos, Zhangir Azerbayev, and Jimmy Ba. 2023. OpenWebMath: An Open Dataset of High-Quality Mathematical Web Text. arXiv:2310.06786 [cs.AI] https://arxiv.org/abs/2310.06786

[75] Guilherme Penedo. 2025. *FineWiki*. https://huggingface.co/datasets/HuggingFaceFW/finewiki Source: Wikimedia Enterprise Snapshot API (https://api.enterprise.wikimedia.com/v2/snapshots). Text licensed under CC BY-SA 4.0 with attribution to Wikipedia contributors..

[76] Guilherme Penedo, Anton Lozhkov, Hynek Kydlíček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. 2025. CodeForces CoTs. https://huggingface.co/datasets/open-r1/codeforces-cots.

[77] Renjie Pi, Grace Lam, Mohammad Shoeybi, Pooya Jannaty, Bryan Catanzaro, and Wei Ping. 2026. On Data Engineering for Scaling LLM Terminal Capabilities. arXiv:2602.21193 [cs.CL] https://arxiv.org/abs/2602.21193

[78] Zhenting Qi, Fan Nie, Alexandre Alahi, James Y. Zou, Himabindu Lakkaraju, Yilun Du, Eric P. Xing, Sham M. Kakade, and Hanlin Zhang. 2025. EvoLM: In Search of Lost Training Dynamics for Language Model Reasoning. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2025, NeurIPS 2025, San Diego, CA, USA, December 2-7, 2025 / Mexico City, Mexico, November 30 - December 5, 2025*, Danielle Belgrave, Cheng Zhang, Laura N. Montoya, Hsuan-Tien Lin, Razvan Pascanu, Piotr Koniusz, Marzyeh Ghassemi, Nancy Chen, Iván Vladimir Meza Ruíz, and Arturo Loaiza-Bonilla (Eds.). http://papers.nips.cc/paper_files/paper/2025/hash/778cc31583604ebf246726055e9f737f-Abstract-Conference.html

[79] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. arXiv:2307.16789 [cs.AI]

[80] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. https://qwen.ai/blog?id=qwen3.5

[81] Morgane Rivière, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagic, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozinska, Dustin Herbison, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshev, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucinska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Juyeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjösund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, and Lilly McNealus. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *CoRR* abs/2408.00118 (2024). arXiv:2408.00118 doi:10.48550/ARXIV.2408.00118

[82] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. 2020. WinoGrande: An Adversarial Winograd Schema Challenge at Scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*. AAAI Press, 8732–8740. doi:10.1609/AAAI.V34I05.6399

[83] Maarten Sap, Hannah Rashkin, Derek Chen, Ronan Le Bras, and Yejin Choi. 2019. Social IQa: Commonsense Reasoning about Social Interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan (Eds.). Association for Computational Linguistics, Hong Kong, China, 4463–4473. doi:10.18653/v1/D19-1454

[84] Erik Schultheis and Dan Alistarh. 2025. LLMQ: Efficient Lower-Precision Pretraining for Consumer GPUs. *CoRR* abs/2512.15306 (2025). arXiv:2512.15306 doi:10.48550/ARXIV.2512.15306

[85] Yikang Shen, Zhen Guo, Tianle Cai, and Zengyi Qin. 2024. JetMoE: Reaching Llama2 Performance with 0.1M Dollars. *CoRR* abs/2404.07413 (2024). arXiv:2404.07413 doi:10.48550/ARXIV.2404.07413

[86] Yikang Shen, Matthew Stallone, Mayank Mishra, Gaoyuan Zhang, Shawn Tan, Aditya Prasad, Adriana Meza Soria, David D Cox, and Rameswar Panda. 2024. Power scheduler: A batch size and token number agnostic learning rate scheduler. *arXiv preprint arXiv:2408.13359* (2024).

[87] Xiaofeng Shi, Lulu Zhao, Hua Zhou, and Donglin Hao. 2024. IndustryCorpus2. Hugging Face dataset. doi:10.57967/hf/3488

[88] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. *CoRR* abs/1909.08053 (2019). arXiv:1909.08053 http://arxiv.org/abs/1909.08053

[89] KaShun SHUM, Yuzhen Huang, Hongjian Zou, dingqi, YiXuan Liao, Xiaoxin Chen, Qian Liu, and Junxian He. 2025. Predictive Data Selection: The Data That Predicts Is the Data That Teaches. In *Forty-second International Conference on Machine Learning*. https://openreview.net/forum?id=tTVYR82Iz6

[90] Skywork-AI. 2023. Skywork Community License. https://huggingface.co/datasets/Skywork/SkyPile-150B/blob/main/Skywork%20Community%20License.pdf. [Accessed 03-12-2025].

[91] Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. 2025. Nemotron-CC: Transforming Common Crawl into a Refined Long-Horizon Pretraining Dataset. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, 2459–2475. https://aclanthology.org/2025.acl-long.123/

[92] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. 2023. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. In *Findings of the Association for Computational Linguistics: ACL 2023*. Association for Computational Linguistics, Toronto, Canada, 13003–13051. doi:10.18653/v1/2023.findings-acl.824

[93] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A Question Answering Challenge Targeting Commonsense Knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4149–4158. doi:10.18653/v1/N19-1421

[94] OpenThoughts-Agent Team. 2025. OpenThoughts-Agent. https://www.open-thoughts.ai/blog/agent.

[95] Evan Pete Walsh, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Shane Arora, Akshita Bhagia, Yuling Gu, Shengyi Huang, Matt Jordan, Nathan Lambert, Dustin Schwenk, Oyvind Tafjord, Taira Anderson, David Atkinson, Faeze Brahman, Christopher Clark, Pradeep Dasigi, Nouha Dziri, Allyson Ettinger, Michal Guerquin, David Heineman, Hamish Ivison, Pang Wei Koh, Jiacheng Liu, Saumya Malik, William Merrill, Lester James Validad Miranda, Jacob Morrison, Tyler Murray, Crystal Nam, Jake Poznanski, Valentina Pyatkin, Aman Rangapur, Michael Schmitz, Sam Skjonsberg, David Wadden, Christopher Wilhelm, Michael Wilson, Luke Zettlemoyer, Ali Farhadi, Noah A. Smith, and Hannaneh Hajishirzi. 2025. 2 OLMo 2 Furious (COLM's Version). In *Second Conference on Language Modeling*. https://openreview.net/forum?id=2ezugTT9kU

[96] Liangdong Wang, Bo-Wen Zhang, Chengwei Wu, Hanyu Zhao, Xiaofeng Shi, Shuhao Gu, Jijie Li, Quanyue Ma, TengFei Pan, and Guang Liu. 2024. CCI3.0-HQ: a large-scale Chinese dataset of high quality designed for pre-training large language models. arXiv:2410.18505 [cs.CL] https://arxiv.org/abs/2410.18505

[97] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. In *Advances in Neural Information Processing Systems*, Vol. 37. Curran Associates, Inc., 95266–95290. doi:10.52202/079017-3018

[98] Tianwen Wei, Liang Zhao, Lichang Zhang, Bo Zhu, Lijie Wang, Haihua Yang, Biye Li, Cheng Cheng, Weiwei Lü, Rui Hu, Chenxia Li, Liu Yang, Xilin Luo, Xuejie Wu, Lunan Liu, Wenjun Cheng, Peng Cheng, Jianhao Zhang, Xiaoyu Zhang, Lei Lin, Xiaokun Wang, Yutuan Ma, Chuanhai Dong, Yanqi Sun, Yifu Chen, Yongyi Peng, Xiaojuan Liang, Shuicheng Yan, Han Fang, and Yahui Zhou. 2023. Skywork: A More Open Bilingual Foundation Model. arXiv:2310.19341 [cs.CL] https://arxiv.org/abs/2310.19341

[99] Kaiyue Wen, Xingyu Dang, Kaifeng Lyu, Tengyu Ma, and Percy Liang. 2026. Fantastic Pretraining Optimizers and Where to Find Them II: Hyperball Optimization. *arXiv preprint arXiv:2606.16899* (2026). https://arxiv.org/abs/2606.16899

[100] Yihao Xiao, Jialong Sun, Zitian Gao, Zeming Wei, Chutian Wang, Ran Tao, Jiaye Teng, and Bryan Dai. 2026. Hyperball May Not Be a Free Lunch. arXiv:2607.22444 [cs.LG] https://arxiv.org/abs/2607.22444

[101] Zijie Yan, Hongxiao Bai, Xin Yao, Dennis Liu, Tong Liu, Hongbin Liu, Pingtian Li, Evan Wu, Shiqing Fan, Li Tao, et al. 2026. Scalable Training of Mixture-of-Experts Models with Megatron Core. *CoRR* abs/2603.07685 (2026). arXiv:2603.07685 https://arxiv.org/abs/2603.07685 Technical report, version 2.

[102] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jian Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. 2025. Qwen3 Technical Report. *CoRR* abs/2505.09388 (2025). arXiv:2505.09388 doi:10.48550/ARXIV.2505.09388

[103] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. 2024. Qwen2 Technical Report. *CoRR* abs/2407.10671 (2024). arXiv:2407.10671 doi:10.48550/ARXIV.2407.10671

[104] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 Technical Report. *CoRR* abs/2412.15115 (2024). arXiv:2412.15115 doi:10.48550/ARXIV.2412.15115

[105] Chen Yang, Ran Le, Yun Xing, Zhenwei An, Zongchao Chen, Wayne Xin Zhao, Yang Song, and Tao Zhang. 2025. ToolMind Technical Report: A Large-Scale, Reasoning-Enhanced Tool-Use Dataset. arXiv:2511.15718 [cs.AI] https://arxiv.org/abs/2511.15718

[106] Chen Yang, Guangyue Peng, Jiaying Zhu, Ran Le, Ruixiang Feng, Tao Zhang, Xiyun Xu, Yang Song, Yiming Jia, Yuntao Wen, Yunzhi Xu, Zekai Wang, Zhenwei An, Zhicong Sun, and Zongchao Chen. 2026. Nanbeige4.1-3B: A Small General Model that Reasons, Aligns, and Acts. arXiv:2602.13367 [cs.AI] https://arxiv.org/abs/2602.13367

[107] Xi Ye, Fangcong Yin, Yinghui He, Joie Zhang, Howard Yen, Tianyu Gao, Greg Durrett, and Danqi Chen. 2025. Longproc: Benchmarking long-context language models on long procedural generation. *arXiv preprint arXiv:2501.05414* (2025).

[108] Hu Yiwen, Huatong Song, Jie Chen, Jia Deng, Jiapeng Wang, Kun Zhou, Yutao Zhu, Jinhao Jiang, Zican Dong, Yang Lu, Xu Miao, Xin Zhao, and Ji-Rong Wen. 2025. YuLan-Mini: Pushing the Limits of Open Data-efficient Language Model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 5374–5400. doi:10.18653/v1/2025.acl-long.268

[109] Bowen Yu, Guanyu Feng, Huanqi Cao, Xiaohan Li, Zhenbo Sun, Haojie Wang, Xiaowei Zhu, Weimin Zheng, and Wenguang Chen. 2021. Chukonu: A Fully-Featured Big Data Processing System by Efficiently Integrating a Native Compute Engine into Spark. *Proc. VLDB Endow.* 15, 4 (2021), 872–885. doi:10.14778/3503585.3503596

[110] Yijiong Yu, Ziyun Dai, Zekun Wang, Wei Wang, Ran Chen, and Ji Pei. 2025. OpenCSG Chinese Corpus: A Series of High-quality Chinese Datasets for LLM Training. arXiv:2501.08197 [cs.CL] https://arxiv.org/abs/2501.08197

[111] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. HellaSwag: Can a Machine Really Finish Your Sentence?. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, Anna Korhonen, David Traum, and Lluís Màrquez (Eds.). Association for Computational Linguistics, Florence, Italy, 4791–4800. doi:10.18653/v1/P19-1472

[112] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. 2025. SageAttention2: Efficient Attention with Thorough Outlier Smoothing and Per-thread INT4 Quantization. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 75097–75119. https://proceedings.mlr.press/v267/zhang25ae.html

[113] Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. 2024. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385* (2024).

[114] Wanyue Zhang, Ziyong Li, Wen Yang, Chunlin Leng, Yinan Bai, Qianlong Du, Chengqing Zong, and Jiajun Zhang. 2024. ChineseWebText 2.0: Large-Scale High-quality Chinese Web Text with Multi-dimensional and fine-grained information. arXiv:2411.19668 [cs.CL] https://arxiv.org/abs/2411.19668

[115] Yifan Zhang, Yifan Luo, Yang Yuan, and Andrew C Yao. 2025. Autonomous Data Selection with Zero-shot Generative Classifiers for Mathematical Texts. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 4168–4189. doi:10.18653/v1/2025.findings-acl.216

[116] Zhengyan Zhang, Yuxian Gu, Xu Han, Shengqi Chen, Chaojun Xiao, Zhenbo Sun, Yuan Yao, Fanchao Qi, Jian Guan, Pei Ke, Yanzheng Cai, Guoyang Zeng, Zhixing Tan, Zhiyuan Liu, Minlie Huang, Wentao Han, Yang Liu, Xiaoyan Zhu, and Maosong Sun. 2021. CPM-2: Large-scale cost-effective pre-trained language models. *AI Open* 2 (2021), 216–224. doi:10.1016/j.aiopen.2021.12.003

[117] Zhengyan Zhang, Xu Han, Hao Zhou, Pei Ke, Yuxian Gu, Deming Ye, Yujia Qin, Yusheng Su, Haozhe Ji, Jian Guan, Fanchao Qi, Xiaozhi Wang, Yanan Zheng, Guoyang Zeng, Huanqi Cao, Shengqi Chen, Daixuan Li, Zhenbo Sun, Zhiyuan Liu, Minlie Huang, Wentao Han, Jie Tang, Juanzi Li, Xiaoyan Zhu, and Maosong Sun. 2021. CPM: A large-scale generative Chinese Pre-trained language model. *AI Open* 2 (2021), 93–99. doi:10.1016/j.aiopen.2021.07.001

[118] Chuyue Zhou, Hongya Lyu, Xinle Lin, Hengyu Zhao, Junshao Guo, Xueren Zhang, Shuaikang Xue, Qiang Ma, Jie Zhou, Yudong Wang, and Zhiyuan Liu. 2026. UltraData-Math. https://huggingface.co/datasets/openbmb/UltraData-Math

[119] Fan Zhou, Zengzhi Wang, Nikhil Ranjan, Zhoujun Cheng, Liping Tang, Guowei He, Zhengzhong Liu, and Eric P. Xing. 2025. MegaMath: Pushing the Limits of Open Math Corpora. *CoRR* abs/2504.02807 (2025). arXiv:2504.02807 doi:10.48550/ARXIV.2504.02807

[120] Zhanpeng Zhou, Wenjie Zhou, Yufei Gu, Juqiu Wang, Guanwei Zhang, Ruobing Xie, Xianbiao Qi, Yushi Bai, Kaifeng Lyu, and Shunyu Yao. 2026. From LR to ELR: A Better Heuristic for Pretraining Dynamics. https://hy.tencent.ai/research/elr

[121] Jingwei Zuo, Maksim Velikanov, Ilyas Chahed, Younes Belkada, Dhia Eddine Rhayem, Guillaume Kunsch, Hakim Hacid, Hamza Yous, Brahim Farhat, Ibrahim Khadraoui, Mugariya Farooq, Giulia Campesan, Ruxandra Cojocaru, Yasser Djilali, Shi Hu, Iheb Chaabane, Puneesh Khanna, Mohamed El Amine Seddik, Ngoc Dung Huynh, Phuc Le Khac, Leen AlQadi, Billel Mokeddem, Mohamed Chami, Abdalgader Abubaker, Mikhail Lubinets, Kacper Piskorski, and Slim Frikha. 2025. Falcon-H1: A Family of Hybrid-Head Language Models Redefining Efficiency and Performance. *CoRR* abs/2507.22448 (2025). arXiv:2507.22448 doi:10.48550/ARXIV.2507.22448

Appendices

A Limitations

This section records limitations of the report's claims that can be improved in the future.

Contamination and processed-data provenance. Most of the training mixture is assembled from already processed or filtered open datasets. Many upstream releases report some decontamination, but we do not provide a strict corpus-wide exact- or near-duplicate audit against every proxy and final benchmark. This limitation may affect the evidential strength of the curriculum comparison. In our post-training experiments, the advantage of curriculum over uniform data recipe persists beyond the base checkpoints, which can support its advantage beyond contamination. However, a corpus-wide contamination audit is still required to determine whether benchmark overlap contributes to that difference.

Chinese capability scope. The pretraining mixture includes Chinese data to provide a basic level of Chinese-language coverage, but Chinese capability is not a primary target of this release and was not the main axis used for optimization or headline model selection. We therefore omit Chinese benchmark scores from the release-facing model-comparison tables, while retaining the Chinese proxy axis used to audit data-mixture decisions. The headline evaluation remains a mathematics, code, reasoning, and knowledge study. Future versions should make the intended language targets explicit before tuning the mixture and evaluation protocol.

Overtrained dense regime. Puro-2B is a dense 2B-parameter model trained on approximately 1.4T tokens, or about 700 tokens per parameter. This is an overtrained, data-rich regime: it is useful for studying data and recipe choices and for obtaining a compact model with low inference cost, but it is not presented as compute-optimal. The chosen point balances RTX 5090 memory and communication limits, attainable benchmark quality, and the level of community support for a dense base model.

Scale-up and architecture scope. The Puro Cost Scaling Law is a fixed-2B, recipe-specific scale-down curve. A larger model or a larger world size would require a new communication and memory design; for example, a smaller vocabulary or another embedding/LM-head partition could be needed before scale-up. The current report therefore does not claim a model-size scale-up law. Especially when scaling up model size, the HBM memory capacity of RTX 5090 can be a bottleneck to deal with.

B Reproduction-Cost Assumptions

Table 6: Reference rental rates used to convert reported or estimated accelerator usage to reproduction cost. The main figures report USD; RMB values are retained for ledger traceability and use an exchange rate of USD:CNY=6.8067 as of 1 July 2026.

Reference accelerator	USD/GPU-hour	RMB/GPU-hour
H200	4.00	27.23
H100	3.25	22.12
A100 80GB	1.79	12.18
A100 40GB	1.29	8.78
A800 80GB	1.79	12.18
RTX 5090	0.31	2.08

This section specifies how every coordinate and the comparison-model frontier in Figure 1 are constructed. The plotted points are determined by the benchmark scores, public source disclosures, and cost-accounting rules described below.

Construction of the cost–performance figure. The coordinates of different models are summarized in Table 7. Our PURO-2B includes a UD ($4.4K) endpoint and the canonical CMA ($6.9K) endpoint. The cost accounting of these two runs is summarized in Table 1. More complete results of the Puro Cost Scaling Law, including different UD training variants and CD/CMA endpoints, are summarized in Figure 13b. The implementation of CMA can be found in Figure 7. The checkpoint lineage and scale-down run details of Puro Cost Scaling Law are provided in Tables 18 and 19, and the fitted scaling law can be found in Figure 2b.

Table 7: Cost and performance coordinates used in Figure 1. GPU-hours and costs are rounded to the nearest unit for presentation; USD costs use the exchange rate in Table 6, and the figure uses the unrounded coordinates. Avg score is the unweighted mean over the 15 tasks used by the figure.

Model	Evidence	Price basis	GPU-hours	Cost (RMB)	Cost (USD)	Avg score
Qwen2-1.5B	6ND, model tokens	H100 equivalent	25,939	573,817	84,302	55.14
Qwen2.5-1.5B	6ND, family tokens	H100 equivalent	66,700	1,475,530	216,776	60.73
Qwen3-1.7B-Base	6ND, model tokens	H100 equivalent	147,261	3,257,664	478,597	65.27
Gemma-2-2B	6ND, model tokens	H100 equivalent	12,560	277,860	40,821	48.60
Gemma-3-1B-PT	6ND, model tokens	H100 equivalent	4,812	106,460	15,640	25.34
Llama-3.2-3B	Reported GPU-hours	H100	460,000	10,176,017	1,495,000	52.17
LFM2.5-1.2B-Base	6ND, family tokens	H100 equivalent	78,828	1,743,808	256,190	53.54
Falcon-H1-Deep-1.5B-Base	6ND, family tokens	H100 equivalent	11,189	247,519	36,364	63.53
Instella-3B	6ND, model tokens	H100 equivalent	30,851	682,470	100,265	55.20
OLMoE-1B-7B-0125	Reported count/time	H100	61,440	1,359,162	199,680	48.70
Yulan-Mini-2.4B	6ND, reported MFU	A100-80GB rate	27,073	329,857	48,461	58.80
SmolLM3-3B-Base	Reported count/time	H100	221,184	4,892,983	718,848	65.85
MobileLLM-R1-950M-base	Reported count/time	A100-80GB rate	36,864	449,151	65,987	54.23
Puro-2B	Measured GPU-hours	RTX 5090	22,514	46,905	6,891	57.81

Performance coordinate. For model m, the vertical coordinate is the unweighted arithmetic mean

$$P_m = \frac{1}{15} \sum_{b \in \mathcal{B}} s_{m,b}, \tag{6}$$

where $s_{m,b}$ is the percentage score on benchmark b. The benchmark set $\mathcal{B}$ contains GSM8K, MATH, sanitized-MBPP, HumanEval, MMLU, MMLU-Pro, ARC-Challenge, ARC-Easy, BoolQ, CommonsenseQA, HellaSwag, PIQA, SocialIQA, WinoGrande, and BBH. No benchmark is weighted by its number of examples. All 15 scores must be present for a plotted model.

Cost evidence hierarchy. We assign the horizontal coordinate (reproduction cost) using the following ordered rules. Reported monetary cost is used directly when it is available. Otherwise, reported GPU-hours are converted using the corresponding RMB/GPU-hour rate in Table 6. If a report gives only GPU count G and elapsed training time T, we first compute $H = GT$ GPU-hours, with $T = 24d$ for a duration of d days. In all these cases, the converted cost is

$$C_m = H_m r_a, \tag{7}$$

where r_a is the reference RMB/GPU-hour price for accelerator a. The audit ledger retains this RMB value; Figure 1 divides it by 6.8067 to report the horizontal coordinate in USD.

If neither monetary cost nor accelerator-hours are reported but a usable training-token count is available, we estimate training compute with $C = 6ND$ [44, 42]. For parameter count N, token count D, accelerator BF16 peak throughput F_a in TFLOP/s, and model FLOPs utilization (MFU) η, the estimate is

$$H_m = \frac{6 N_m D_m}{3600 \times 10^{12} \times \eta F_a}, \qquad C_m = H_m r_a. \tag{8}$$

For token-only entries, we use an H100-equivalent estimate with $F_a = 989.5$ TFLOP/s and $\eta = 0.70$. Model-specific token counts are preferred; when only a family-level training budget is disclosed, that use is marked as an estimate in the coordinate table. Models lacking both accelerator usage and a usable token count receive no cost coordinate and are omitted from the figure. As we posit an ideal MFU assumption in our accounting, we estimate the cost lower bound of reproducing our counterpart models in most cases.

Public sources for Table 7. The Qwen rows use the public training-token disclosures in the Qwen2, Qwen2.5, and Qwen3 technical reports [103, 104, 102]. Gemma-2-2B and Gemma-3-1B-PT use Google's Gemma report and model-card disclosures for the corresponding model sizes and token budgets [81, 33, 31, 34]. Llama-3.2-3B uses the official Llama 3.2 model card, which reports the 3B checkpoint's H100 GPU-hours directly [59]. LFM2.5-1.2B-Base uses Liquid AI's LFM2.5 release note for the disclosed family-level training budget [51]. The Falcon-H1-Deep-1.5B-Base row uses the Falcon-H1 technical report for the token budget and H100 training pool disclosure [121]. Instella-3B uses the Instella report and model card for the two base-pretraining stages [52, 3]. OLMoE-1B-7B-0125 uses the OLMoE paper's H100 count and elapsed-time disclosure [64]. SmolLM3-3B-Base uses the Hugging Face training report for the reported H100 count and duration [7].

Special cases. Our Puro-2B point uses measured active-training time rather than $6ND$: 6,009.46 GPU-hours for Phase 1 plus 16,504.95 GPU-hours for Phase 2. Its local RTX 5090 rate is $12{,}000/(8 \times 30 \times 24) = 2.0833$ RMB/GPU-hour, giving $22{,}514.41 \times 2.0833 = 46{,}905.03$ RMB, or $6,891. Yulan-Mini-2.4B uses Equation (8) with its reported 51.57% MFU and the 312-TFLOP/s A800 BF16 peak [108]. Its GPU-hours are priced with the unified A100 80GB rental rate in place of the rarely published public A800 rental price. MobileLLM-R1 reports 128 GPUs but not their model [62]. We use the A100 80GB GPUs disclosed by the original MobileLLM training-cost report as the MobileLLM-series reference [61]. Two 4–5 day pretraining phases and two 1–2 day mid-training phases imply 30,720–43,008 accelerator-hours. We price this range with the A100 80GB rate and use the midpoint as its plotted coordinate.

Pareto frontier. To construct the dashed comparison frontier, we first remove Puro-2B. A comparison model m is Pareto-optimal if there is no other comparison model j such that

$$C_j \le C_m, \qquad P_j \ge P_m, \tag{9}$$

with at least one strict inequality. Operationally, at an identical cost we retain only the highest-scoring point, sort the remaining points by increasing cost, and retain a point only when its score is strictly larger than every lower-cost score seen so far. The retained comparison points are connected in cost order.

C Production Training Details

Production run setup. Our canonical PURO-2B run consists of two phases. Both phases use sequence length 4,096, global batch size 1,536, and micro-batch size 2. Selected matrix weights use MuonH with zero weight decay, while the remaining parameters use AdamW with weight decay 0.1; both phases use blockwise E4M3 FP8. The LR schedule below corresponds to the base learning rate, and the Hyperball weight LR is 10 times the base learning rate. Phase 1 trains on 438.8B with 24 GPUs and reaches validation loss 2.730; its median measured throughput is 238 TFLOP/s per GPU. From Phase 1 to Phase 2, we extend our compute resources, from 24 to 96 GPUs. Phase 2 resumes from this checkpoint, consumes 960.0B and continues to 1.4T cumulative tokens, and ends at validation loss 2.488 with median throughput 192 TFLOP/s per GPU. The learning rate decays from 5×10^{-3} to 1.04×10^{-3} in Phase 1 and from 1.04×10^{-3} to 1×10^{-5} in Phase 2, matching the power-then-linear schedule shown in Figure 12. The Phase 2 token ledger is summarized in Table 18; the exact transition and pool accounting is frozen in Table 20. The materialized Phase 2 pool already includes the early transition range, while Phase 1 replay is an additional consumed stream.

Table 8: The resulting audit table combines the GPUs cluster, configuration, base learning rate schedule, and throughput.

Phase	Phase tokens	Cumulative	GPUs	TP/PP/DP	LR schedule	TFLOP/s/GPU
Phase 1	438.84B	438.84B	24	1/2/12	$5.00 \times 10^{-3} \to 1.04 \times 10^{-3}$ (power)	238
Phase 2	959.99B	1.40T	96	1/4/24	$1.04 \times 10^{-3} \to 1.00 \times 10^{-5}$ (linear)	192

Table 9: Optimizer parameter groups used in both production pretraining phases. Learning rate entries are relative to the base schedule.

Parameter group	Optimizer	Weight decay	Learning rate
Selected attention and MLP matrices	MuonH	0	10× base schedule
Embeddings, normalization layers, LM head, and other parameters	AdamW	0.1	Base schedule

Phase 1 learning rate schedule. The scalar learning rate is the base rate. The MuonH-controlled matrix groups apply an optimizer multiplier of 10, while norm-variant parameters use their separately configured scalar optimizer. With optimizer step k, the Phase 1 base schedule is

$$\eta_{\text{base}}(k) = \begin{cases} 5 \times 10^{-3}\, k/1000, & 0 \le k \le 1000, \\ 5 \times 10^{-4} + 4.5 \times 10^{-3} \left(1 + \dfrac{k - 1000}{1000}\right)^{-1/2}, & k > 1000. \end{cases} \tag{10}$$

Thus $p = 1/2$, $\tau = 1{,}000$ steps, warmup is 1,000 steps (1,536,000 samples), and the 5×10^{-4} minimum is an asymptotic floor. The power decay shape follows the power schedule [86] and supports continual training in the first phase.

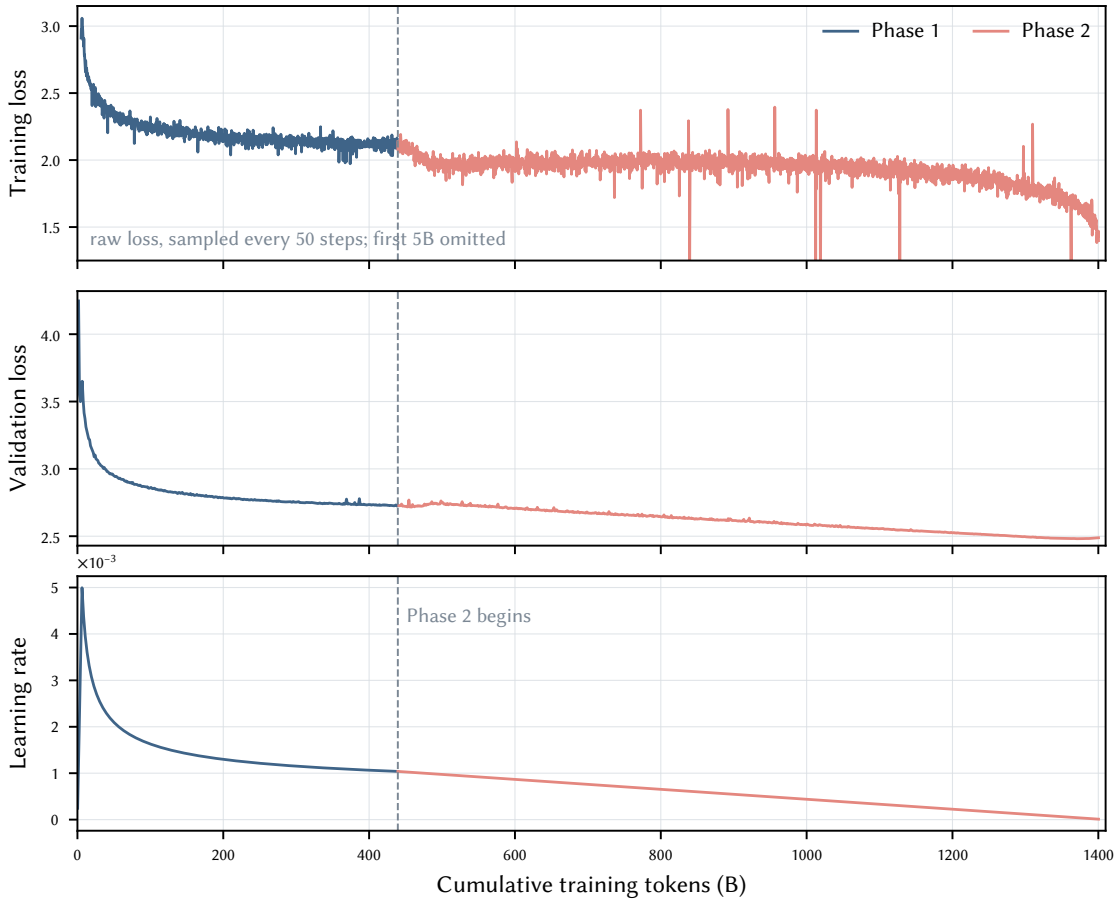


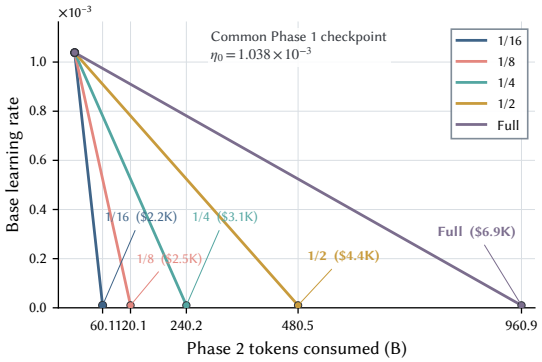
Figure 12: Concatenated production traces. Training loss is raw (not smoothed) and sampled every 50 optimizer steps; the first 5B tokens are omitted from that panel to keep the later trajectory legible. Every logged validation event is retained. The vertical line marks the Phase 1/Phase 2 run boundary; the Phase 2 run begins with the distribution transition. The learning rate schedule corresponds to the base LR and the Hyperball weight LR is 10 times base LR.

Phase 2 learning rate schedules. Each Phase 2 run loads the same Phase 1 endpoint and therefore starts from the same base learning rate, namely the terminal Phase 1 value $\eta_0 = 1.04 \times 10^{-3}$. We reset the Phase 2-local schedule coordinate at that shared checkpoint, so $s = 0$ always means the Phase 1 endpoint, not the run's recorded global optimizer step. Thus, for every run j, $\eta_{\text{base},j}(0) = \eta_0$; the runs differ only in their linear decay-sample budgets. With decay-sample budget S_j and terminal rate $\eta_{\min}$, the base schedule is

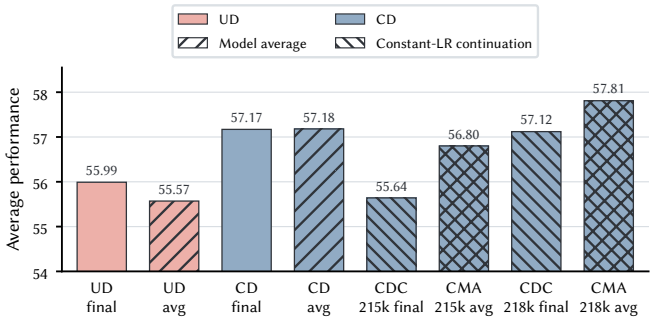
$$\eta_{\text{base},j}(s) = \eta_0 + (\eta_{\min} - \eta_0)\,\frac{s}{S_j}, \qquad 0 \le s \le S_j. \tag{11}$$

Here s is Phase 2 samples consumed after the common checkpoint. As in Phase 1, MuonH-controlled matrix groups multiply each base rate by 10. The five schedules share the same starting rate and $\eta_{\min} = 10^{-5}$, but have different horizons of approximately 60.1, 120.1, 240.2, 480.5, and 960.9B tokens, as shown in Figure 13a. These are the UD scaling points; the matched CD and CMA endpoints use the full Phase 2 budget. Our canonical CMA run uses the curriculum order and replaces the LR decay in the final steps (starting from step 218,000, lasting the last 29B tokens) of pretraining with constant-LR training. The details can be found in Section H.

FP8 implementation pipeline. The complete pipeline of our FP8 implementation is presented in Figure 14. The linear-layer GEMMs, including Fprop, Dgrad, Wgrad uses blockwise E4M3 for operands and activations. The hidden and residual flow, as well as LayerNorm and Embedding layers, use BF16. The gradient, optimizer-state accumulation and softmax remain FP32.



(a) The five independent Phase 2 base learning rate schedules used in the UD points of Figure 2b. All runs load the same Phase 1 checkpoint and therefore share the same starting base rate. They differ only in decay horizon and all decay linearly to 10^{-5} for base LR.



(b) Endpoint evaluation scores for eight Phase 2 configurations. *final* means using the final checkpoint, and *avg* means computing the checkpoint average as discussed in Section H.3. CDC/CMA *215k* and *218k* mean continual training starting from step 215k and 218k. The vertical axis is the same 15-benchmark performance defined in Tables 4 and 5. The underlying endpoint scores are reported in Table 19.

Figure 13: **Phase 2 budget schedules and endpoint comparison.** The schedule panel gives the common Phase 1 starting point and the independent UD Phase 2 horizons. The endpoint panel compares the UD, CD, CDC, and CMA configurations.

D MuonH Scaling-Ladder Analysis

Experimental setup. We construct a five-model ladder spanning model scales from 0.17B to 1.7B. Every model is trained from scratch for 20 tokens per scaling parameter (TPP=20), following the compute-optimal token-to-parameter ratio reported in prior work [42]. All runs use the same data distribution, tokenizer, sequence length of 4,096, global batch size of 512, validation set, and linear learning rate decay with a 1% warmup. The validation set uses a subset of the Nemotron-CC corpus [91, 8]. The exact architecture and token budgets are shown in Table 11. We compare two paired recipes: (i) blockwise-FP8 MuonH against a tuned blockwise-FP8 Muon baseline to evaluate the complete optimizer recipes, and (ii) blockwise-FP8 MuonH against BF16 MuonH to isolate numerical precision. MuonH uses a base LR of 10^{-2} and a Hyperball multiplier of 2, hence a Hyperball weight LR of 2×10^{-2}. The updated ordinary-Muon baseline fixes weight decay at 0.8 and uses ordinary LRs 5×10^{-3}, 4×10^{-3}, 4×10^{-3}, 3×10^{-3}, and 3×10^{-3} from 0.17B through 1.7B. The 0.17B and 0.33B choices are anchored by the reported LR/weight-decay searches; the larger points are conservative size-scaled extrapolations. The search results that motivate the first two LR/weight-decay anchors are tabulated in Table 10. The two precision runs use identical MuonH hyperparameters. Because the optimizer pair uses tuned, non-identical learning rate schedules, this is a complete-recipe comparison rather than an isolated hyperball-constraint ablation. These two experiment results are summarized in Figure 10.

Compute-equivalent fitting. For each run, we use the standard dense-transformer approximation

$$C = 6ND, \tag{12}$$

where D is the number of training tokens and $N = D/20$ is the scaling parameter count implied by the ladder construction. To compare a baseline b and variant v, we jointly fit the ten observations with

$$L_g(C) = L_\infty + A_g \left(\frac{C}{C_0}\right)^{-\alpha}, \qquad g \in \{b, v\}, \tag{13}$$

sharing L_∞ and α while allowing a group-specific amplitude. The resulting horizontal multiplier is

$$\kappa_{v \leftarrow b} = \left(\frac{A_b}{A_v}\right)^{1/\alpha}. \tag{14}$$

Thus, the variant requires $C/\kappa_{v \leftarrow b}$ to match the fitted baseline loss at compute C. This restricted shared-shape fit estimates a horizontal efficiency shift; it is not used to select a learning rate schedule or to predict the absolute loss of the 2B production run.

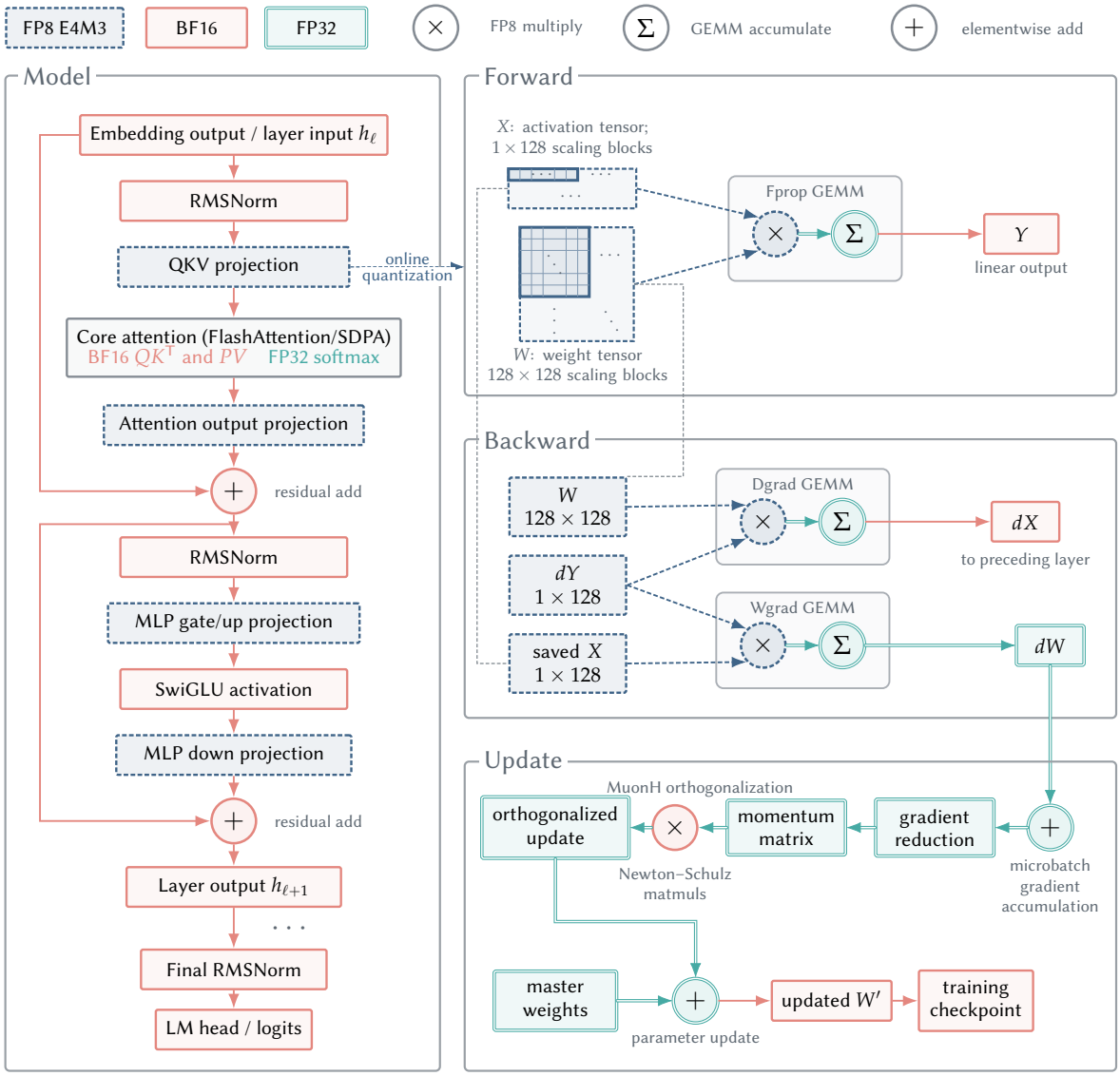


Figure 14: Precision flow for the FP8 recipe. Linear-layer GEMMs use blockwise E4M3 for operands and saved activations. The surrounding outputs, communication tensors, and checkpoint weights remain BF16, while gradient and optimizer-state accumulation remains FP32.

As shown in Figure 10a, MuonH has a fitted compute-equivalent multiplier of $1.19\times$ relative to the updated Muon baseline. Leaving out one model size at a time gives 1.17–$1.28\times$, showing that the estimate is not determined by one endpoint. Applying this horizontal factor to the 1.68×10^{22}-FLOP budget of a 2B/1.4T-token run gives 1.41×10^{22} FLOPs at matched Muon loss, corresponding to 16.1% less compute. This transfer assumes that the optimizer's horizontal efficiency factor persists beyond the 20-TPP ladder; we therefore report it as an estimated compute-equivalent saving rather than an absolute large-run loss prediction. The leave-one-out range measures sensitivity to individual ladder points, not uncertainty from the selected functional form.

Precision and throughput decomposition. To compare training cost without including evaluation, checkpoint, or experiment-tracking intervals, we convert theoretical compute to GPU-hours using

$$H = \frac{C}{3600 \times 10^{12} \times \bar{p}}, \tag{15}$$

where $\bar{p}$ is the median per-GPU throughput in TFLOP/s over training-step history. Because H is measured in aggregate GPU-hours, the world-size factor cancels. Figure 10b reports the matched validation-loss gap across all five sizes but shows throughput only for the 1.7B run. Smaller models have smaller GEMMs and need not expose

Table 10: Completed LR–weight-decay search results used to select the updated blockwise-FP8 Muon baseline anchors. The search stages are labeled initial or extended. Bold rows mark the best completed setting at each anchor size.

Model	Search stage	Ordinary LR	Weight decay	Final validation loss
0.17B	initial	2×10^{-3}	0.1	3.413030
0.17B	initial	2×10^{-3}	0.2	3.406015
0.17B	initial	2×10^{-3}	0.4	3.391572
0.17B	initial	2×10^{-3}	0.8	3.375170
0.17B	initial	3×10^{-3}	0.1	3.414363
0.17B	initial	3×10^{-3}	0.2	3.405061
0.17B	initial	3×10^{-3}	0.4	3.383846
0.17B	initial	3×10^{-3}	0.8	3.361907
0.17B	initial	4×10^{-3}	0.1	3.413549
0.17B	initial	4×10^{-3}	0.2	3.398726
0.17B	initial	4×10^{-3}	0.4	3.381147
0.17B	initial	4×10^{-3}	0.8	3.357248
0.17B	extended	4×10^{-3}	1.6	3.352910
0.17B	**extended**	$\mathbf{5 \times 10^{-3}}$	**0.8**	**3.349810**
0.17B	extended	5×10^{-3}	1.6	3.360720
0.17B	extended	6×10^{-3}	0.8	3.356287
0.17B	extended	6×10^{-3}	1.6	3.370632
0.33B	initial	2×10^{-3}	0.2	3.156636
0.33B	initial	2×10^{-3}	0.4	3.142686
0.33B	initial	2×10^{-3}	0.8	3.122845
0.33B	initial	3×10^{-3}	0.2	3.150765
0.33B	initial	3×10^{-3}	0.8	3.116963
0.33B	extended	3×10^{-3}	1.6	3.117189
0.33B	**initial**	$\mathbf{4 \times 10^{-3}}$	**0.8**	**3.114583**
0.33B	extended	4×10^{-3}	1.6	3.130430
0.33B	initial	5×10^{-3}	0.8	3.120008
0.33B	extended	5×10^{-3}	1.6	3.139534

Scale	L	d_{model}	d_{ff}	H_Q	H_{KV}	d_{ch}	d_{QK}	d_V	MBS	Steps	Tokens	$C = 6ND$
0.17B	28	512	1536	8	8	64	128	128	4	1700	3.57B	3.81×10^{18}
0.33B	28	768	2304	16	8	64	128	128	4	3100	6.50B	1.27×10^{19}
0.60B	28	1024	3072	16	8	128	128	128	2	5400	11.32B	3.85×10^{19}
1.09B	28	1536	4608	16	8	128	128	128	1	10200	21.39B	1.37×10^{20}
1.70B	28	2048	6144	16	8	128	128	128	1	16400	34.39B	3.55×10^{20}

Table 11: Architecture and training setup of the MuonH scaling ladder. All models use a sequence length of 4,096, global batch size 512, 28 layers, and 20 training tokens per scaling parameter. H_Q and H_{KV} denote query and grouped-query KV heads; d_{ch} is the configured attention channel width (`kv_channels`), and d_{QK} and d_V are the configured QK and value head dimensions.

the throughput behavior relevant to the 2B production configuration, so their throughput ratios are omitted from the main figure rather than treated as part of the production proxy.

At fixed theoretical compute, blockwise FP8 is consistently 0.0031–0.0039 higher in validation loss than BF16 across the ladder. A BF16-versus-FP8 fit in theoretical-compute space estimates 98.0% effective-compute retention; leave-one-size-out fits range from 97.6–98.1%. This corresponds to a 2.0% nominal-compute penalty at matched quality. At 1.7B, median throughput increases by 1.36×. Using this largest-scale measured throughput ratio as a proxy for the 2B setting and accounting for the fitted precision penalty, we obtain a ladder-derived, quality-adjusted speedup estimate of 1.34×, corresponding to an estimated 25.2% reduction in GPU-hours at matched quality. The FP8 speedup is a ladder-derived estimate. It combines the five-size quality fit with the measured 1.7B throughput ratio, then uses that ratio as a proxy for the 2B production setting.

MuonH diagnostic traces. To make the optimizer comparison more concrete, this appendix plots Figure 15, the learning rate and update diagnostics for the FC2[8] matrix in the 170M-parameter BF16 experiment used for Figure 5. The aligned and base Muon runs record per-group effective learning rates, weight norms, and raw update norms. The MuonH run records the shared base schedule rather than these per-group fields, so its effective LR target is reconstructed as $10\eta_t^{\text{base}}$ from the configured Hyperball multiplier. Its weight norm is therefore not drawn as an inferred measurement; projection keeps it at the prescribed initial radius.

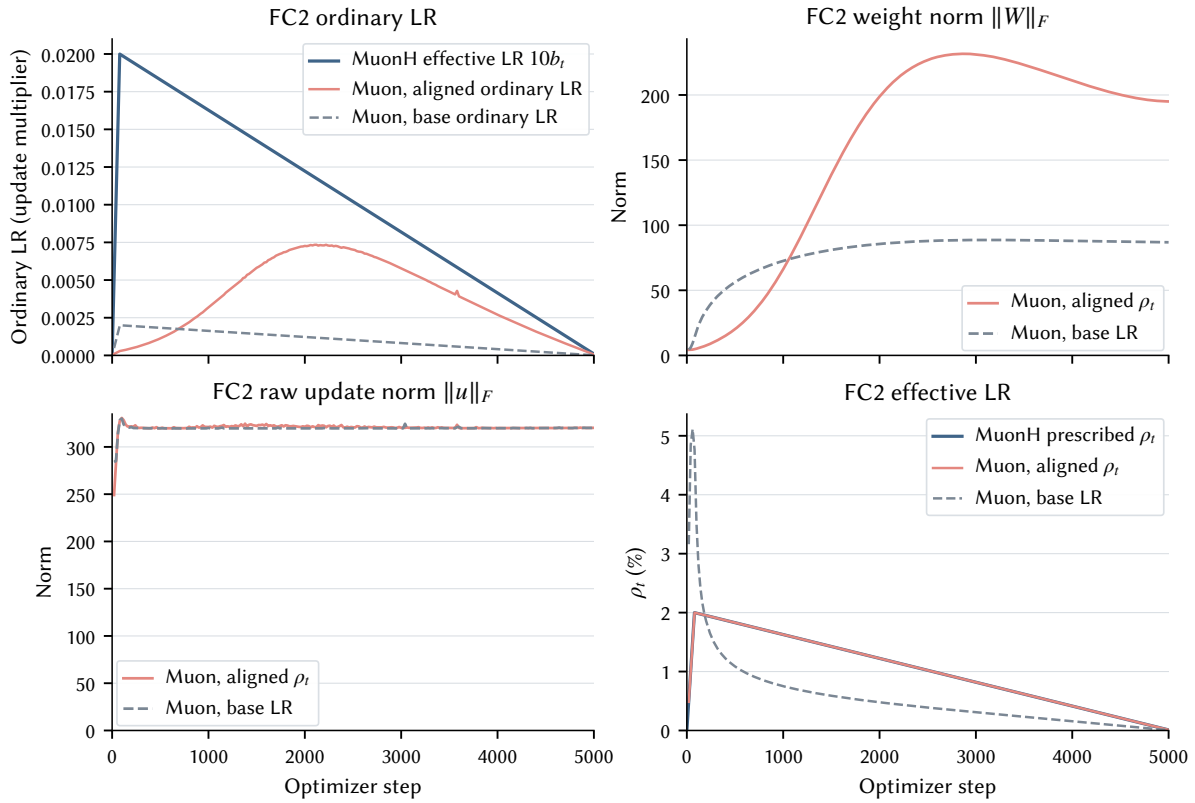


Figure 15: FC2 represents the second layer of FFN. Here we present the FC2 diagnostics for the BF16 MuonH comparison in Figure 5. The upper-left panel shows the ordinary LR, namely the scalar coefficient that multiplies the ordinary-Muon optimizer update; the lower-right panel shows the induced effective LR. The MuonH curves in both panels are reconstructed from the shared schedule, while the norm panels show the aligned/base Muon traces.

E Post-Training Details

This appendix provides the data accounting, matched training controls, evaluation details, and per-run results for the post-training experiments in Sections 3.5 and 4.4. It follows the same progression as the main text. The GSM8K-based SFT provides a targeted mathematics comparison, the Math&Code SFT with replay tests whether the difference persists after a longer and broader SFT process, and the Tulu-3 mixed-domain SFT examines transfer beyond mathematics.

E.1 Shared Construction and Evaluation Protocol

Dataset construction and accounting. The three SFT settings use the same conversation schema, Qwen3-1.7B tokenizer, Qwen ChatML format, 4,096-token context, assistant-only loss, and document-isolated packing. A packed sequence may contain multiple conversations, but each conversation retains an independent attention context. We split every materialized dataset into 90% training data and 10% validation data, then train for two epochs. The tables below report assistant-supervised token counts before packing, so they exclude runtime-added EOD labels and padding.

[8] FC2 represents the second layer of FFN.

We use *UD-based* to denote an SFT model initialized from the uniform data ordering with learning-rate decay checkpoint. We use *CMA-based* to denote an SFT model initialized from the curriculum model average checkpoint. Within each setting and repetition, the UD-based and CMA-based models use the same materialized examples, data order, tokenizer, optimizer, learning-rate schedule, and training budget. The pretraining initialization is the only difference within each paired comparison.

Each setting is repeated with three matched SFT data orders. These repetitions measure whether the direction of the UD–CMA difference is stable across changes in SFT data order. Because the CMA checkpoint jointly reflects curriculum ordering, late constant-LR training, and checkpoint averaging, the experiments compare the complete CMA recipe with UD rather than isolating one CMA component.

Training configuration. We continue to use the MuonH optimizer, as in pretraining [99]. The base learning rate follows a cosine schedule from 1×10^{-5} to 1×10^{-7}. MuonH-managed matrix parameters use a $10\times$ multiplier for their Hyperball weight learning rate. The UD-based and CMA-based models use identical optimizer settings within every SFT setup.

The GSM8K-based experiment runs on one node with eight RTX 5090 GPUs. It uses tensor parallelism 1, pipeline parallelism 2, micro-batch size 1, global batch size 160, and sequence length 4,096. The Math&Code and Tulu-3 settings use training durations and batch configurations adapted to their data budgets. These configurations remain matched between the UD-based and CMA-based models within each setting.

Evaluation protocol. Evaluation checkpoints are fixed before inspecting the benchmark results. The UD-based and CMA-based models are evaluated at the same training steps within each setting. The reported endpoint is step 172 for GSM8K-based SFT, step 2,431 for Math&Code SFT with replay, and step 449 for Tulu-3 mixed-domain SFT.

For the two mathematics-oriented settings, we evaluate all 1,319 GSM8K test examples using zero-shot ChatML prompting and greedy generation. The answer extractor first truncates any continuation that begins a new user turn. It prioritizes explicitly marked final answers, then checks standard GSM8K and boxed-answer formats, and finally uses the last numeric token as a fallback. Exact numeric equivalence after normalization determines correctness.

This flexible extraction reduces score differences caused only by answer formatting. We also use an explicit-answer-only variant as a sensitivity check and compare the sets of questions solved by the two models. For correct-answer sets C and U from the CMA-based and UD-based models, respectively, we report

$$\mathrm{IoU}(C, U) = \frac{|C \cap U|}{|C \cup U|}.$$

We additionally report C-only and U-only counts, which measure questions solved exclusively by the CMA-based and UD-based models.

For Tulu-3 mixed-domain SFT, we evaluate the final models on the same 15-task benchmark suite used in the base-model scorecard. The suite includes MMLU-Pro and BBH, and its primary summary is an unweighted macro-average over the 15 task scores. IFEval is reported separately as an instruction-following diagnostic. All broad-evaluation numbers are averaged across the same three SFT repetitions.

E.2 GSM8K-Based SFT

Data and training budget. The GSM8K-based setting provides the most controlled test of whether the pretraining difference survives SFT. It emphasizes GSM-oriented mathematics data and does not include pretraining replay. The dataset contains 191,767 source conversations and 41.42M assistant-supervised tokens. After packing, it contains 15,292 training sequences and reaches its final evaluation point after 172 optimizer steps.

Per-run results and answer analysis. The endpoint mean is 66.89% for the UD-based models and 68.66% for the CMA-based models. The resulting CMA advantage is 1.77 percentage points. All three endpoint repetitions favor CMA, with differences ranging from 1.36 to 2.12 points. The intermediate evaluations show the same overall direction.

Flexible extraction is intended to remove differences caused only by output format. Under this protocol, every endpoint repetition contains more C-only than U-only questions. The CMA-based models therefore solve more questions missed by their paired UD-based models. The explicit-answer-only sensitivity evaluation preserves the same mean difference, which supports the interpretation that the gain reflects stronger mathematical problem solving rather than answer formatting alone.

Table 12: Composition of the GSM8K-based SFT dataset. Token shares are computed from assistant-supervised tokens before packing. Small deviations from the target shares arise from indivisible records.

Component	Included source families	Supervised tokens	Token share
GSM-oriented math	GSM8K train; Tulu grade-filtered math; Tulu OpenMath GSM; OpenMathInstruct augmented GSM8K; MetaMathQA GSM rephrased	37.28M	89.998%
Instruction	PersonaHub instruction following	2.07M	5.000%
General	WildChat; NoRobots	2.07M	5.001%
Total		41.42M	100.000%

Table 13: Paired GSM8K results for the GSM8K-based SFT setting. UD and CMA are accuracies in percent, and Δ is CMA minus UD in percentage points. C-only and U-only count questions solved exclusively by the CMA-based and UD-based models.

Run	Step	UD	CMA	Δ	IoU	C-only	U-only
1	172	67.32	69.14	+1.82	.754	138	114
2	172	67.10	69.22	+2.12	.721	160	132
3	172	66.26	67.63	+1.36	.730	147	129
Mean	172	66.89	68.66	+1.77	–	–	–

E.3 Math&Code SFT with Replay

Data and training budget. The Math&Code setting tests whether the CMA advantage persists after a substantially longer SFT process on a broader data distribution. The mixture contains mathematics data, code data, and a small replay component from the final part of the Phase 2 curriculum. It contains 2,014,933 source conversations and 530.86M assistant-supervised tokens. After packing, it contains 194,530 sequences and reaches its final evaluation point after 2,431 optimizer steps.

The replay examples occupy 4.96% of the source conversations but only 2.46% of the assistant-supervised tokens. The dataset is a legacy materialized mixture whose replay slots were replaced in place. Since its original manifest did not preserve component-level token totals, we recomputed the values below with the same tokenizer and target mask used by the SFT loader. Its packing manifest also records 697 retained overlength rows.

Table 14: Composition of the Math&Code SFT with replay dataset. Assistant-supervised token counts are recomputed from the materialized JSONL using the training tokenizer and target mask.

Component	Included source families	Supervised tokens	Token share
Mathematics	OpenMathInstruct math/GSM variants; MetaMathQA GSM/MATH variants; GSM8K train	430.97M	81.183%
Code	OpenCodeInstruct generic/algorithmic; Magicoder Evol-Instruct	86.85M	16.360%
Curriculum replay	Samples from the final Phase 2 curriculum parquet, formatted as SFT continuation data	13.04M	2.456%
Total		530.86M	100.000%

Per-run results and persistence after longer SFT. A possible explanation for the original base-model difference is that CMA receives higher-quality data near the end of pretraining. Under this explanation, its advantage could be temporary and disappear after sufficient SFT on a new distribution. The Math&Code setting provides a stronger test of this possibility because it uses more training data, more optimizer steps, and a broader mixture than the GSM8K-based setting.

After this longer SFT process, the UD-based models reach 74.10% mean GSM8K accuracy, while the CMA-based models reach 76.12%. CMA is ahead in all three repetitions, and its mean advantage is 2.02 percentage points. The observed differences range from 1.21 to 3.26 points. The explicit-answer-only sensitivity evaluation gives the same qualitative and nearly identical quantitative conclusion.

These results show that the CMA advantage is not immediately overwritten by longer SFT on a broader mixture. Its mean size is also slightly larger than in the GSM8K-based setting. Since the two settings use different data mixtures and training budgets, their absolute GSM8K scores should not be compared directly. The relevant comparison is the paired difference between UD-based and CMA-based models within each setting.

Table 15: Paired endpoint GSM8K results for Math&Code SFT with replay. UD and CMA are accuracies in percent, and Δ is CMA minus UD in percentage points.

Run	UD	CMA	Δ	IoU	C-only	U-only
1	73.24	76.50	+3.26	.792	136	93
2	75.06	76.27	+1.21	.798	120	104
3	74.00	75.59	+1.59	.786	129	108
Mean	74.10	76.12	+2.02	–	–	–

E.4 Tulu-3 Mixed-Domain SFT

Data and training budget. The first two settings mainly test mathematical capability. The Tulu-3 mixed-domain setting examines whether the UD–CMA difference remains after SFT on a broader instruction distribution. Its mixture covers general data, mathematics, code, instruction following, and safety. It contains 264,612 source conversations and 82.25M assistant-supervised tokens.

After packing, the dataset contains 39,914 sequences. Both pretraining initializations are trained for two epochs and evaluated at the predefined final step 449. This setup uses the same three matched SFT repetitions as the mathematics experiments.

Table 16: Composition of the Tulu-3 mixed-domain SFT dataset. Category shares are defined by assistant-supervised tokens before packing.

Component	Included source families	Supervised tokens	Token share
General	FLAN; WildChat; NoRobots; OASST; Aya; TableGPT; SciRIFF	37.01M	45.001%
Mathematics	Persona Math; grade-filtered math; OpenMath GSM8K; NuminaMath; intermediate algebra	20.56M	24.999%
Code	Evol CodeAlpaca HumanEval; PersonaHub code	8.22M	10.000%
Instruction	PersonaHub instruction following	8.22M	10.000%
Safety	WildGuardMix; WildJailbreak; CoCoNot	8.22M	10.000%
Total		82.25M	100.000%

Broad capability and instruction-following results. Across the 15-task benchmark suite, the UD-based models reach a macro-average of 53.96%, while the CMA-based models reach 55.13%. The difference is 1.17 percentage points. CMA is higher on 10 of the 15 benchmarks, which shows that its aggregate advantage extends beyond mathematics. The gains are nevertheless uneven across individual tasks.

IFEval is reported separately because it directly measures instruction following rather than the broader capability average. The UD-based models reach 41.71%, while the CMA-based models reach 43.07%. This gives CMA an additional advantage of 1.36 percentage points on instruction following. The direction is consistent with the higher 15-task aggregate, although neither result implies that every capability improves.

E.5 Summary of the Three Comparisons

The three settings answer complementary questions about post-training persistence. GSM8K-based SFT shows that the CMA-based models retain a mathematics advantage without pretraining replay. Math&Code SFT with replay shows that this advantage remains after a longer SFT process on a broader data distribution. Tulu-3 mixed-domain SFT further shows a higher aggregate score and stronger instruction following beyond the targeted mathematics settings.

The improvement is not uniform across all tasks. Some benchmarks favor the UD-based models, even though the CMA-based models perform better on the overall mixed-domain average. The current experiments therefore

Table 17: Tulu-3 mixed-domain SFT results averaged across three repetitions. UD and CMA are percentages, and Δ is CMA minus UD in percentage points. The 15-task macro-average excludes IFEval, which is reported separately as an instruction-following diagnostic.

Benchmark	UD	CMA	Δ
GSM8K	59.92	59.23	−0.68
MATH	22.39	26.18	+3.79
HumanEval	42.27	40.65	−1.63
MBPP	50.97	50.84	−0.13
MMLU	53.97	55.80	+1.83
MMLU-Pro	25.73	26.11	+0.38
ARC-C	63.28	67.23	+3.95
ARC-E	80.90	83.60	+2.70
BoolQ	74.18	69.57	−4.61
CommonsenseQA	66.09	68.69	+2.59
HellaSwag	43.69	50.94	+7.25
PIQA	68.79	69.01	+0.22
SIQA	61.05	63.19	+2.13
WinoGrande	51.62	52.46	+0.84
BBH	44.58	43.52	−1.06
15-task macro-average	53.96	55.13	+1.17
IFEval	41.71	43.07	+1.36

support the complete CMA recipe rather than a claim of universal improvement. This task-level variation also motivates more controlled curriculum design and component-wise ablations in future work.

F Learning Rate Schedule Diagnostics

We provide more detailed diagnostics and fitting results for learning rate schedule diagnostics in this section.

F.1 Multi-Power Law for Effective Learning Rate Schedules

Preliminary. The Multi-Power Law (MPL) was introduced to predict loss curves across learning rate schedules from a small number of training runs [57]. We adapt the same construction to an effective learning rate signal for one selected weight group. Let q_t denote either the scalar optimizer learning rate coefficient η_t or the induced effective LR ρ_t. We define its post-warmup trapezoidal exposure as

$$S_q(t) = \sum_{k=t_{\text{warm}}+1}^{t} \frac{q_{k-1} + q_k}{2} \Delta t_k,$$

and let S_w denote the corresponding exposure accumulated during warmup under the same choice of q. The resulting model is

$$\begin{aligned} L(t) &= L_0 + A\left[S_q(t) + S_w\right]^{-\alpha} + B \sum_{t_{\text{warm}} < k \le t} \Delta q_k \, G(x_k(t)), \qquad \Delta q_k = q_k - q_{k-1}, \\ x_k(t) &= C q_k^{-\gamma}\left[S_q(t) - S_q(k-1)\right], \qquad\qquad G(x) = 1 - (1+x)^{-\beta}. \end{aligned} \tag{16}$$

The intuition is to decompose loss evolution into an exposure-controlled baseline and a schedule-shape correction. The first power-law term is Chinchilla-like: it treats the cumulative q-exposure $S_q(t) + S_w$ as an effective optimization-time coordinate and captures the training progress expected from that accumulated exposure. This term alone, however, does not distinguish schedules that accumulate similar exposure through different decay patterns.

The summation term accounts for the additional loss reduction induced by decreasing the learning rate signal. For a decaying schedule, $\Delta q_k < 0$. A decrease at step k therefore contributes a negative correction

$$B \Delta q_k G(x_k(t)) = -B\left(q_{k-1} - q_k\right) G(x_k(t))$$

to the loss at a later step t. This benefit is not assumed to appear instantaneously. Instead, $x_k(t)$ measures the exposure accumulated after the change, rescaled by the post-change signal level q_k, while $G(x_k(t))$ represents the fraction of the eventual loss reduction that has been realized by time t. As subsequent exposure accumulates, $G(x)$ approaches one, with the unrealized fraction $1 - G(x) = (1 + x)^{-\beta}$ decaying according to a power law. Finally, we approximate the total schedule-shape correction by superposing the delayed responses induced by all preceding changes in q. Thus, MPL combines one power law for cumulative training progress with a collection of power-law responses to reductions in the effective learning rate signal.

A useful analytical simplification assumes that every reduction is fully realized as soon as it occurs, corresponding to $G(x) = 1$. The response-weighted sum then telescopes. With $q_{\text{warm}} = q_{t_{\text{warm}}}$, we obtain

$$\begin{aligned} L_{G=1}(t) &= L_0 + A\left[S_q(t) + S_w\right]^{-\alpha} + B \sum_{t_{\text{warm}} < k \le t} \Delta q_k \\ &= L_0 + A\left[S_q(t) + S_w\right]^{-\alpha} + B\left(q_t - q_{\text{warm}}\right). \end{aligned} \tag{17}$$

For a decaying schedule and $B > 0$, the final term is non-positive and lowers the predicted loss relative to the accumulated-exposure baseline. This instantaneous-response approximation discards when the individual reductions occurred: beyond their contribution to $S_q(t)$, the correction retains only the net decrease from q_{warm} to q_t. We therefore use it as a simplified interpretation of the full response model rather than as a claim that learning rate reductions take effect instantaneously in actual training.

We test this transfer on the two BF16 ordinary-Muon runs already used in Figures 5 and 15. For each run, we fit the post-warmup validation curve using either $q_t = \eta_t$ or $q_t = \rho_t$, with the final 20% of validation targets held out. Because the two signals have different units, each is normalized by its own post-warmup peak before fitting; this only rescales the fitted constants. The fit follows the three-stage MPL protocol: power-only, power plus a schedule-drop term, and the full response above. As a simpler comparison, we also fit the $G(x) = 1$ model in Equation (17); unlike the formal response, this simplification retains only the current change from the warmup endpoint.

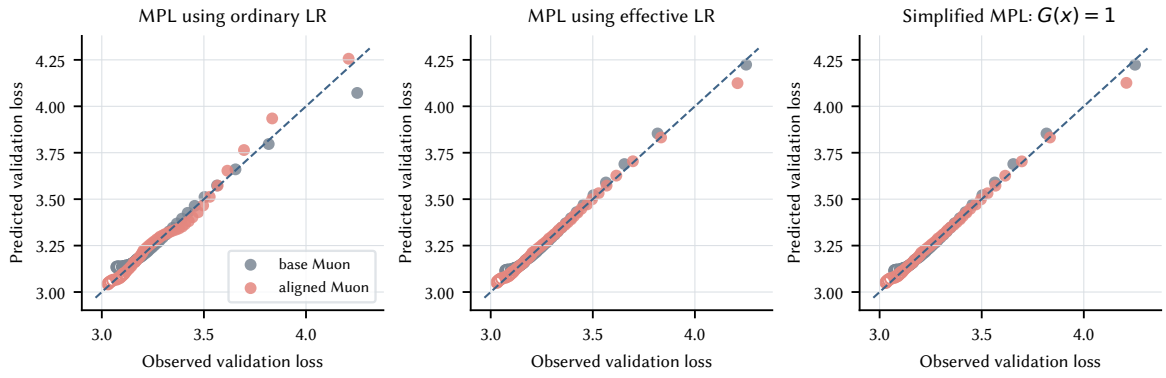


Figure 16: MPL transfer to an effective learning rate signal for the FC2 group (MPL down-projection weight). All panels compare predicted with observed validation loss; filled markers are fitting targets and hollow markers are the final 20% holdout. **Left:** The formal MPL fit using the ordinary LR. **Center:** The formal MPL fit using the induced effective LR. **Right:** The simplified $G(x) = 1$ model in Equation (17), also using the induced effective LR. The two signals are normalized separately, and colors distinguish base and effective-LR-aligned Muon.

The effective LR coordinate gives a lower mean held-out RMSE in this small FC2 diagnostic (0.0210 versus 0.0265 for ordinary LR), with the largest gain for base Muon (0.0270 versus 0.0422). For aligned Muon, the two representations are comparably predictive (0.0149 and 0.0108). The simplified $G(x) = 1$ effective LR fit has a mean held-out RMSE of 0.0207, with 0.0270 for base Muon and 0.0144 for aligned Muon, matching the vanilla MPL of effective LR on these two curves.

Then, from the perspective of Equation (17), the loss decreasing is strongly related to the effective LR decreasing. And as shown in Figure 5, the ordinary Muon run results in an LR schedule decay aggressively in the early phase, while the MuonH and aligned Muon runs use linear LR decay. The early effective LR decay in the ordinary Muon run results in the loss of potential to decay further near the end of training. Thus, the MuonH and aligned Muon runs finally surpass the ordinary run, which is not by coincidence from this diagnostic.

F.2 WSD Sweeps and Limited-Compute Schedule Estimation

The controlled sweeps isolate peak, horizon, and terminal-decay length. The source runs use a 0.6B decoder-only model, BF16 training, sequence length 4,096, GBS 512, MuonH multiplier 3, linear WSD decay, and one seed per configuration. TPP 20, 50, and 100 correspond to 11.32, 28.31, and 56.62B tokens. At TPP 20, all five positive decay ratios $\{0.2, 0.4, 0.6, 0.8, 1.0\}$ are complete for effective peaks $\{0.008, 0.012, 0.016, 0.020, 0.024\}$. Peaks 0.008 and 0.012 additionally cover TPP 50 and 100. Final loss is taken from the run summary at the nominal target step, which avoids replacing the endpoint with the last observation of a sparsely sampled history. GBS is fixed, but the two lower-peak TPP20 families use micro-batch size 2 whereas peaks 0.016–0.024 use micro-batch size 4; the peak trend is therefore a practical design diagnostic rather than a perfectly isolated micro-batch ablation.

Formal MPL turns a few anchor schedules into a cross-schedule trend estimate. For the WSD experiments below, q_t is the post-warmup effective LR and the formal response is the one in Equation (16). Only post-warmup validation targets are fitted. The three-stage procedure first fits (L_0, A, α) on the first 25% of each anchor curve, then adds the instantaneous LR-drop coefficient B on the complete curves, and finally refines all formal-MPL parameters. Six initializations are compared; each curve contributes at most 16 fitting targets, while the final 10% receives weight 4. All reported WSD fits converged in all three stages.

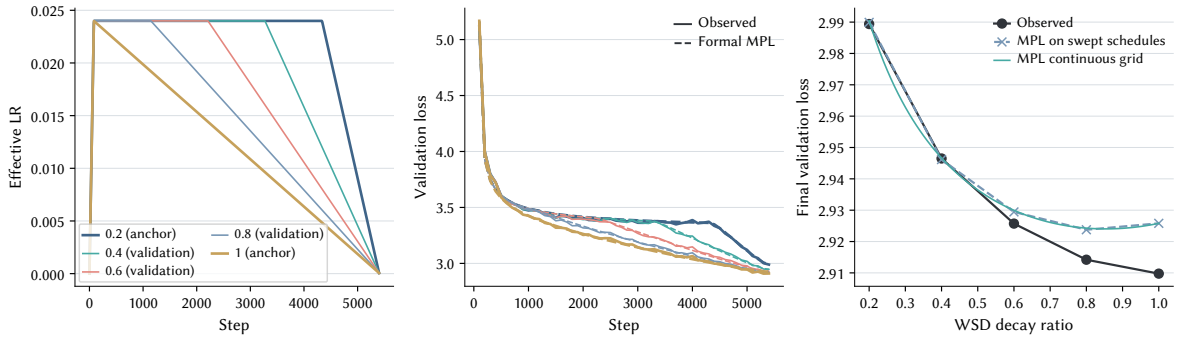


Figure 17: **A two-anchor MPL example at effective peak** 0.024 **and TPP 20.** Only decay ratios 0.2 and 1.0 are fitted; ratios 0.4, 0.6, and 0.8 are unseen validation schedules. **Left:** WSD schedules. **Center:** Held-out loss curves, with observations shown as solid lines and MPL predictions as dashed lines. **Right:** Final-loss ranking over the swept and continuous decay grids. The two anchors place the estimated minimum near ratio 0.85, consistent with the long-decay end of the direct sweep.

A two-anchor protocol recovers the peak-to-decay trend at fixed horizon. Each fixed-peak TPP20 fit sees only ratios 0.2 and 1.0, leaving 0.4, 0.6, and 0.8 unseen. Across these 15 unseen schedules, mean curve RMSE is 0.0157 and endpoint MAE is 0.0067. The predicted continuous-grid ratio rises as 0.33, 0.57, 0.67, 0.75, and 0.85 over peaks 0.008 through 0.024. This agrees with the monotone rise in the lower edge of the directly observed competitive region, although the individual observed grid minima remain noisy.

The estimator is a screening tool rather than an optimum certificate. The diagnostic helps choose which peak-rate and decay settings to test next. Broad plateaus, one seed per configuration, and noisy minima limit the precision of any inferred optimum. The continual-training role of the Phase 1 power schedule is likewise a design property of its open-ended tail; these WSD sweeps support the long-decay features, not the exact power exponent.

F.3 A Prescribed Effective LR Can Induce a Hill-Like Scalar LR in Ordinary Muon

The warmup–stable–decay (WSD) diagnostic in Figure 19 ramps the effective LR to 0.02, holds it there, and begins terminal decay at step 4,000. During the stable interval, the weight norm of the MLP down-projection matrices grows while the orthogonalized Muon-update norm remains roughly constant. Ordinary Muon must therefore increase its scalar LR to maintain the predefined effective LR, before reducing it during terminal decay. The run remains numerically stable and reaches validation loss 3.0678 after 5,000 steps. This constructed example shows why a scalar LR curve cannot be interpreted without its induced effective LR schedule; it demonstrates feasibility, not optimality or a universal requirement for ordinary Muon.

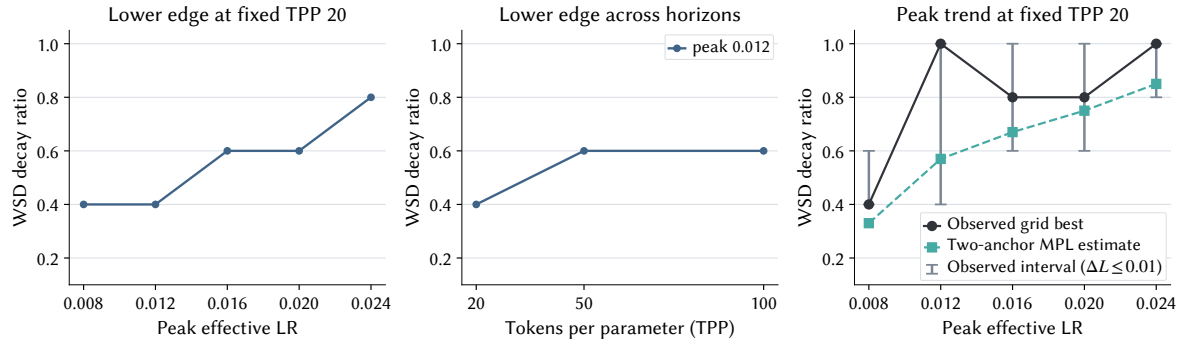
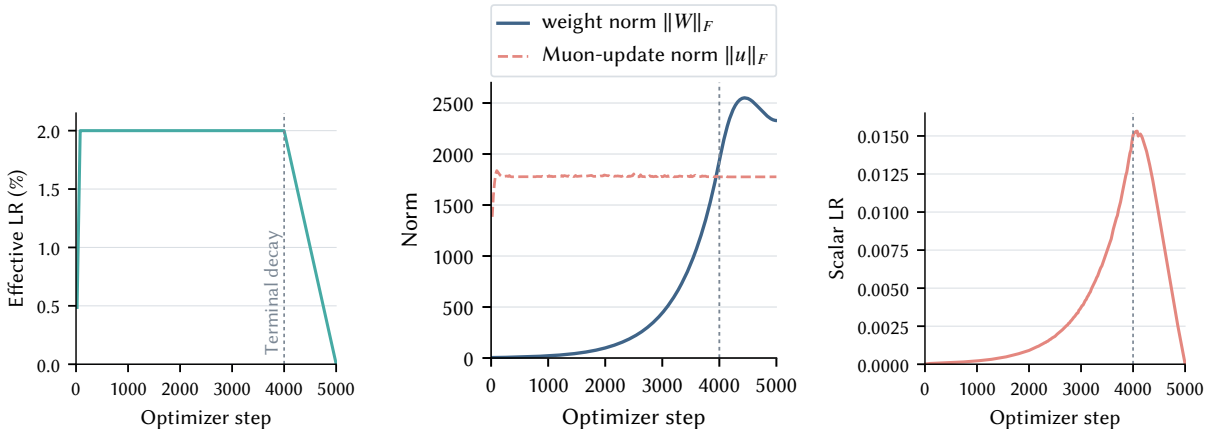


Figure 18: WSD decay trends and the limited-compute diagnostic. **Left:** At fixed TPP 20, the lower edge of the region within 0.01 validation loss of the minimum rises nondecreasingly with effective peak LR. **Center:** The same lower edge across horizons for peak 0.012. **Right:** Black points are the best ratios on the observed TPP20 grid, gray capped intervals contain every ratio within 0.01 loss of the minimum, and green squares are continuous-ratio estimates from MPL fitted only to ratios 0.2 and 1.0. The intervals describe competitive configurations, not uncertainty across seeds.



(a) Predefined effective LR (b) Weight and Muon-update norms (c) Scalar LR for ordinary Muon

Figure 19: Indirect effective LR control in an ordinary-Muon WSD diagnostic, shown on linear axes. All quantities refer to MLP down-projection matrices. **(a)** The predefined effective LR holds near 2% after warmup. **(b)** The weight norm grows while the orthogonalized Muon-update norm stays roughly constant. **(c)** The scalar LR must therefore rise to maintain the effective LR before falling during terminal decay.

G Puro-2B Scaling Checkpoint Ledger

The following ledger freezes the identifiers and accounting coordinates used by Figure 2(b). Phase 1 is shared by all rows. Phase 2 tokens are schedule consumption, so the production Phase 2 pool is not added a second time when the early transition range is listed. The endpoint comparison is visualized in Figures 7 and 13b.

Table 18: Checkpoint lineage for the five UD points in the Puro Cost Scaling Law, the CD endpoint, and the canonical CMA endpoint. GPU-hours are total active-training GPU-hours and costs use the normalized RTX 5090 rate and are reported in USD. Here, *SMA6* (simple moving average) denotes the equal-weight average of model parameters from six late checkpoints on the step-218,000 constant-LR continuation; optimizer states are not averaged (details in Section H.3).

Released checkpoint ID	Phase 2 tokens	Total tokens	Phase 2 ordering	Model averaging	Total GPU-h	Cost (USD)
`Puro-2B-UD-Phase2-1of16`	60.00B	498.83B	UD global reshuffle	No	7,041	$2.16K
`Puro-2B-UD-Phase2-1of8`	120.00B	558.83B	UD global reshuffle	No	8,073	$2.47K
`Puro-2B-UD-Phase2-1of4`	240.00B	678.83B	UD global reshuffle	No	10,136	$3.10K
`Puro-2B-UD-Phase2-1of2`	480.00B	918.83B	UD global reshuffle	No	14,262	$4.37K
`Puro-2B-UD`	959.99B	1.40T	UD global reshuffle	No	22,514	$6.89K
`Puro-2B-CD`	959.99B	1.40T	CD component-local curriculum; decay final	No	22,514	$6.89K
`Puro-2B-CMA`	959.99B	1.40T	CMA curriculum + late constant LR	Yes (SMA6)	22,514	$6.89K

Table 19: 15-benchmark score sheet for the UD, CD, CDC, and CMA branches(detailed definitions in Section 3.4). Scores are percentages, and Avg15 is the unweighted arithmetic mean over the 15 displayed benchmarks. The canonical Puro-2B row(CMA 218k avg) is the released six-checkpoint SMA endpoint.

Model	GSM8K	MATH	MBPP	HumanEval	MMLU	MMLU-Pro	ARC-C	ARC-E	BoolQ	CSQA	HSwag	PIQA	SIQA	WinoGrande	BBH	Avg15
UD 1/16	47.23	21.78	45.53	18.90	49.71	20.67	61.02	79.19	75.11	65.68	62.28	75.30	60.18	59.51	37.66	51.98
UD 1/8	48.22	24.64	46.69	25.00	51.37	22.29	63.39	79.54	77.16	64.78	61.25	75.90	60.24	59.98	38.09	53.24
UD 1/4	42.30	26.02	48.64	23.78	53.96	23.83	66.44	81.83	74.83	65.27	62.59	76.17	63.10	60.22	41.20	54.01
UD 1/2 ($4.4K)	56.03	27.86	49.81	20.73	54.82	24.02	66.78	84.66	77.34	67.65	63.62	75.84	62.90	59.91	41.14	55.54
UD decay avg	55.72	26.70	50.58	18.90	56.23	24.87	68.81	84.13	79.27	67.16	63.32	77.69	62.64	60.30	37.29	55.57
UD full	55.72	26.94	50.19	20.73	56.22	25.55	70.17	84.83	79.48	67.49	63.30	77.58	62.64	60.38	38.75	56.00
CD decay final	57.77	29.68	52.14	25.00	57.11	25.56	73.56	85.89	76.24	68.88	63.38	75.46	64.43	60.93	41.45	57.17
CD decay avg	57.32	29.30	51.75	26.22	57.12	26.50	73.56	86.07	76.18	69.29	62.96	75.63	64.33	60.69	40.80	57.18
CDC 215k final	54.74	27.60	46.30	22.56	56.28	25.44	73.90	84.83	73.52	67.49	62.46	74.92	62.85	61.33	40.39	55.64
CMA 215k avg	55.95	28.18	50.19	25.00	57.05	26.09	75.93	86.07	74.80	67.73	62.72	75.41	62.69	61.48	42.78	56.80
CDC 218k final	57.47	29.22	50.19	30.49	56.45	25.12	73.90	84.13	78.10	67.49	63.02	75.08	63.51	61.48	41.19	57.12
CMA 218k avg ($6.9K)	59.67	30.30	52.92	31.10	57.44	25.81	74.24	86.07	76.85	67.81	63.21	75.52	63.61	60.62	42.05	57.81

H Curriculum Construction and Model Averaging

This appendix specifies our concrete Phase 2 curriculum and model-averaging configuration. The curriculum first assigns each example a component-local token-percentile coordinate under the configured preference order of its own component. Scores are therefore used only to order examples within a component; we do not assume that scores are comparable across components. Equal percentile ranges from all components are then aligned into production curriculum buckets, which approximately preserves the configured component mixture while allowing each scored component to progress through its own preference order.

The Phase 2 schedule begins with a transition that combines deterministic Phase 1 replay with the earliest ranges of the Phase 2 curriculum. The released endpoint subsequently uses a constant learning rate continuation and an equal-weight parameter average over six late checkpoints. We first describe the scalable construction of the curriculum coordinates and buckets, then specify the transition and UD control, and finally document the continuation, averaging rule, and scope of the available comparisons.

Throughout this appendix, UD denotes the uniform-data ordering with learning rate decay, CD denotes the component-local curriculum ordering with the same decay, CDC denotes curriculum followed by constant-LR continuation without averaging, and CMA denotes the production continuation with six-checkpoint averaging.

H.1 Scalable Construction of Component-Local Curriculum Buckets

For each component, the curriculum construction must determine where an example lies within that component's ordered token range. An exact implementation could sort every document by

$$(\text{configured score}, \text{stable hash}),$$

compute cumulative token counts over the sorted rows, and divide the resulting token range into 376 equal-percentile intervals. Applying document-level distributed sorts and cumulative windows to the full Phase 2 pool would, however, incur substantial Spark shuffle and serialization overhead. We therefore approximate the same component-local token coordinate using a smaller collection of aggregated internal bins.

For a component with a usable score, preprocessing first applies the configured score direction so that the resulting sequence runs from the less-preferred to the more-preferred score range. Spark's approximate-quantile operation then estimates 256 score intervals with relative error 0.01. Each interval is subdivided into 64 deterministic hash shards. The score intervals determine the coarse preference order, while the hash shards provide deterministic ordering and finer token-accounting granularity among examples with equal or nearly equal scores. Thus, a scored component contains up to 256 score-ordered strata, each with 64 deterministic subdivisions, although repeated score values can produce fewer distinct strata. Null scores are placed at the configured least-preferred end.

A component without a usable score instead uses 4096 deterministic random shards. These shards define a reproducible component-local order, but we do not interpret that order as a quality or difficulty ranking.

Preprocessing next aggregates the token count of every internal bin. For internal bin j in component c, let $T_{c,j}$ be its token count and let

$$T_c = \sum_j T_{c,j}$$

be the total number of tokens in the component. The token fraction preceding bin j is

$$p_{c,j} = \frac{\sum_{i<j} T_{c,i}}{T_c}. \tag{18}$$

The implementation additionally uses a deterministic within-bin position to spread examples over the token interval occupied by that bin. For exposition, let $u_{c,j}(x) \in [0,1)$ denote the resulting fractional position of example x within bin j. Its component-local token coordinate can then be written as

$$p(x) = \frac{\sum_{i<j(x)} T_{c,i} + u_{c,j(x)}(x) T_{c,j(x)}}{T_c}. \tag{19}$$

Thus, $p(x) = 0.25$ means that x appears after approximately 25% of the token mass of its own component, irrespective of how many documents produce those tokens. Using token mass rather than document count prevents a short document and a very long document from contributing equally to the curriculum coordinate.

Each example is mapped to one of 376 production curriculum buckets:

$$b(x) = \min\{375,\ \lfloor 376\, p(x) \rfloor\}. \tag{20}$$

Equivalently, bucket k receives from every component the examples whose component-local token coordinates lie approximately in

$$\left[\frac{k}{376}, \frac{k+1}{376}\right), \qquad k = 0, \ldots, 375.$$

Consequently, each production bucket draws approximately the same token fraction from every component. This alignment approximately preserves the configured component mixture across the curriculum while allowing different components to follow their own source-local preference orders. In particular, the construction does not impose a single global quality ranking over examples from different sources.

H.2 Phase Transition and UD Control

The Phase 2 schedule begins with a transition from the Phase 1 mixture to the ordered Phase 2 pool. The materialized transition contains approximately 43.9B tokens: 21.9B tokens of deterministic Phase 1 replay and 21.9B tokens from the earliest 2.34% of the ordered Phase 2 range. The latter is a subset of the frozen Phase 2 component pool rather than an additional copy of Phase 2 data. Therefore, only the Phase 1 replay is added to the component-pool total when computing the complete production schedule.

Table 20: Token ledger for the Phase 2 transition and production schedule. The Phase 2 early range is already included in the component pool; the production total adds only the Phase 1 replay.

Stream or quantity	Tokens	Definition
Phase 1 replay	21.93B	Deterministic stable-hash sample from the Phase 1 mixture.
Phase 2 early range	21.93B	Earliest approximately 2.34% of the ordered Phase 2 pool.
Realized transition	43.87B	Phase 1 replay combined with the Phase 2 early range.
Remaining Phase 2 range	916.13B	Portion of the Phase 2 pool following the early transition range.
Phase 2 component pool	938.06B	Early range plus the remaining Phase 2 range.
Production Phase 2 schedule	959.99B	Phase 2 component pool plus Phase 1 replay.

The aligned bucket construction approximately preserves the groups recorded in the frozen preprocessing manifest throughout the non-transition portion of Phase 2. These scheduler diagnostics use the manifest taxonomy rather than the semantic-domain taxonomy used in the main data tables. The two taxonomies differ for the code-only MegaMath-Code partition: the frozen preprocessing manifest retains its original Mathematics label, whereas Tables 3 and 22 report that partition under Code.

Table 21: Observed manifest-group token-share ranges across non-transition Phase 2 curriculum buckets.

Manifest group	Observed token-share range
Chinese	9.25%–9.54%
Code	6.37%–8.73%
English	58.32%–59.79%
Mathematics	22.38%–22.98%
Instruction-formatted	1.30%–1.40%

Examples in the transition portion are deterministically shuffled with seed `20260601`, and the subsequent curriculum buckets use seed `20260602`. The UD stream is constructed from the same materialized Phase 2 component pool, but removes the aligned curriculum order through a deterministic global reshuffle with seed `20260609`. Thus, the CD and UD streams match the Phase 2 token multiset at the pool level while differing in its presentation order. The resulting streams are the UD and CD inputs used in the endpoint comparison.

H.3 Constant-LR Continuation and Checkpoint Averaging

The available late-stage configurations include averaging without launching a new continuation, a constant-LR continuation resumed from step 215,000, and a constant-LR continuation resumed from step 218,000. The production export uses the branch resumed from step 218,000. These configurations were not constructed as a matched ablation over continuation entry points. Their comparison therefore motivates a production configuration choice but does not establish that step 218,000 is an optimal resume point.

The CDC controls use the same curriculum order and these constant-LR continuations but report the unaveraged final checkpoints. CMA denotes the step-218,000 continuation with equal-weight averaging of six late checkpoints.

At step 218,000, the logged base learning rate coefficient is 4.08×10^{-5}. The MuonH matrix-parameter group applies a multiplier of 10, yielding a group-level coefficient of 4.08×10^{-4}. The continuation holds these coefficients fixed instead of following the remaining terminal decay. For a MuonH-wrapped matrix, the group-level coefficient is the prescribed effective LR defined in Section 3.3.

The production model equally averages the model parameters stored at the following six optimizer steps:

222,100, 222,200, 222,300,
222,400, 222,500, 222,569.

The averaging window spans 469 optimizer steps, corresponding to approximately 2.95B training tokens under the production batch configuration. Most neighboring checkpoints are separated by 100 optimizer steps, or approximately 0.63B tokens, while the final interval contains 69 steps.

Only model parameters are averaged; optimizer states are not. The released model therefore contains the equal-weight arithmetic mean of the six checkpoints above rather than the parameters of the unaveraged final iterate. Our implementation uses this simple model average, whereas the reference CMA method evaluates several averaging rules and uses an exponential moving average over the final six checkpoints as its default configuration [56].

I Data Recipe Details

Table 22: Dataset-family accounting for the stationary Phase 1 and Phase 2 component pools.

Dataset / family	Domain	P1 tokens	P2 tokens	Materialization	Upstream source	Declared upstream terms[a]
FineWeb-Edu-CN	Chinese	19.73B	34.79B	Both: Score-filtered	opencsg/Fineweb-Edu-Chinese-V2.1 [110]	OpenCSG Community License and Apache 2.0
FineWeb-Edu-Chinese V2.2	Chinese	16.50B	28.11B	P1: Score-filtered; P2: No additional sampling	opencsg/Fineweb-Edu-Chinese-V2.2 [110]	Apache 2.0 metadata; web-source terms also apply
ChineseWebText2.0 (quality-filtered subset)	Chinese	11.05B	17.50B	Both: Score-filtered	CASIA-LM/ChineseWebText2.0 [114]	Apache 2.0 metadata
Deduplicated merged Chinese web	Chinese	–	5.77B	P2: Score-filtered	CCI{,2,3}, SkyPile-150B, WanJuan1.0, IndustryCorpus{,2}, and WuDaoCorpus2.0 [96, 98, 38, 39, 87, 116, 117]	Mixed upstream terms; see appendix license note
Baidu-Baike	Chinese	1.19B	1.19B	Both: No additional sampling	mohamedah/baidu_baike	MIT metadata; upstream content rights are not clarified
FineWiki-CN	Chinese	1.10B	1.10B	Both: No additional sampling	HuggingFaceFW/finewiki [75]	CC BY-SA 4.0; older Wikipedia content may also be GFDL
UNDL ZH-EN Aligned	Chinese	1.75B	–	P1: No additional sampling	bot-yaya/undl_zh2en_aligned	MIT metadata; underlying UN document terms also apply
Alpaca-Zh	Chinese	0.01B	0.01B	Both: No additional sampling	hfl/alpaca_zh_51k [25]	Apache 2.0
MegaMath-Code	Code	–	42.77B	P2: No additional sampling	LLM360/MegaMath [119]	ODC-By 1.0
Nemotron Synthetic Code	Code	17.09B	25.64B	Both: Random subsample	nvidia/Nemotron-Pretraining-Code-v1 (Synthetic-Code) [8]	NVIDIA Data Agreement for Model Training; gated, no raw-data redistribution
Swallow-Code-v2	Code	14.12B	9.30B	Both: Score-filtered	tokyotech-llm/swallow-code-v2 [29]	Apache 2.0 and original code licenses
CoderForge Trajectories	Code	–	15.78B	P2: No additional sampling	togethercomputer/CoderForge-Preview [5]	No dataset-level license declared; per-row licenses are provided
StackExchange	Code	–	5.54B	P2: Random subsample	togethercomputer/RedPajama-Data-1T [20]	CC BY-SA 2.5/3.0/4.0
Python Code Large	Code	–	4.37B	P2: No additional sampling	ajibawa-2023/Python-Code-Large	MIT metadata; upstream code provenance undocumented
Python-Edu	Code	3.41B	–	P1: No additional sampling	HuggingFaceTB/smollm-corpus [54, 9]	ODC-By 1.0 and original code licenses
Codeforces-CoTs	Code	–	2.93B	P2: No additional sampling	open-r1/codeforces-cots [76]	CC BY 4.0 metadata; upstream Codeforces terms also apply

Continued on next page

Table 22: Dataset-family accounting for the stationary Phase 1 and Phase 2 component pools. (Continued)

Dataset / family	Domain	P1 tokens	P2 tokens	Materialization	Upstream source	Declared upstream terms[a]
GitHub Top Code	Code	–	1.55B	P2: No additional sampling	ronantakizawa/github-top-code	MIT metadata; original repository licenses apply
Jupyter Agent	Code	–	0.23B	P2: No additional sampling	jupyter-agent/jupyter-agent-dataset [19]	Apache 2.0; referenced source terms also apply
LongCodeU CU-DFA	Code	–	0.02B	P2: No additional sampling	longcodeu/longcodeu-dataset [107]	Apache 2.0
Nemotron HQ	English	107.51B	447.97B	Both: Score-filtered	nvidia/Nemotron-CC-v2 [91, 8]	NVIDIA Data Agreement for Model Training; gated, no raw-data redistribution
Nemotron HQ Synthetic	English	97.67B	39.64B	Both: Score-filtered	nvidia/Nemotron-CC-v2 [91, 8]	NVIDIA Data Agreement for Model Training; gated, no raw-data redistribution
FineWeb-Edu-EN	English	59.79B	–	P1: Score-filtered	HuggingFaceTB/smollm-corpus [9]	ODC-By 1.0; Common Crawl terms also apply
Cosmopedia-v2	English	27.41B	27.41B	Both: No additional sampling	HuggingFaceTB/smollm-corpus [9]	ODC-By 1.0
DCLM-Dedup	English	–	33.77B	P2: Score-filtered	mlfoundations/dclm-baseline-1.0 [49]	CC BY 4.0; Common Crawl terms also apply
ArXiv	English	28.93B	–	P1: No additional sampling	togethercomputer/RedPajama-Data-1T [20]	Metadata CC0 1.0; article content has author-selected licenses
FineWiki-EN	English	–	8.74B	P2: No additional sampling	HuggingFaceFW/finewiki [75]	CC BY-SA 4.0; older Wikipedia content may also be GFDL
UltraData-Math	Math	–	118.33B	P2: No additional sampling	openbmb/UltraData-Math [118]	Apache 2.0 on the current dataset card
SwallowMath-v2	Math	9.99B	13.32B	Both: Random subsample	tokyotech-llm/swallow-math-v2 [29]	Apache 2.0
Nemotron-CC-Math	Math	11.48B	2.47B	Both: Score-filtered	nvidia/Nemotron-CC-Math-v1 [8]	NVIDIA Data Agreement/Open Data terms; production revision to pin
MegaMath-Web-Pro	Math	–	13.45B	P2: No additional sampling	LLM360/MegaMath [119]	ODC-By 1.0
OpenWebMath	Math	–	13.23B	P2: No additional sampling	open-web-math/open-web-math [74]	ODC-By 1.0; Common Crawl terms also apply
FineMath	Math	10.10B	–	P1: No additional sampling	HuggingFaceTB/finemath [2]	ODC-By 1.0
AutoMathText	Math	–	8.71B	P2: No additional sampling	math-ai/AutoMathText [115]	CC BY-SA 4.0
STEM Reasoning Complex	Math	–	1.21B	P2: No additional sampling	galaxyMindAiLabs/stem-reasoning-complex	Apache 2.0
NuminaMath-CoT	Math	–	0.44B	P2: No additional sampling	AI-MO/NuminaMath-CoT [48]	Apache 2.0

Continued on next page

Table 22: Dataset-family accounting for the stationary Phase 1 and Phase 2 component pools. (Continued)

Dataset / family	Domain	P1 tokens	P2 tokens	Materialization	Upstream source	Declared upstream terms[a]
FineProofs SFT	Math	–	0.13B	P2: No additional sampling	lm-provers/FineProofs-SFT	Apache 2.0
Nemotron Terminal Corpus	SFT / instruction	–	6.27B	P2: No additional sampling	nvidia/Nemotron-Terminal-Corpus [77]	CC BY 4.0 on the current dataset card; production revision to pin
JiuZhang3.0 PT-CoT	SFT / instruction	–	3.58B	P2: No additional sampling	ToheartZhang/JiuZhang3.0-Corpus-PT-CoT	Not specified on the upstream dataset card
Tulu-3 SFT	SFT / instruction	–	0.64B	P2: No additional sampling	allenai/tulu-3-sft-mixture [46]	ODC-By 1.0 mixture; stricter subset terms may apply
ToolMind	SFT / instruction	–	0.53B	P2: No additional sampling	Nanbeige/ToolMind [105]	Apache 2.0
ToolBench	SFT / instruction	–	0.47B	P2: No additional sampling	tuandunghcmut/toolbench-v1 [79]	Apache 2.0 metadata
ToolMind-Web-QA	SFT / instruction	–	0.43B	P2: No additional sampling	Nanbeige/ToolMind-Web-QA [106]	Apache 2.0
Multi-Turn Long Context	SFT / instruction	–	0.22B	P2: No additional sampling	TreeAILab/Multi-turn_Long-context_Benchmark_for_LLMs [47]	CC BY 4.0
SlimOrca	SFT / instruction	–	0.20B	P2: No additional sampling	Open-Orca/SlimOrca [65, 53]	MIT
LongAlpaca	SFT / instruction	–	0.12B	P2: No additional sampling	Yukang/LongAlpaca-12k [15]	No license declared on the current dataset card
Organic CoT	SFT / instruction	–	0.11B	P2: No additional sampling	CodonProject/Organic-Reasoning-195k (ingested as A03HCY/Organic-CoT-Reasoning-SFT)	Apache 2.0
OpenThoughts Agent	SFT / instruction	–	0.11B	P2: No additional sampling	open-thoughts/OpenThoughts-Agent-v1-SFT [94]	Apache 2.0

[a] Token counts are tokenizer-dependent materialized exposure, not necessarily unique upstream content; a family can aggregate overlapping configurations. They exclude the Phase 2 transition and replay. The code-only MegaMath-Code partition is reported under Code although its raw scheduling manifest used the broader math label. Terms describe current public metadata unless otherwise noted.

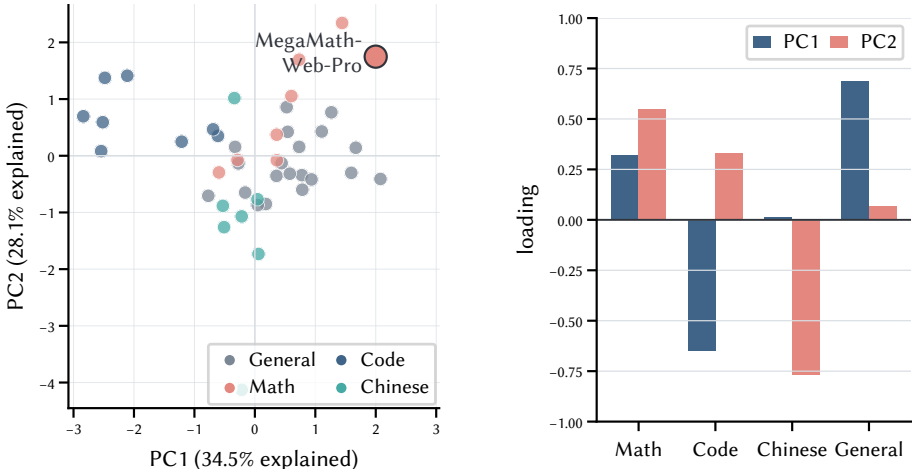
Component accounting and license scope. Table 22 reports the materialized data components used in the two pretraining phases. Its token counts come from the component-level materialization records and therefore reflect the sampled, tokenized components actually used by the training recipe. The table identifies the upstream dataset or source family and states the current public license or terms that we could verify. A missing license is reported as "to verify" or "not specified". Nemotron-CC-v2 and the Synthetic-Code partition of Nemotron-Pretraining-Code-v1 use NVIDIA's data agreement, which permits model training but prohibits redistribution of the raw dataset [67]. The merged Chinese web component uses the same CCI, SkyPile-150B, WanJuan1.0, IndustryCorpus, and WuDaoCorpus2.0 source families documented by the Kaiyuan recipe and inherits their respective terms [96, 98, 38, 39, 87, 116, 117, 70, 90]. Due to these terms, we also release our data preprocessing framework for data construction.

Proxy measurement protocol. We use a Qwen3-0.6B model for proxy experiments. The proxy experiment starts from a checkpoint trained on 86B tokens on the same broad base mixture. Then it receives approximately 8.4B continuation tokens. It corresponds to 2,000 steps with a sequence length of 4,096 and a global batch size of 1,024. The continuation differs only in its candidate source or within-source slice. We evaluate the final checkpoint on a suite of benchmarks and aggregate evaluation scores into Math, Code, Chinese, and General axes. Math combines GSM8K and MATH, Code combines MBPP and HumanEval, Chinese combines C-Eval and CMMLU, and General summarizes the remaining knowledge and commonsense tasks.

We adopt a continuous data schedule in the proxy experiments. Let $r(t)$ denote the candidate fraction at continuation step t. The candidate share follows

$$r(t) = \min\left(0.8,\ 0.8 \times \frac{t}{1600}\right), \qquad 0 \le t \le 2000,$$

so the candidate ramps from 0% to 80% during the first 1,600 steps and remains at 80% for the final 400 steps. This schedule controls the distribution shift to avoid abrupt changes in distribution for measurement. The resulting feature vector depends on the experiment setup, including proxy scale, continuation schedule, and evaluation suite.



(a) Candidate profiles (b) PCA loadings

Figure 20: Proxy evidence used in recipe design: **(a)** Candidate profiles on the first two principal components and **(b)** the capability contrasts exposed by the PCA loadings. PCA is descriptive; the comparisons do not establish causal gains.

Proxy Capability Dimensions The raw proxy export contains 39 source or score-quantile slices evaluated on 15 benchmarks. We use four capability dimensions to make the benchmark panel interpretable: Math averages GSM8K and MATH; Code averages MBPP and HumanEval; Chinese averages C-Eval and CMMLU; and General averages the remaining nine English knowledge and commonsense tasks. The raw percentage averages and their within-panel ranks over the 39 data slices are reported in Table 24. For the PCA visualization, each benchmark b is first converted to a population z-score across the 39 candidates,

$$z_{i,b} = \frac{x_{i,b} - \mu_b}{\sigma_b}.$$

The PCA visualization averages the corresponding z-score columns within each dimension and then standardizes the four aggregate axes once more before fitting PCA. The first two components explain 34.5% and 28.1% of the standardized four-axis variance. Panel (b) shows the resulting loading contrasts: PC1 primarily contrasts General with Code, while PC2 primarily contrasts Math with Chinese. These directions summarize co-variation in this panel.

Table 23: Raw proxy benchmark scores (percentage points) before per-task standardization. Dataset labels follow the canonical family names in Table 22. Source qualifiers remain in the Slice column. The redundant DCLM-Dedup-50B probes are omitted because they duplicate DCLM-Dedup in the production recipe.

Source	Slice	GSM8K	MATH	MBPP	HE	CEval	CMMLU	MMLU	ARC-C	ARC-E	BoolQ	CSQA	HSwag	PIQA	SIQA	WinoG
Algebraic Stack Train	random	0.38	0.96	3.11	5.49	29.78	30.37	30.82	28.14	61.20	62.32	47.91	38.71	65.61	43.14	52.41
ArXiv	random	0.68	1.04	1.95	5.49	29.06	30.53	30.85	26.44	58.91	62.26	47.50	38.39	66.92	43.86	52.49
AutoMathText	random	0.61	1.44	6.61	4.88	29.58	30.47	31.03	29.15	65.08	57.40	49.63	38.70	67.30	44.98	52.57
Cosmopedia-v2	random	0.38	1.54	5.06	5.49	30.97	30.35	31.24	27.12	63.14	53.09	47.83	40.09	67.68	44.11	52.80
DCLM-Dedup	q00	0.91	0.96	1.95	4.88	29.39	31.04	32.03	29.83	63.67	62.32	51.02	40.69	68.72	44.42	53.20
DCLM-Dedup	q25	0.91	1.34	3.50	7.32	28.73	30.71	30.95	26.78	61.02	60.49	49.30	40.63	68.66	44.68	53.59
DCLM-Dedup	q50	0.38	0.88	1.56	4.88	29.75	30.42	30.59	28.47	61.20	60.34	49.47	40.70	67.85	44.32	54.30
DCLM-Dedup	q75	1.14	0.92	2.72	5.49	30.79	30.68	30.11	28.14	58.02	61.77	48.65	40.10	68.55	43.45	52.88
FLAN	random	0.61	0.84	1.17	3.05	29.28	30.49	30.37	28.14	60.14	60.15	47.91	39.46	67.19	43.76	52.17
PES2O	random	0.45	0.66	2.72	5.49	29.37	30.84	31.48	26.78	60.32	56.67	47.26	38.12	66.27	43.04	53.20
FineMath	random	1.59	1.74	3.89	3.05	28.70	30.67	30.84	26.78	61.55	62.72	47.83	38.64	67.36	44.42	50.43
FineWeb-Edu-CN	q00	0.83	0.56	1.17	6.10	31.83	32.44	30.76	27.46	60.49	55.69	48.24	38.36	67.79	43.65	52.72
FineWeb-Edu-CN	q25	0.76	0.60	4.28	4.88	29.61	31.24	30.59	29.49	60.49	56.12	48.48	38.53	67.25	43.96	53.28
FineWeb-Edu-CN	q50	0.38	0.76	3.11	5.49	29.05	31.16	30.92	28.47	61.38	52.51	49.30	39.27	67.41	44.32	54.14
FineWeb-Edu-CN	q75	0.99	0.90	2.33	6.71	28.51	30.39	30.44	29.15	61.02	55.38	47.91	38.48	67.85	43.76	52.64
FineWeb-Edu-EN	q00	0.61	1.06	2.72	4.88	29.85	30.58	32.48	30.85	66.84	59.88	49.63	39.90	68.99	43.04	52.33
FineWeb-Edu-EN	q25	0.38	0.68	2.72	5.49	30.00	30.24	31.51	25.76	61.55	58.69	49.47	40.29	68.12	45.39	52.88
FineWeb-Edu-EN	q50	0.68	1.56	2.72	4.88	29.34	30.16	30.77	29.83	60.32	61.25	50.12	41.32	68.06	45.14	52.49
FineWeb-Edu-EN	q75	0.45	1.14	2.33	5.49	30.16	30.56	31.54	26.78	61.20	55.11	49.63	40.92	69.15	44.83	52.49
FineWiki-EN	random	0.99	0.70	2.72	6.71	30.17	30.37	29.98	28.14	63.49	61.68	48.32	38.77	66.59	44.63	51.62
MegaMath-Code	random	1.14	1.30	13.23	10.37	29.64	30.60	31.14	28.81	61.90	62.05	46.76	38.53	66.49	43.19	51.70
MegaMath-Web	random	0.99	0.62	3.11	2.44	29.68	30.18	30.73	28.47	62.96	50.61	47.67	38.30	66.49	43.30	51.30
MegaMath-Web-Pro	random	3.03	1.90	3.50	5.49	29.93	30.53	31.86	30.51	63.14	62.78	49.80	39.64	67.03	44.47	51.54
Nemotron-CC-Math	random	1.82	3.74	2.72	6.71	29.13	30.48	31.36	29.83	62.08	61.99	48.98	38.96	66.70	43.91	53.28
Nemotron HQ	random (medium-high-quality)	0.61	1.32	1.56	6.10	29.33	30.55	30.82	27.46	59.79	58.35	50.04	40.72	68.34	44.83	53.04
Nemotron HQ	random	0.53	1.24	2.33	6.71	29.17	30.45	30.80	31.53	62.26	60.21	48.08	41.31	68.17	43.30	52.80
Nemotron HQ Synthetic	random	0.38	1.12	5.06	6.71	30.60	30.74	31.22	27.80	63.67	60.40	47.50	40.94	68.93	43.81	52.25
Nemotron Synthetic Code	random	0.53	0.70	7.39	10.37	29.22	30.45	30.32	25.42	60.85	57.25	46.36	38.18	66.54	44.22	51.07
OpenWebMath	random	0.53	1.18	3.11	4.88	29.27	30.65	31.09	29.83	61.90	53.55	47.99	38.48	67.95	44.98	52.96
StackExchange	random	0.45	0.54	5.84	4.88	29.24	30.17	30.64	29.83	60.85	55.41	47.75	39.14	67.30	43.91	52.96
Stack-v2-Smol	random	0.91	1.10	5.84	4.88	30.23	30.07	30.13	29.49	59.08	62.42	45.78	37.87	67.14	43.24	50.51
StarCoder Tokens	random	0.68	0.50	6.61	4.27	29.68	29.92	30.89	27.80	61.55	62.23	45.86	38.11	67.30	42.94	53.75
Swallow-Code-v2	q00	0.61	0.90	8.95	10.37	28.20	30.71	30.55	28.14	60.49	60.40	47.99	38.83	67.30	43.40	52.64
Swallow-Code-v2	q25	0.30	0.82	9.34	9.15	28.95	30.65	30.50	28.14	58.91	59.66	48.48	38.57	66.59	42.84	52.17
Swallow-Code-v2	q75	0.68	0.42	8.95	9.76	29.71	30.74	31.13	26.10	61.55	60.89	46.11	38.63	66.70	42.99	51.70
SwallowMath-v2	random	1.21	4.60	0.39	7.32	30.26	30.18	30.80	27.12	61.38	62.11	47.34	38.60	67.57	43.76	51.07
Deduplicated merged Chinese web	q00	0.53	0.76	2.33	4.27	30.65	30.96	30.77	27.46	59.61	59.05	47.91	38.33	67.79	44.37	52.72
Deduplicated merged Chinese web	q50	0.83	0.68	3.89	4.88	30.40	30.65	30.74	26.10	59.79	57.43	47.91	38.73	67.19	44.47	51.85
Deduplicated merged Chinese web	q75	0.45	0.60	2.72	4.88	29.74	31.03	30.82	27.12	60.14	57.74	47.26	38.97	67.14	44.06	51.93

Table 24: Raw proxy benchmark aggregates and descending ranks by data slice. Dataset labels follow the canonical family names in Table 22; source qualifiers remain in the Slice column. Math, Code, and CN average 2 benchmarks each; General (the English knowledge and commonsense axis) averages 9 benchmarks; Overall averages all 15 raw benchmark scores. The ranks are computed over the 39 displayed non-duplicate slices; rank 1 is highest.

Source	Slice	Math avg	Math rank	Code avg	Code rank	CN avg	CN rank	General avg	General rank	Overall avg	Overall rank
MegaMath-Code	random	1.22	5	11.80	1	30.12	17	47.84	17	34.46	1
MegaMath-Web-Pro	random	2.46	3	4.50	18	30.23	11	48.97	3	34.34	2
DCLM-Dedup	q00	0.94	14	3.42	35	30.21	14	49.54	1	34.34	3
FineWeb-Edu-EN	q00	0.83	19	3.80	29	30.21	14	49.33	2	34.24	4
Nemotron-CC-Math	random	2.78	2	4.71	13	29.80	30	48.57	7	34.11	5
Nemotron HQ Synthetic	random	0.75	24	5.88	6	30.67	4	48.50	8	34.08	6
Swallow-Code-v2	q00	0.76	22	9.66	2	29.45	38	47.75	21	33.97	7
AutoMathText	random	1.02	9	5.75	7	30.02	23	48.43	10	33.96	8
Nemotron HQ	random	0.89	15	4.52	16	29.81	29	48.72	5	33.93	9
FineWeb-Edu-EN	q50	1.12	7	3.80	29	29.75	34	48.81	4	33.91	10
DCLM-Dedup	q25	1.12	6	5.41	9	29.72	35	48.46	9	33.91	11
Swallow-Code-v2	q75	0.55	36	9.36	3	30.23	12	47.31	32	33.74	12
DCLM-Dedup	q50	0.63	30	3.22	37	30.09	21	48.58	6	33.67	13
Swallow-Code-v2	q25	0.56	34	9.24	4	29.80	32	47.32	31	33.67	14
FineWiki-EN	random	0.84	18	4.71	13	30.27	10	48.14	13	33.66	15
SwallowMath-v2	random	2.90	1	3.85	27	30.22	13	47.75	20	33.58	16
DCLM-Dedup	q75	1.03	8	4.11	22	30.73	3	47.96	14	33.56	17
FineWeb-Edu-EN	q25	0.53	37	4.11	22	30.12	18	48.18	11	33.54	18
Nemotron HQ	random (medium-high-quality)	0.96	11	3.83	28	29.94	25	48.15	12	33.52	19
StarCoder Tokens	random	0.59	32	5.44	8	29.80	31	47.83	18	33.47	20
FineWeb-Edu-EN	q75	0.80	21	3.91	26	30.36	9	47.96	15	33.45	21
Cosmopedia-v2	random	0.96	12	5.28	12	30.66	5	47.46	29	33.39	22
Algebraic Stack Train	random	0.67	28	4.30	20	30.07	22	47.81	19	33.36	23
FineMath	random	1.67	4	3.47	34	29.68	37	47.84	16	33.35	24
FineWeb-Edu-CN	q25	0.68	27	4.58	15	30.43	7	47.58	24	33.30	25
StackExchange	random	0.49	39	5.36	10	29.70	36	47.53	26	33.26	26
Nemotron Synthetic Code	random	0.61	31	8.88	5	29.84	28	46.69	38	33.26	27
Stack-v2-Smol	random	1.00	10	5.36	10	30.15	16	47.30	33	33.25	28
OpenWebMath	random	0.85	17	4.00	25	29.96	24	47.64	23	33.22	29
FineWeb-Edu-CN	q00	0.69	26	3.63	33	32.13	1	47.24	35	33.21	30
FineWeb-Edu-CN	q50	0.57	33	4.30	20	30.11	19	47.52	27	33.18	31
Deduplicated merged Chinese web	q00	0.65	29	3.30	36	30.80	2	47.56	25	33.17	32
FineWeb-Edu-CN	q75	0.94	13	4.52	16	29.45	39	47.40	30	33.10	33
ArXiv	random	0.86	16	3.72	32	29.80	33	47.51	28	33.09	34
Deduplicated merged Chinese web	q50	0.76	22	4.38	19	30.52	6	47.13	36	33.04	35
FLAN	random	0.72	25	2.11	39	29.89	27	47.70	22	32.98	36
Deduplicated merged Chinese web	q75	0.53	38	3.80	29	30.39	8	47.24	34	32.97	37
PES2O	random	0.56	35	4.11	22	30.11	19	47.02	37	32.84	38
MegaMath-Web	random	0.81	20	2.77	38	29.93	26	46.65	39	32.46	39